

Как работает TLS, в технических подробностях

 tls.dxdt.blog/tls.html

Введение

SSL и TLS представляют собой развитие одной и той же технологии. Аббревиатура TLS (Transport Layer Security) появилась в качестве замены обозначения SSL (Secure Sockets Layer) после того, как протокол окончательно стал интернет-стандартом. Такая замена вызвана юридическими аспектами, так как спецификация SSL изначально принадлежала компании Netscape. И сейчас нередко названия SSL и TLS продолжают использовать в качестве синонимов, но каноническим именем является TLS, а протоколы семейства SSL давно окончательно устарели и не должны использоваться.

TLS/SSL изначально разработали для защиты коммерческих транзакций, проводимых через Интернет. То есть, основной целью являлось получение относительно безопасного канала для осуществления покупок или управления банковским счётом - хотя, ни первое, ни второе ещё не обрели популярность у рядовых пользователей во времена становления SSL, поскольку на дворе была середина 90-х годов XX века. Зато в современном Интернете на TLS/SSL полагаются не только и не столько в деятельности "по продаже товаров", но и при решении гораздо более общей задачи сохранения "приватности" и конфиденциальности важной информации.

TLS используется и в современных мессенджерах, и в "мобильных приложениях", что бы под этим термином ни подразумевалось. Однако одним из самых распространённых применений TLS в Интернете остаётся HTTPS. HTTPS стремительно вытесняет незащищённую версию (HTTP) в Интернете: основной объём веб-трафика сейчас передаётся в зашифрованном виде, при этом и так небольшая доля открытого трафика продолжает уменьшаться. Таким образом, почти весь значимый веб-трафик сейчас зашифрован. А, например, в HTTP/2 использование TLS предписано протоколом по умолчанию.

Благодаря такому положению дел, TLS - один из самых изученных, исследованных протоколов современного Интернета. Вместе с тем, история этого протокола показывает, насколько мало бывает практического толка от исследований. Например, в 2010 году корпорация Microsoft исправила дефект в реализации SSL своего веб-сервера IIS - CVE-2010-3332; а соответствующая уязвимость была продемонстрирована в 2003 году. То есть, семь лет результаты исследования считались чистой теорией, не заслуживающей практического исправления. Впрочем, начиная, примерно, с 2014 года, интерес к поиску уязвимостей в TLS сильно возрос,

а обнаруженные уязвимости стали привлекать внимание "самых широких слоёв" интернет-пользователей. Благодаря этому процесс исправления дефектов приобрёл некоторую оперативность и непрерывность.

SSL (не TLS) внедрили впервые в качестве проприетарной технологии, реализованной в браузере Netscape Navigator, одном из первых веб-браузеров. Версия 1 протокола не была опубликована, а так и осталась внутренней разработкой Netscape, развивавшейся в 1994-95 годах. SSLv2 - следующую, вторую версию протокола - опубликовали, однако спецификация так и не вышла из состояния черновика (draft). Более того, хоть в SSLv2 и скорректировали отдельные дефекты и уязвимости v1, протокол оказался ненадёжным, содержащим серьёзные архитектурные огрехи. SSLv2 очень давно и без оговорок признан небезопасным, поэтому сейчас не должен использоваться. Если постараться, то в Сети всё ещё можно найти архаичные серверы, которые поддерживают SSLv2, поскольку вытеснение дефектных технологий идёт медленно. Однако, если ваш сервер или клиентское ПО всё ещё поддерживает SSLv2, можно смело сказать, что у вас просто нет безопасного канала, вы не поддерживаете TLS вообще. Более того, на практике было продемонстрировано, что наличие сервера с SSLv2, который использует те же ключи, что и современный TLS-сервер, ставит под угрозу сессии TLS (даже если это разные физические серверы, а трафик к ним доставляется независимо).

SSLv3 - это развитие SSLv2, но с весьма существенными доработками. SSLv3 представлен в 1996 году. Этот протокол получил полноценную спецификацию RFC, но значительно позже своего появления, и в статусе исторического документа: [RFC 6101](#). На время SSLv3 пришлось становление TLS как интернет-технологии. Именно на базе SSLv3 и появился протокол TLS.

Сейчас существует четыре версии TLS, все они описаны в RFC: [TLS 1.0](#), [TLS 1.1](#), [TLS 1.2](#) и вышедшая в 2018 году версия [TLS 1.3](#). TLS 1.0 во многом повторяет SSLv3, за исключением ряда деталей, которые, впрочем, являются весьма важными. (В криптографических протоколах детали важны как ни в каких других протоколах Интернета.) Ключевым отличием TLS от SSL является наличие поддержки целого ряда расширений, позволяющих реализовать современные методы защиты информации. TLS 1.1 и 1.2 достаточно близки к 1.0, но в версии 1.2 появились заметные улучшения, например, используется другая основа псевдослучайной функции и введена поддержка шифров в режиме аутентифицированного шифрования.

Несмотря на то, что номер следующей версии - 1.3 - получил лишь прибавку единицы в "младшем разряде", TLS 1.3 *весьма и весьма существенно* отличается от всех предыдущих версий TLS. В частности, радикально изменена логика установления соединения, а сам протокол стал архитектурно стройнее, благодаря тому, что из спецификации удалены многие сообщения (элементы протокола), а оставшиеся -

заметно переработаны. По причине фундаментальных различий версий до 1.3 и 1.3, в данном тексте устаревающие версии TLS 1.0, 1.1, 1.2 рассматриваются, по возможности, отдельно, при этом TLS 1.3 описывается даже более детально, чем предыдущие версии. В современном, "общемировом" Интернете TLS 1.3 быстро стал основной версией. Так, протокол версии 1.3 получил RFC в 2018 году, а уже в июле 2019 года поддержку TLS 1.3 для веба реализовали Google, Facebook и Cloudflare, а также основные ветки браузеров Chrome и Firefox, что закрыло большие объёмы веб-трафика. Уже в 2022 году поддержка версии TLS 1.3 стала обычным требованием к более или менее современному веб-сайту. Старые версии TLS остались в "промышленных" и "исторических" приложениях.

Рекомендованными для применения версиями являются TLS 1.3 и 1.2. В 2021 году вышел [RFC 8996](#), который переводит версии ниже 1.2 в разряд нерекомендуемых. При этом, исторически, TLS 1.0 повсеместно поддерживался много лет - данный протокол совместим даже с древним браузером IE 6. TLS 1.1 всё ещё представляет ценность, поскольку эта версия поддерживается практически всеми сколь-нибудь распространёнными программными сервисами веба, что важно для разработанных давно, но всё ещё используемых, приложений, которые трудно или даже невозможно обновить. Но, в том что касается веба, все современные браузеры поддерживают версии 1.2 и 1.3, так что какие-то проблемы при исключении версий 1.0, 1.1 могут возникнуть только с аномально старыми сборками и, очень редко, на стороне сервера, при взаимодействии с действительно древними приложениями (однако такими приложениями могут оказаться веб-интерфейсы роутеров или ещё какого-то оборудования, не обязательно сетевого).

TLS 1.0 можно назвать версией 3.1 SSL. Собственно, именно так и сделано в спецификациях: SSL 3.1 - это TLS 1.0; 3.2 - TLS 1.1; 3.3 - TLS 1.2; 3.4 - TLS 1.3. В шестнадцатеричной записи: 0x0301, 0x0302, 0x0303, 0x0304.

С 2015 года SSLv3, следом за публикацией очередных уязвимостей, перешёл в статус прямо *нерекомендуемого*. Этот статус вполне официальный, каким только официальным он может быть в рамках технологических традиций Интернета: в июне 2015 года выпущен документ [RFC 7568](#), требующий исключить SSLv3 из разряда поддерживаемых клиентами и серверами протоколов. Остались только версии TLS.

Самая современная версия TLS - 1.3. Активная работа в рамках IETF над новой версией TLS началась в 2014 году. Одной из причин разработки нового стандарта стал резко возросший интерес научно-технического сообщества к архитектуре TLS. Сыграло роль и обнаружение нескольких существенных дефектов в самом протоколе, что гораздо опаснее наличия дефектов в конкретных реализациях. С момента публикации RFC TLS 1.2 прошло почти шесть лет - за это время информационный ландшафт в Интернете успел сильно измениться. RFC TLS 1.3 опубликован в августе 2018 года, этому предшествовало большое количество

черновиков (draft). Интересно, что протокол был реализован на практике в нескольких черновых вариантах и поддерживался до публикации RFC, но, так как draft-версии отличались друг от друга в существенных моментах, такая поддержка требовала указания специальной версии протокола (с префиксом 0x7F, например: 0x7F1C - draft-28). Так, в браузере Chrome была реализована версия draft-17. Это же относится и к сервисам, поддерживавшим TLS 1.3 до выхода RFC. Спецификация draft-28 практически не отличалась от финального RFC (собственно, единственное значимое отличие касалось только указываемого номера версии: для RFC-версии нужно указывать 0x0304), поэтому соответствующие библиотеки могли быть быстро переведены на RFC-версию.

Версия TLS 1.3 не только несовместима с предыдущими, но и построена на другой инженерной идеологии. Различие обусловлено переосмыслением модели угроз, в рамках которой проектируется новый протокол: TLS попытались сделать и более быстрым (по крайней мере, потенциально), и более защищённым, и более скрытным. Что касается последнего аспекта, то речь идёт о совершенно новом для TLS направлении: минимизации утечек так называемой метаданных, то есть, данных о том, какие сообщения передаются внутри TLS-сессии, в каком состоянии сессия находится в данный момент времени. Существенные усилия были направлены на то, чтобы даже хорошо оснащённая третья сторона не могла узнать ничего более, кроме как о факте установления TLS-соединения.

Предназначение TLS

TLS ставит своей целью создание между двумя узлами сети защищённого от прослушивания и подмены информации канала связи, пригодного для передачи произвольных данных в обоих направлениях, а также проверку того, что обмен данными происходит между именно теми узлами, для которых канал и планировался. Эти задачи называются, соответственно, обеспечением конфиденциальности, целостности и подлинности соединения (аутентификации). Из фундаментальных задач защиты информации, TLS не охватывает только одну: обеспечение доступности информации - и эта задача находится далеко за рамками данного протокола.

Предполагается, что TLS работает поверх существующего между узлами "поточкового" соединения (с которым обычно связан "сокет" - откуда старое название SSL, Secure Sockets Layer). Сейчас, в большинстве случаев, TLS будет работать поверх TCP. Именно TCP отвечает за установление соединения, разбивку данных на пакеты, гарантированную доставку этих пакетов (при наличии соединения, естественно) и за разные другие телекоммуникационные моменты. TLS предполагает, что между узлами установлено надёжное соединение, поэтому, например, протокол не охватывает повторную отправку потерянных пакетов данных. (Для работы в условиях

негарантированной доставки и неустойчивой связи существует особый, родственный протокол - DTLS, который мы здесь не рассматриваем.) TLS использует установленный канал связи для передачи TLS-сообщений, одного из базовых "строительных блоков" протокола. Эти сообщения кардинальным образом отличаются от пакетов данных (IP-пакетов в TCP/IP) и представляют собой структуру более высокого уровня. TCP позволяет надёжно обмениваться данными, но содержимое пакетов (за исключением специальных случаев) может быть легко прочитано кем-то, кто имеет доступ к каналу связи. Хуже того, этот кто-то может заменить пакеты или изменить передаваемые в них данные. Именно для защиты от этих угроз и используется TLS.

Другими словами, модель угроз TLS предполагает, что атакующий может как угодно вмешиваться в канал связи, в том числе активно подменять пакеты и вообще - прерывать связь. Ключевые задачи TLS: 1) обеспечить конфиденциальность, то есть реализовать защиту от утечек передаваемой информации; 2) обеспечить обнаружение подмены, то есть реализовать сохранение целостности передаваемой информации; 3) обеспечить аутентификацию узлов, то есть дать механизм проверки подлинности источника сообщений. Кроме того, в TLS 1.3 существенное внимание уделено задаче сокрытия метаданных (это подзадача из первого пункта, обеспечения конфиденциальности), под метаданными понимается совокупность таких сведений о TLS-соединении, которые позволяют косвенно судить о передаваемых в защищённом режиме данных. Например, к метаданным относятся сведения об открытых криптографических ключах узлов, о времени соединения, об адресах, именах узлов и так далее.

TLS *как-то* справляется с описанными только что задачами, но не следует полагать, что TLS решает их *полностью*. Такое мнение является *большой ошибкой, к сожалению, весьма распространённой*. Доказательством того, что TLS не решает данные задачи полностью, являются уязвимости, обнаруженные в самом протоколе (а не только в его реализациях). Протокол и его реализации пытаются решить описанные задачи на максимально доступном уровне надёжности. Однако TLS в целом не обладает *доказанной стойкостью*, как не обладают ей и многие важнейшие составляющие части протокола. Обычно, в качестве *экстремального* примера данного наблюдения приводят такую рекомендацию: не следует доверять TLS и связанным технологиям свою жизнь или жизнь других людей.

TLS использует концепцию "клиент-сервер", которая, в логике данного протокола, во многом совпадает с логикой "клиент-сервер" TCP: например, и там, и там соединение инициирует клиент.

Логика TLS

TLS работает с записями (records). Записи находятся в фундаменте протокола. Это нижний транспортный уровень TLS. Сообщения TLS, относящиеся к верхним уровням, могут быть разбиты на несколько записей. Это достаточно важный момент: *в некоторых случаях TLS-сообщения требуют сборки из нескольких записей, что существенно влияет на логику обработки состояний протокола.*

Если рассматривать сеанс TLS на уровне условного сокета (TCP), то каждая передаваемая TLS-запись представляет собой блок, состоящий из короткого заголовка и, собственно, самих данных. Обратите внимание, что в TLS используется сетевой порядок байтов (старший байт идёт первым, слева направо - "тупоконечное" представление).

В TLS, за некоторыми исключениями, используется формат, в котором представление структуры данных начинается с заголовка, обозначающего тип данных и их длину. Части этого заголовка, определяющие структуру, *всегда определены строго*: то есть, определено количество байтов, выделенных для записи типа, перечислены все допустимые значения этих байтов и способ их интерпретации; определено количество байтов для записи длины полезной нагрузки, способ интерпретации записи. Так, сообщения, внутри которых невозможно отказаться от полей переменной длины, начинаются с заголовка, содержащего запись с количеством байтов, составляющих сообщение. При этом, в определённых спецификацией случаях, запись типа может быть пустой (то есть, тип не указывается вовсе, байты для его записи не резервируются). Для некоторых блоков данных может не указываться длина, но *это всегда означает*, что полезные данные имеют строго зафиксированную спецификацией длину в байтах - например, два байта для обозначения версии (см. ниже). Такой подход, который немного напоминает префиксные коды, позволяет избежать многих проблем при обработке данных. Так, сразу исключается целый класс "инъекций", основанный на дополнении пакетов. Естественно, изменить запись, добавив что-то в конец - возможно, но придётся скорректировать и значение длины пакета, которое *обрабатывается раньше полезной нагрузки*. В целом, строгое определение полей при помощи заголовков фиксированного формата - это именно тот способ задания структур данных, который нужно использовать в протоколах безопасности.

Разберём заголовок TLS-записи. Заголовок имеет длину 5 байтов, и следующий формат:

00	Тип (1 байт)
01	Версия протокола (2 байта)
03	Длина данных (2 байта)

Тип - это тип записи (фактически, тип определяет то, как будет интерпретироваться содержимое записи). Определено четыре типа: **20 (0x14)** - сообщение Change Cipher Spec (CCS), в версии 1.3 данное сообщение передаётся только как "фиктивное", не имеет логической роли в самом протоколе, но может повлиять на процесс обработки соединения в конкретной реализации TLS; в более ранних версиях - CCS играет важную роль, обозначая момент перехода к защищённому обмену сообщениями; **21 (0x15)** - сообщение Alert (это не обязательно предупреждение, есть вполне "фатальные alert-ы"); **22 (0x16)** - сообщение Handshake (начало установления соединения); **23 (0x17)** - Application Data (запись содержит данные приложения - то есть, полезную нагрузку); трактовка типа Application Data существенно различается в версии 1.3 и предыдущих версиях.

При передаче информации основные преобразования с шифрами и кодами аутентификации как раз происходят в блоке данных записи с типом 23 (0x17) Application Data. В TLS 1.3 зашифрование информации узлами начинается существенно раньше, чем в предшествующих версиях: зашифрованы уже сообщения Handshake (тип 0x16). До версии 1.3 - записи, отличные от Application Data, обычно передаются в открытом виде, но, тем не менее, тоже могут быть зашифрованы, в зависимости от текущего состояния TLS-соединения.

Версия протокола - это номер версии, записанный в двух байтах, как указано выше: 03 00 - это SSLv3, 03 01 - TLS 1.0; 03 02 - TLS 1.1; 03 03 - TLS 1.2; *TLS 1.3 в этом поле предписывает использовать значение 03 03, то есть, фиктивную версию 1.2, само поле при этом игнорируется.*

Длина данных - количество байтов, содержащих данные (сами байты данных следуют после заголовка). Так как используется сетевой порядок байтов, для определения длины нужно значение первого слева байта (h) умножить на 2^8 и прибавить значение второго (младшего, l) байта, вот так: $h * 256 + l$. Пустой блок данных (длина нуль) допускается протоколом, но иногда вызывает сбои в клиентах, написанных с ошибками. Максимальная допустимая длина данных записи в TLS 1.3 - 2^{14} байтов.

Пример заголовка записи TLS 1.1 (значения байтов - в шестнадцатеричной записи):

16 03 02 03 FE

Раскодируем: это сообщение Handshake (0x16); версия 3.2 (0x03,0x02)- то есть TLS 1.1; длина данных - 0x03FE байтов (1022 байта).

Заголовок всегда передаётся в открытом виде. В том числе, при использовании TLS 1.3. Однако в TLS 1.3 предусмотрено сокрытие реального типа записи (для защищённых записей) - подробнее этот момент разобран [в отдельном подразделе](#).

За заголовком должно следовать определённое число байтов (≥ 0), которые представляют собой содержимое данной записи. Максимальное значение в версиях до 1.3 - 18432 байта. $18432 = 16384 + 2048$. То есть, 16 килобайт + два раза по килобайту. Такое значение может показаться странным. Действительно, "базовая" версия записи, описанная в спецификации, предполагает блок данных в 2^{14} байтов, то есть 16 килобайт. Но такой размер относится только к записям, содержащим открытый текст (TLSPlaintext). Например, к сообщениям, управляющим установлением соединения (Handshake). Здесь, действительно, максимальная длина - 16К.

Для защищённых записей ситуация меняется. Во-первых, в TLS до версии 1.3 было возможно использование сжатия данных. Под "сжатием" здесь имеется в виду использование кодирования, минимизирующего количество битов, необходимых для записи данных - тот же процесс, который многим знаком по программам-архиваторам. Собственно, спецификация TLS предписывала, что сжиматься должны все данные защищённой записи. Но на практике это довольно давно означает не совсем то, о чём можно подумать (*это нередкая для TLS ситуация*): на практике, сжатие не используется (или, если хотите, используется только "тривиальное сжатие"). Дело в том, что TLS всех версий среди перечня алгоритмов сжатия содержит алгоритм null, означающий, что никакого сжатия в реальности не производится, а данные кодируются "как есть". Этот алгоритм служит вариантом по умолчанию (раньше можно было встретить алгоритм DEFLATE, но он давно относится к нерекомендованным из-за обнаруженных уязвимостей). Тем не менее, реально сжатые данные могут, в ряде случаев, оказаться длиннее, из-за особенностей алгоритма сжатия (это выглядит контринтуитивным, но для любого практического алгоритма сжатия можно подобрать входные данные таким образом, что результат кодирования "со сжатием" окажется длиннее исходных данных). TLS отводит на такое увеличение длины 1024 байта (хорошо реализованные алгоритмы используют значительно меньше). В TLS 1.3 сжатие прямо запрещено спецификацией, поэтому данное "дополнение" не используется.

Во-вторых, для проверки целостности зашифрованных данных к ним приписывается код аутентификации сообщения (MAC - значение, позволяющее проверить, что сообщение не было изменено; эти коды подробно [описаны ниже](#)) и дополнительная информация (вроде векторов инициализации шифров) - на это в TLS старых версий отводится ещё 1024 байта. Итого - 2048 байтов, пусть данный момент и вносит некоторую сумятицу в организацию протокола. Естественно, универсальные - и, потому, фиксированные, - 16К в качестве максимального значения смотрелись бы лучше. В реальности, не каждая запись TLS дотягивает и до 16К - обычно длина заметно меньше. Однако эта ситуация меняется в связи с введением постквантовых криптосистем: ключи и сопроводительная информация в этих криптосистемах часто требуют многих килобайтов для записи, настолько многих, что это прямо приводит к

превышению максимального значения длины TLS-записи и, соответственно, к фрагментации TLS-сообщений.

В TLS 1.3 ситуацию с определением длины TLS-записей упростили: для защищённых данных дополнительно допускается только 256 байтов для кода аутентификации, сжатие запрещено. То есть, максимальный размер: $16384 + 256 = 16640$. При этом в 16384 байта входят байты дополнения (record padding) - байты со значением 0, которые могут приписываться в конец блока защищаемых данных для того, чтобы скрыть длину полезного сообщения, а также для выравнивания размера записи.

Код аутентификации сообщения - важнейший элемент защиты информации в TLS. В версиях до 1.3 обычно используется HMAC - алгоритм вычисления кода аутентификации, базирующийся на той или иной хеш-функции; типичный выбор: SHA-1 или SHA-256. Код аутентификации приписывается к блоку данных. Один из архитектурных дефектов версий TLS до 1.3 состоит в том, что соответствующие спецификации предписывают сперва вычислять MAC, а потом зашифровывать сообщение. То есть, вычисленный код аутентификации присоединяется к открытому тексту сообщения, а потом всё вместе зашифровывается. При этом принимающая сторона должна сперва расшифровать полученные данные, а потом проверить MAC. Такой метод легко приводит к возникновению "криптографических оракулов", на использовании которых основано несколько эффективных атак против TLS (эти атаки подробно рассмотрены в [специальном разделе](#)).

Современный подход: код аутентификации добавляется после зашифрования открытого текста, то есть MAC вычисляется для уже зашифрованного сообщения и прикрепляется к нему ("сперва шифр, потом - MAC"). Для внедрения современного подхода в спецификации добавлено специальное расширение, позволяющее клиенту и серверу договориться о том, в какой последовательности они генерируют коды аутентификации - [RFC 7366](#). Современная версия TLS 1.3 использует шифры только в режиме аутентифицированного шифрования (а именно - AEAD). Этот режим не нуждается в отдельном MAC, так как целостность данных гарантирует сам алгоритм зашифрования (подробное описание [дано ниже](#), в разделе, посвящённом шифрам). Пример: AES в режиме GCM, этот вариант является предпочтительным, поддерживается всем современным ПО, и имеет большое распространение.

Используемые криптосистемы в TLS объединяются в типовые шифронаборы (Cipher Suites, иногда используется русскоязычный термин "криптонаборы"). Чтобы начать обмен информацией по защищённому каналу, клиент и сервер должны согласовать используемый шифронабор. Очевидно, что параметры шифров и обмена сопутствующей информацией должны быть совместимы между собой. Согласование проводится на этапе установления соединения (Handshake). Композиции шифронаборов существенно отличаются в TLS 1.3 и в предшествующих версиях. Так, в TLS 1.2 и раньше в шифронабор входят:

- криптосистема, используемая для аутентификации (аутентифицируются сервер и значение сеансового секрета);
- шифр, который служит для защиты передаваемых полезных данных в симметричном режиме;
- хеш-функция, являющаяся основой для HMAC.

В TLS 1.3 криптосистемы, служащие для аутентификации узлов и получения общего секрета, отделены от, собственно, шифров, что потребовало изменения организации реестра и механизма нумерации шифронаборов (а каждый из них обозначается двухбайтовым индексом). Кроме этих индексов существуют стандартные обозначения (символьные имена) для шифронаборов. Например, выбор в TLS 1.1 типового шифронабора TLS_RSA_WITH_AES_128_CBC_SHA означает, что будет использована криптосистема RSA для передачи сеансового ключа, AES со 128-битным ключом в режиме CBC в качестве симметричного шифра (то есть, защищаемые данные приложения будут зашифрованы симметричным шифром AES), SHA-1 в качестве хеш-функции HMAC. Для TLS 1.3 ситуация отличается. Шифронаборов здесь меньше, а их имена, соответственно, содержат только указания на шифр и хеш-функцию, например: TLS_AES_256_GCM_SHA384 - шифр AES в режиме GCM и хеш-функция SHA-384. Шифронаборы строго фиксированы, состав закреплён в RFC, каждому приписан свой индекс, а [реестр ведёт IANA](#).

Шифронаборы имеют важнейшее значение для безопасности TLS. Выбор нестойкой комбинации означает, что достаточной защиты конкретная реализация TLS не обеспечивает. Поэтому в TLS 1.3 число допустимых шифронаборов резко сокращено - спецификация содержит только пять шифронаборов, а изначально, в 2019 году, рекомендовано было лишь четыре. Применительно к вебу это означает вот что: браузеры используют встроенный комплект шифронаборов, обычно это 10-15 вариантов, которые могут использоваться с разными версиями TLS; на сервере поддерживаемые шифронаборы настраиваются администратором сервера. Реестр IANA в настоящий момент содержит свыше 300 (трёх сотен, да) шифронаборов (в число которых входят и так называемые "псевдошифры", которые используются в качестве сигналов). Среди шифронаборов встречается экзотика вроде TLS_KRB5_WITH_RC4_128_SHA и безнадёжно устаревшие варианты, например - TLS_RSA_EXPORT_WITH_RC4_40_MD5. Современные серверы и клиенты не должны использовать подобного. Наличие того или иного шифронабора в реестре IANA означает, что этому шифронабору присвоен индекс (номер) и буквенное обозначение, не более того: нужно понимать, что данный реестр не имеет *никакого отношения* к тому, какие шифры и криптосистемы рекомендованы или не рекомендованы для использования.

Добротным вариантом сейчас считается, как минимум, TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA256, допустимый для TLS 1.2: то есть, генерация общего секрета проводится по протоколу Диффи-Хеллмана на

эллиптических кривых, для аутентификации данных и параметров на этапе установления соединения используется криптосистема электронной подписи ECDSA (тоже работающая на эллиптических кривых), полезная нагрузка зашифровывается AES с 256-битным ключом в режиме GCM, а в качестве хеш-функции служит SHA-256 (здесь используется при генерации сеансовых ключей). Обратите внимание, что постквантовая стойкость (то есть, использование криптосистем, стойких к гипотетическим атакам на квантовом компьютере), на практике возможна только для TLS 1.3, где шифры и криптосистемы получения симметричного секрета *разделены*. Так как постквантовая стойкость сейчас (2025) обеспечивается только при получении симметричного секрета, то и ссылку на постквантовую криптосистему нужно искать, что называется, "в другом разделе": то есть, эта криптосистема (обычно, ML-KEM), не указывается в названии шифронабора для TLS 1.3. (Но в будущем схема именования может поменяться.) Постквантовая стойкость будет рассмотрена отдельно ниже.

Первоначальное соединение (Handshake)

(TLS Handshake иногда в современных русскоязычных статьях называют "рукопожатием", что является прямым, и не очень удачным, переводом английского термина; этот вариант мы не станем использовать.)

Клиент и сервер должны договориться об используемых шифрах и методах аутентификации, согласовать ключи и другие параметры сеанса связи. Набор согласованных параметров называется криптографическим *контекстом*. Согласование контекста происходит при установлении соединения, путём обмена специальными сообщениями - Handshake-сообщениями. Такой обмен в TLS осуществляется *поверх* обмена записями. Каждое Handshake-сообщение содержит специальный заголовок, состоящий из четырёх байтов. Первый байт обозначает код типа сообщения, три следующих байта - длину сообщения.

В TLS 1.3 установление соединения организовано таким образом, что уже на втором шаге используется защищённый режим. Поэтому разбор Handshake версии 1.3 требует, как минимум, использования протокола Диффи-Хеллмана и симметричных шифров. К TLS 1.2 и раньше - этот момент не относится. Поэтому мы сначала подробно разберём установление соединения в 1.2, потом рассмотрим протокол Диффи-Хеллмана (применительно к TLS) и симметричные шифры, после этого перейдём к анализу Handshake версии 1.3. (Заметьте, что для лучшего понимания принципов построения TLS 1.3 - необходимо понимание версии 1.2, особенно с учётом того, что 1.3 отличается весьма существенно: увидев, от каких моментов отказались, можно сделать весьма полезные выводы об архитектурных особенностях практических криптографических протоколов.)

Сообщения Handshake TLS 1.2 (и ранее)

Handshake-сообщения отправляются в виде набора TLS-записей, которые мы обсудили выше. Это означает, что заголовку сообщения предшествует заголовок TLS-записи, где тип сообщения определён кодом 22 (0x16) - см. выше. Несколько Handshake-сообщений могут быть переданы в одной записи - это распространённый случай. А самое интересное, что одно сообщение может быть разбито на несколько записей, передаваемых последовательно - спецификация допускает и это, например, можно "нарезать" сообщения на записи длиной в один байт полезной нагрузки каждое. К счастью, последний вариант скорее теоретический, на практике встречается исчезающе редко. Все эти случаи должны корректно поддерживаться добротной реализацией TLS. Таким образом, находящийся ниже протокола установления соединения уровень TLS-записей нужно рассматривать как некий потоковый псевдосокет, куда сообщения записываются как есть, и где они могут быть разрезаны на блоки для передачи. Единственное, что запрещает спецификация TLS - это нарушение порядка при доставке записей. Всё это верно и для TLS 1.3.

Спецификация предполагает, что любой клиент и сервер TLS по умолчанию уже согласовали шифронабор null, без MAC и с "нулевым" сжатием. То есть клиент и сервер могут обмениваться сообщениями в виде открытого текста без аутентификации. Установление соединения всегда начинается клиентом. В типичной конфигурации, TLS здесь оказывается схож с TCP, так как, например, при работе с сервером по HTTPS - TCP-соединение так же иницирует клиент. Благодаря историческому совпадению принципов построения технологий, клиент HTTPS, TLS, TCP - один и тот же узел. Можно предложить конфигурации протоколов, когда это не так, но в наиболее распространённой ситуации веба - подобные конфигурации едва ли возможны.

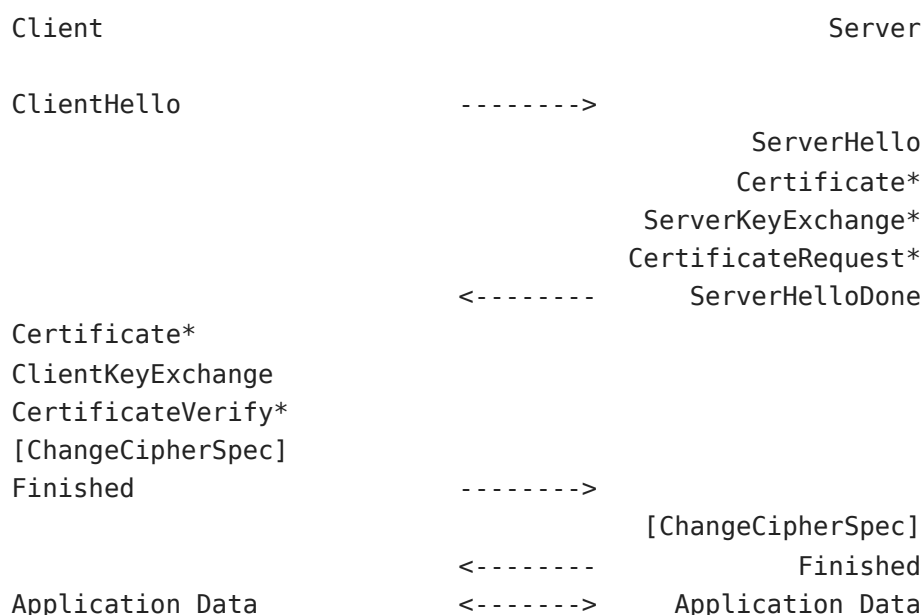


Схема обмена Handshake-сообщениями TLS 1.2

(ChangeCipherSpec - не является сообщением Handshake).

Источник: RFC 5246

Первым сообщением в протоколе установления TLS-соединения всегда является сообщение ClientHello (тип 01). Сообщение содержит следующие данные:

1. версия протокола - максимальная версия, которую готов поддерживать клиент (обратите ещё раз внимание - речь в этом разделе идёт о протоколе версии 1.2 или более ранней; в 1.3 это поле присутствует, но согласование версии выполняется иначе);
2. 32 байта случайных значений - ClientRandom. Изначально, в спецификации рекомендовалось использовать первые 4 байта для передачи UNIX-таймстемпа, а оставшиеся 28 - заполнять результатом работы криптографического генератора псевдослучайных чисел. Однако сейчас многие браузеры и веб-серверы генерируют все 32-байта случайным образом. Забегая чуть вперёд: предполагалось, что наличие таймстемпа в handshake-сообщениях может помочь при обнаружении проблем с подменой времени на том или ином узле. Однако данный метод сейчас никак не используется, поэтому часто все байты ClientRandom случайны. Такой подход закреплён и в TLS 1.3;
3. идентификатор TLS-сессии - SessionID: TLS позволяет возобновлять ранее установленные сессии, используя сокращённый вариант протокола установления соединения. Идентификатор сессии как раз содержит номер такой сессии, параметры которой (возможно) сохранены на сервере (подробнее мы рассмотрим этот вариант ниже). В TLS 1.3 данное поле может быть пустым, а если присутствует, то играет другую роль: обнаружив в ClientHello поле SessionID, сервер включает его значение в свой ответ без изменений, *при этом, SessionID в 1.3 не используется в качестве номера сессии, но может служить инструментом имитации сессий версии 1.2*;
4. список шифронаборов, которые поддерживает клиент - Cipher Suites. Порядок шифронаборов в списке отражает их степень предпочтения клиентом (предпочтительные передаются первыми слева), этот порядок - всего лишь рекомендация, и сервер далеко не всегда ей следует;
5. список поддерживаемых методов сжатия - Compression Methods, порядок, опять же, соответствует степени предпочтения, но обычно в этом поле лишь одно значение - null, так как сжатие не рекомендуется использовать. В TLS 1.3 - сжатие прямо запрещено, но, по "историческим причинам", данное поле сохраняется, с фиксированным значением null;
6. данные, определяющие параметры различных расширений протокола установления соединения (расширения ClientHello).

Каждое из полей в сообщении ClientHello имеет свой формат, а полю предшествуют данные о его длине и, в случае расширений, дополнительный заголовок. Заголовок обычно содержит тип расширения и длину его данных. Такая структура позволяет

реализациям корректно разбирать сообщения. Например, список шифронаборов может быть представлен в следующем виде (шестнадцатеричная запись):

```
[...] 00 06 c0 2b c0 2f c0 29
```

00 06 - шесть байтов занимают идентификаторы шифронаборов;

c0 2b - шифронабор 0xc02b - TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256;

c0 2f - шифронабор 0xc02f - TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256;

c0 29 - шифронабор 0xc029 - TLS_ECDH_RSA_WITH_AES_128_CBC_SHA256.

Расширения ClientHello позволяют использовать различные нововведения, которые появились позже, чем структура данного сообщения. Так, например, в расширениях могут быть перечислены эллиптические кривые, которые готов использовать клиент. Самым распространённым расширением является SNI (Server Name Indication), позволяющее браузеру (или другому TLS-клиенту) указать имя сервера, с которым он пытается установить соединение. SNI необходимо, например, для того, чтобы на одном IP-адресе могли корректно откликаться по HTTPS веб-серверы, размещённые под разными именами хостов (доменами).

Разберём ClientHello в деталях. В качестве примера возьмём специальный образец ClientHello, сгенерированный утилитой опроса TLS-серверов в исследовательских целях. В качестве имени сервера используется sbrf.ru.

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000 16		; Первый байт заголовка TLS-записи - тип 22, сообщение Handshake.
0001 03 02 00 65		; Версия TLS 1.1 (03 02), длина пакета данных - 0065 (101 байт).
0005 01		; Первый байт сообщения TLS: 01 - тип ClientHello.
0006 00 00 61		; Длина блока данных - 000061 (97 байтов).
0009 03 02		; Версия TLS 1.1 (03 02) клиента.
0011 53 53 4c 20 53		; 32 байта случайных значений, ClientRandom
[...]		
0043 00		; Длина поля SessionID - 0 байтов, так как SessionID в данном сообщении
		не используется (в случае браузерного ClientHello будет
указана длина		записи SessionID, за которой последует значение

SessionID).

0044 00 16 ; Длина поля со списком шифронаборов (Cipher Suites) - 22
байта: в данном ClientHello указано 11 шифронаборов, по два байта на
каждый из них.

0046 c0 2b ; Шифронаборы.

[...]

0068 01 00 ; Длина поля CompressionMethods (1 байт) и значение этого
поля (00 - null, без сжатия: клиент не поддерживает сжатие).

0070 00 22 ; Длина поля Extensions (0022 - 34 байта), поле, в нашем
случае, содержит расширения: SNI, Elliptic Curves, Elliptic Curves Point
Formats.

0072 00 00 ; Расширение SNI (Server Name Indication, тип 0000). Данное
расширение позволяет указать имя сервера, с которым будет
происходить соединение по протоколу HTTPS (в данном случае - HTTPS, но SNI может
использоваться и для других протоколов).

0073 00 0c ; Длина данных в расширении SNI.

0075 00 0a 00 00 07 ; Заголовок расширения SNI - длина списка имён (000a - 10
байтов), тип имени (hostname - 00), длина строки с именем (7 байтов).

0080 73 62 72 66 2e 72 75 ; Значение имени SNI: sbrf.ru

0086 00 0a ; Тип расширения - Elliptic Curves (000a), данное
расширение позволяет клиенту указать список эллиптических кривых, с которыми
этот клиент умеет работать.

0088 00 08 ; Длина данных расширения (8 байтов).

0090 00 06 ; Длина списка кривых (6 байтов - три кривые, по два байта
на идентификатор).

0092 00 17 00 18 00 19 ; Идентификаторы кривых.

0098 00 0b 00 02 01 00 ; Расширение Elliptic Curves Point Format, тип
(000b), длина

полный (0002), длина списка (01), формат записи точек -
 в аффинном (то есть, полностью передаются две координаты x,y
 представлении; есть форматы со сжатием, где
 передаётся только одна координата, x, а для второй указывается
 знак).

Согласно спецификации, после отправки ClientHello, клиент ожидает ответа сервера. В ответ может прийти либо сообщение об ошибке, в виде Alert, либо сообщение ServerHello.

Alert - короткие сообщения, содержащие информацию об уровне ошибки и о её типе. Сообщение Alert, например, может сигнализировать о сбое установления соединения. Рассмотрим такой вариант подробнее:

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	15	; Тип сообщения - 0x15 - Alert.
0001	03 01	; Версия протокола - 0x0301 - TLS 1.0.
0003	00 02	; Длина сообщения (два байта).
0005	02	; Уровень ошибки - 0x02 - Fatal (фатальная, соединение невозможно).
0006	28	; Состав ошибки - 0x28 - Handshake Failure (сбой установления соединения).

Если же сервер смог успешно обработать ClientHello, то он отвечает сообщением ServerHello. Это сообщение начинается с заголовка из четырёх байтов: тип сообщения (один байт) и длина (три байта). ServerHello (*мы пока рассматриваем версии до 1.3*) содержит следующие поля:

1. версию протокола, которую будут использовать клиент и сервер;
2. 32 байта случайных значений - ServerRandom. С этой строкой ситуация такая же, как и с Client Random: первые четыре байта могут быть таймстемпом, а могут и не быть. Интересным свойством данной метки времени является то, что она может послужить некоторым справочным материалом при последующем анализе записанного трафика: если сервер отвечает достоверным значением времени, то, учитывая, что полный набор сообщений Handshake содержит сообщение, криптографически удостоверяющее целостность данных (см. ниже про сообщение Finished), получаем подписанную сервером квитанцию о времени соединения по его часам, с точностью до секунды - иногда эти данные помогают правильно составить запрос в адрес администрации сервера об извлечении записей из логов;
3. идентификатор сессии - SessionID, присвоенный новой сессии сервером (в случае TLS 1.3 - данное поле обязательно повторяет SessionID клиента);

4. выбранный сервером шифронабор - Cipher Suite, этот шифронабор будет использоваться в дальнейшем и клиентом, и сервером. Сервер выбирает шифронабор из предложенных клиентом в ClientHello, но не обязательно следует приоритету клиента: распространена как раз обратная ситуация, когда сервер исключительно сам определяет предпочтительный шифронабор;
5. выбранный сервером метод сжатия - скорее всего, это null;
6. некоторый набор расширений.

В современных реализациях TLS раздел "Расширения" (Extensions) имеет очень большое значение, так как в нём передаются параметры, определяющие схему работы TLS-соединения. Предназначение ServerHello - согласовать параметры этого соединения. TLS здесь использует хорошо известную схему "запрос-ответ-подтверждение" (за исключением варианта ускоренного установления соединения), а сообщения ClientHello и ServerHello представляют собой, соответственно, запрос и ответ. Эти сообщения не содержат никакой секретной информации, а в версиях протокола до 1.3 передаются в открытом виде (в TLS 1.3 ситуация поменялась - практически сразу же сообщения Handshake передаются в зашифрованном виде; тем не менее, непосредственно ServerHello и ряд его расширений всё равно открыты). Данные сообщения, особенно - ClientHello, повсеместно используются в системах DPI для обнаружения факта установления (или попытки установления) TLS-соединения.

В рамках каждой TLS-сессии клиент и сервер согласовывают следующие параметры (RFC 5246):

1. роли узлов: какой узел является клиентом, а какой - сервером. Распределение этих ролей происходит "естественным образом", при установлении соединения - клиент всегда начинает сессию;
2. алгоритм PRF (псевдослучайная функция, PseudoRandom Function) - используется для вычисления сеансового ключа на основе данных, переданных в ClientHello и ServerHello; клиент и сервер должны использовать общий, согласованный алгоритм PRF;
3. шифр - данные в TLS зашифровываются симметричным алгоритмом; для успешного зашифрования потока данных внутри сессии оба узла должны использовать один и тот же алгоритм, в согласованном режиме. Узлы согласуют тип шифра (поточный, блочный), сам шифр, режим использования шифра, размер ключа, размер блока (для блочных шифров), инициализирующие векторы, а также дополнительные параметры (если они требуются, например, в случае с AEAD-режимами);
4. алгоритм вычисления кода аутентификации сообщения (MAC) - в рамках сессии необходимо использовать согласованный алгоритм аутентификации. Для AEAD-режимов отдельно MAC не используется (поскольку аутентификация состава данных реализуется AEAD-алгоритмом непосредственно);

5. алгоритм сжатия - также должен быть общим, однако сжатие, фактически, не используется, что означает общий алгоритм null;
6. основной секрет (MasterSecret) - массив из 48 секретных байтов, известный серверу и клиенту. Основное предназначение данного массива - определение сеансового ключа. 384-битное значение MasterSecret позволяет вычислить секретные ключи сессии. Если это значение стало известно третьей стороне, то она сможет расшифровать трафик сессии. В TLS 1.3 используется более сложная схема, состоящая из секретов разных уровней, которые соответствуют стадиям работы протокола;
7. случайные данные клиента (ClientRandom) - 32 байта случайных значений, передаются в ClientHello;
8. случайные данные сервера (ServerRandom) - 32 байта случайных значений, передаются в ServerHello.

Эти параметры позволяют построить контекст для работы криптосистем, которые будут обрабатывать защищённые TLS-записи на стороне сервера и клиента. Помимо ClientHello и ServerHello в ходе установления соединения узлы обмениваются несколькими другими сообщениями.

Со стороны сервера это следующие сообщения (конкретный состав и содержание - зависят от выбранного сервером режима работы).

Certificate. Ключевым аспектом для современных реализаций TLS является использование TLS-сертификатов (ранее они назывались SSL-сертификатами, сейчас рекомендуется вариант с TLS), мы рассмотрим их подробнее ниже, в специальном разделе. Сертификаты передаются сервером в сообщении Certificate. Это сообщение присутствует практически всегда. Серверный сертификат содержит открытый ключ сервера. В теории, можно установить TLS-соединение без отправки серверных сертификатов. Это так называемый "анонимный" режим, однако он не используется на практике, как и режимы TLS без шифров. Так, свыше 99% веб-серверов, поддерживающих TLS, передают TLS-сертификаты. В типичной конфигурации, сообщение Certificate будет включать несколько TLS-сертификатов, среди которых один серверный сертификат и так называемые "промежуточные сертификаты", позволяющие клиенту выстроить цепочку валидации. Серверный сертификат - это сертификат, который по имени соответствует серверу и содержит серверный открытый ключ электронной подписи (в большинстве случаев сейчас (2025) - RSA или ECDSA). Спецификация предписывает строгий порядок следования сертификатов в сообщении: первым должен идти серверный сертификат, а последующие - в порядке удостоверения предыдущего. Однако на практике серверы нередко отправляют сертификаты в произвольном порядке. На клиентской стороне браузеры тоже достаточно свободно относятся к порядку сертификатов, пытаясь выстроить из них корректную цепочку, применяя различные перестановки. Такое положение дел прямо нарушает спецификацию TLS версий 1.0, 1.1, 1.2, но такова

реальность: игнорируя криптографические строгости - реализации допускают вольности (последние нередко ведут к уязвимостям). Хотя, достаточно сложно найти веские причины для строго порядка следования сертификатов. В TLS 1.3 данное требование смягчили, сделав допустимым произвольный порядок промежуточных сертификатов (серверный всё равно ожидается первым).

ServerKeyExchange. Сообщение, содержащее серверную часть данных, необходимых для вычисления общего сеансового ключа. Сообщение может отсутствовать, а в TLS 1.3 оно не используется, так как новая схема Handshake для обмена криптографическими параметрами основана на расширениях Hello-сообщений. В версии TLS 1.2 и ранее ServerKeyExchange обычно содержит параметры протокола Диффи-Хеллмана (DH), однако исторический вариант предусматривает и передачу временного ключа RSA. Если используется классический вариант DH, то в сообщении ServerKeyExchange передаётся значение модуля (простое число) и серверный открытый ключ. В варианте на эллиптических кривых (ECDH) - идентификатор самой кривой и, аналогично DH, открытый ключ сервера. Параметры подписываются сервером (за исключением экзотических вариантов обмена, вроде анонимного DH), клиент может проверить подпись при помощи открытого ключа сервера из TLS-сертификата. *Этот шаг позволяет клиенту убедиться, что параметры получены именно от того сервера, который имеет доступ к секретному ключу от серверного сертификата.* В зависимости от используемой криптосистемы, подпись может быть RSA или ECDSA, другие варианты - редки. Подпись на параметрах DH очень важна, так как позволяет защитить соединение от атаки типа "человек посередине". В разных версиях TLS подпись вычисляется для разных наборов данных. В TLS 1.0, 1.1 (при использовании RSA, что является основным вариантом) - от объединения значений двух хеш-функций - MD5 и SHA-1, взятых от параметров обмена DH. В TLS 1.2 - от значения хеш-функции, заданной в используемом шифронаборе, вычисленного для объединения ServerRandom, ClientRandom и параметров обмена DH (ECDH). В современной (2025) ситуации сообщение ServerKeyExchange можно увидеть в большинстве TLS-сессий (для версий, отличных от 1.3).

CertificateRequest. В TLS возможна обоюдная (двухсторонняя) аутентификация узлов, использующая TLS-сертификаты. Сертификаты могут быть выпущены для самых разных имён и, соответственно, они пригодны для аутентификации клиента. Обычно, клиентские сертификаты используются при доступе к банковским, платёжным системам, к корпоративным веб-шлюзам различного назначения, а также к государственным информационным системам через веб-интерфейс. Сервер может запросить клиентский сертификат при помощи сообщения CertificateRequest. Сообщение содержит список поддерживаемых сервером типов сертификатов и типов криптосистем - здесь указываются криптосистемы, относящиеся к процедуре валидации клиентского сертификата: алгоритмы подписи, хеш-функции. Также в

составе CertificateRequest могут быть переданы имена удостоверяющих центров, ключи которых сервер будет использовать для проверки клиентского сертификата.

ServerHelloDone. В TLS 1.2 и раньше - это сообщение сигнализирует об окончании списка, возглавляемого ServerHello. Сообщение имеет нулевую длину (в TLS-записи ServerHelloDone соответствует 4 байта, так как присутствует заголовок записи) и служит простым флагом, обозначающим, что сервер передал свою часть начальных данных и теперь ожидает ответа от клиента.

Итак, сервер отвечает на ClientHello последовательностью сообщений TLS Handshake, максимум - пятью сообщениями. Типичный для TLS 1.2 случай: передача сервером четырёх сообщений - ServerHello, Certificate, ServerKeyExchange (с параметрами алгоритма Диффи-Хеллмана), ServerHelloDone. После передачи ServerHelloDone, сервер ожидает ответа клиента. Клиент должен ответить своим набором сообщений, который описан ниже.

Certificate (для клиента) - это сообщение содержит клиентский сертификат, если он был запрошен сервером. Если у клиента сертификата нет, а сервер его запрашивает, то клиент либо пропускает данное сообщение (SSLv3), либо отвечает пустым сообщением с типом Certificate (TLS). Клиентский сертификат требуется для двухсторонней аутентификации. Все современные браузеры, работающие на десктопе, поддерживают клиентские сертификаты. Клиентский сертификат, в ряде случаев, может заменить пару "логин/пароль" для аутентификации на том или ином онлайн-ресурсе (для выполнения аутентификации клиенту требуется доступ к секретному ключу, который соответствует сертификату; сам по себе сертификат клиента не достаточен для аутентификации).

ClientKeyExchange - клиентская часть обмена данными, которые используются при получении сеансовых ключей. Содержание этого сообщения зависит от того, какой шифронабор выбран. В версиях TLS до 1.3 есть два основных типа - RSA и несколько вариантов протокола Диффи-Хеллмана. В случае использования RSA, клиент генерирует 48-байтовый случайный секрет (при этом первые два байта содержат версию используемого протокола), зашифровывает его открытым RSA-ключом сервера (этот ключ передаётся в составе серверного TLS-сертификата или в сообщении ServerKeyExchange) и передаёт зашифрованные данные на сервер. Сервер может расшифровать значение, используя соответствующий секретный ключ. *Данная схема прямо запрещена в TLS 1.3, да и для других версий является скорее исторической, так как обладает целым рядом недостатков.* Например, если секретный серверный ключ (от сертификата сервера) станет известен третьей стороне, то она сможет расшифровать ранее записанный TLS-трафик. Тем не менее, обмен сеансовым ключом при помощи RSA продолжает использоваться в некоторых реализациях TLS (кроме 1.3). Современный метод - использование протокола Диффи-Хеллмана. В этом случае, ClientKeyExchange содержит открытый ключ DH.

Этот ключ генерируется клиентом либо в соответствии с параметрами, переданными сервером в `ServerKeyExchange`, либо в соответствии с параметрами, указанными в серверном TLS-сертификате, если последний поддерживает DH. Случай с передачей параметров классического DH в составе сертификата - сейчас является экзотическим, таких сертификатов в "дикой природе" для веба не встречается. Параметры DH (или ECDH), переданные сервером, подписываются серверным секретным ключом; клиент проверяет подпись, используя открытый ключ сервера.

CertificateVerify - если клиент передал в ответ на запрос сервера свой сертификат, то серверу требуется некоторый механизм, позволяющий проверить, что клиент действительно обладает секретным ключом, связанным с сертификатом. Для этого клиент подписывает массив переданных и принятых ранее сообщений `Handshake`. Такая подпись, если её значение удастся успешно проверить открытым ключом из сертификата, удостоверит факт наличия секретного ключа у клиента. Сообщение передаётся только если был передан клиентский сертификат в сообщении `Certificate`. В TLS 1.3 аналог этого сообщения перенесён на сторону сервера и служит механизмом ранней аутентификации криптографических параметров (подробнее схема рассматривается ниже, в описании `Handshake` версии 1.3).

Клиентские сертификаты используются редко. Поэтому в типичном случае клиент, на данном этапе, передаёт одно сообщение - `ClientKeyExchange`. Это сообщение является, согласно спецификации, обязательным.

Следом за сообщением `ClientKeyExchange`, если оно было единственным, либо за `CertificateVerify`, клиент должен передать сообщение `ChangeCipherSpec`. Важный момент: *это сообщение не является сообщением Handshake*. Да, в спецификации TLS встречаются такие, не совсем прозрачные, моменты, когда вроде бы логичное течение протокола прерывается специальными "вставками". При разработке TLS 1.3 от этой сомнительной практики отказались, но `ChangeCipherSpec` всё равно рекомендуется передавать, правда, в качестве фиктивного сообщения, которое просто игнорируется узлами (сообщение нужно для того, чтобы "обмануть" неверно настроенные промежуточные узлы, замаскировав соединение версии 1.3 под предыдущие версии).

ChangeCipherSpec - это специальное сообщение-сигнал, обозначающее, что с данного момента клиент переходит на выбранный шифр, а следующие *TLS-записи* будут зашифрованы. `ChangeCipherSpec` (CCS) имеет собственный тип и, соответственно, передаётся в отдельной TLS-записи. Таким образом, CCS имеет весьма важное значение в TLS (но только раньше версии 1.3), отделяя открытую часть сеанса связи от защищённой.

Со стороны клиента установление соединения завершается отправкой сообщения `Finished`.

Finished. Это сообщение передаётся в зашифрованной TLS-записи, так как следует за сигналом ChangeCipherSpec, который обозначает момент перехода к защищённому обмену информацией. Finished является первым защищённым сообщением в рамках нового сеанса TLS. Вспомните, что к моменту его отправки, если всё прошло успешно, сервер и клиент уже согласовали все необходимые параметры защищённой сессии: шифронабор, сеансовый ключ, алгоритм кода аутентификации сообщения. Предназначение сообщений Finished - получить криптографически стойкое подтверждение, что сервер согласовал эти параметры именно с тем узлом, с которым предполагалось, то же самое - и для клиента, который получит сообщение Finished от сервера.

Клиентское Finished содержит отпечаток - значение хеш-функции, - от всех предыдущих сообщений Handshake, отправленных и клиентом, и сервером. Получив сообщение Finished в защищённой TLS-записи, сервер может вычислить значение хеш-функции от известных ему предшествовавших сообщений и сравнить результат. Таким образом подтверждается подлинность сообщений и, соответственно, оказываются криптографически защищены выбранный шифронабор и другие параметры соединения. Впрочем, данная защита не лишена недостатков: например, так как сообщения ClientHello и ServerHello не содержат подписей, Finished никак не защищает сессию от атак, основанных на подмене параметров *до отправки* ChangeCipherSpec. Этот дефект протокола использован в на шумевшей в 2015 году атаке Logjam (атаки на TLS/SSL подробно рассматриваются в специальном разделе). В TLS 1.3 аутентификация параметров сессии начинается раньше: а именно - сервер незамедлительно удостоверяет первую часть сообщений Handshake, включая выбор криптосистем, при помощи серверного сообщения CertificateVerify.

В ответ на Finished со стороны клиента, сервер, в случае успешного установления соединения, отправляет свою пару ChangeCipherSpec и Finished. На стороне клиента полученное серверное сообщение Finished так же используется для проверки, что сообщения Handshake не подверглись модификации активным атакующим, который перехватил канал. Состав Finished сервера отличается от клиентского варианта, так как включает клиентское сообщение Finished (и отличную от клиентской дополнительную строку байтов).

Третьей стороне, активно перехватывающей канал связи, для того, чтобы успешно произвести подмену сообщений Handshake, потребуется вычислить сеансовый ключ и другие секретные параметры (если они использовались), после чего сфабриковать корректное сообщение Finished. Это вычислительно трудная задача, обычно неразрешимая на практике, если выбраны правильные настройки протокола и корректная реализация. Однако, в случае использования нестойких шифронаборов, атакующий может на лету вычислить секретный сеансовый ключ и успешно подделать сообщение Finished.

После того как клиент и сервер обменялись парами ChangeCipherSpec и Finished - защищённое соединение успешно установлено и данные могут передаваться в защищённой форме.

Посмотрим на ServerHello (TLS 1.1) и последующие сообщения сервера в деталях, примером послужит ответ сервера на сообщение ClientHello, которое разбиралось выше. В этом примере несколько сообщений TLS передаются в одной TLS-записи.

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	16 03 02 (0x0302).	; Тип записи - Handshake (0x16) - версия TLS 1.1
0003	07 02	; Длина пакета данных (0x0702 = 1794 байта - в этой записи
		содержатся все отправленные сервером сообщения - ServerHello, Certificate, ServerKeyExchange, ServerHelloDone - отсюда и такая длина).
ServerHello. Начало сообщения		
0005	02	; Тип сообщения: ServerHello (02).
0006	00 00 46	; Длина данных: (0x000046 = 70 байтов - это данные ServerHello).
0009	03 02	; Версия протокола: 0x0302 - TLS 1.1.
0011	55 dc	; 32 байта ServerRandom, в начале поля сервер указывает
		timestamp.
[...]		
0043	20	; Длина SessionID - (0x20 = 32 байта).
0044	81 19	; 32 байта, содержащие переданный сервером
идентификатор		сессии (SessionID).
[...]		
0076	c0 14	; Выбранный сервером шифронабор - 0xc014 -
Диффи-Хеллмана		TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (протокол
		на эллиптических кривых - для сеансового ключа,
		RSA - для подписи, AES с 256-битным ключом в
режиме CBC		в качестве потокового шифра, SHA-1 для MAC).

0078 00 ; Выбранный сервером метод сжатия - 00 - null.

Certificate. Начало сообщения

0079 0b ; Сообщение Certificate (тип - 0x0b = 11).

0080 00 05 21 ; Длина сообщения (0x000521 = 1313 байтов).

0083 00 05 1e ; Длина данных сертификатов (0x00051e = 1310 байтов).

0086 00 05 1b ; Длина поля с сертификатом. Каждый из сертификатов, указанных в сообщении Certificate, предваряется тремя байтами, содержащими длину записи данного сертификата. Также передаётся общая длина набора сертификатов - отсюда и эти "вложенные" поля с числом байтов.

0089 30 82 05 17 30 ; Байты, составляющие сертификат (сам сертификат здесь не приводится). Сертификат передаётся сервером в бинарном представлении, с использованием определённого кодирования. На практике это означает, что передаются сертификаты в общепринятом формате DER, который поддерживается большинством утилит: например, из пакета OpenSSL и др.

[...]

ServerKeyExchange. Начало сообщения

1396 0c 00 01 8b ; Сообщение ServerKeyExchange (тип - 0x0c = 12), длина данных (0x00018b = 395 байтов). Так как выбран шифронабор с ECDHE, то сообщение ServerKeyExchange содержит параметры именно для алгоритма ECDHE. Каких-то специальных флагов, обозначающих ServerKeyExchange как ECDHE, - не предусмотрено, всё определяется шифронабором. Использование DH на эллиптических кривых описано в RFC 4492 (<https://tools.ietf.org/html/rfc4492>).

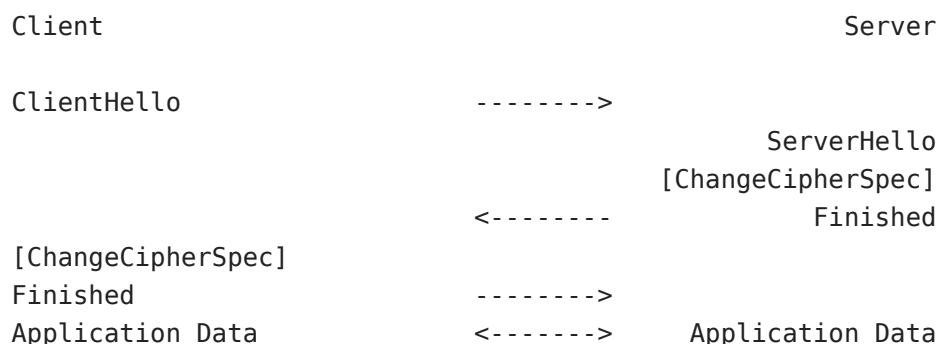
1400	03	; Тип кривой (0x03 - типовая именованная кривая).
1401	00 19	; Идентификатор выбранной сервером кривой (0x0019 -
secp521r1,		реестр кривых ведёт IANA, каждая из типовых
кривых содержит		в своём определении необходимые для DH параметры,
которые строго		зафиксированы: генератор, разрядность группы
точек и др.).		
1403	85	; Длина поля публичного ключа, которое идёт следом
		(0x85 = 133 байта).
1404	04 00 8f 96 86	; 133 байта, представляющие собой публичный ключ
ECDHE		сервера. Публичный ключ - это точка на кривой,
которую сервер вычислил,		используя параметры кривой. В соответствии с
форматом записи		точек, указанным клиентом в расширении
ClientHello, передаются		аффинные координаты точки (x,y) (запись
предваряется заголовком).		Клиенту параметры кривой известны потому, что
используется		типовая кривая.
[...]		
1537	01 00	; Длина подписи, соответствующей параметрам DH
ключа DH следует		(0x0100 - 256 байтов). За значением открытого
ключом,		серверная подпись, выполненная по алгоритму и с
сертификате		которые соответствуют указанным в серверном
представления подписи		(в нашем случае - RSA). В TLS 1.2 формат
добавлены поля,		несколько другой: в состав ServerKeyExchange
на параметрах DH		обозначающие используемый для генерации подписи
подписи		набор из хеш-функции и криптосистемы электронной
		(например, RSA+SHA-256).
1539	25 23 f5	; Значение подписи (полностью не приводится).
[...]		
ServerHelloDone. Начало сообщения		

1795 0e 00 00 00 ; Сообщение ServerHelloDone (тип - 0x0e = 14). Это сообщение имеет нулевую длину, поэтому следом за типом идут три байта с нулевыми значениями (это кодирующие длину байты).

Сокращённый формат Handshake в TLS 1.2 (и раньше)

Установление TLS-соединения - многоступенчатый процесс, достаточно сложный и требующий проведения заметного количества вычислительных операций. Особенно жадными в плане процессорного времени оказываются операции проверки подписей и выполнения других действий, связанных с асимметричными криптосистемами. Каждая итерация, связанная с отправкой запроса и получением ответа, вносит дополнительную задержку, а для массовых сервисов миллисекунды ожидания превращаются в весьма ощутимые затраты на оборудование. Для экономии ресурсов существует сокращённая версия Handshake.

В составе ServerHello сервер передаёт SessionID - идентификатор новой сессии. Этот идентификатор может быть использован клиентом позже, в составе ClientHello (см. выше). Предполагается, что на стороне сервера (и клиента) сохранены необходимые параметры, которые могут быть быстро восстановлены, возобновив тем самым сессию.



Установление соединения по сокращённой схеме
Источник: RFC 5246

В случае использования сокращённой схемы, сразу после получения ClientHello, содержащего валидный идентификатор сессии SessionID, сервер отвечает сообщениями ServerHello, ChangeCipherSpec и Finished (этот вариант, кстати, напоминает типовой Handshake в TLS 1.3). После того, как клиент пришлёт свои CCS и Finished, сессия возобновляется и узлы могут сразу начать обмен данными в защищённом режиме.

Сокращённый сценарий содержит существенно меньше сообщений, позволяет не использовать вычислительно затратные операции проверки подписей и генерации общего секрета, кроме того, из-за меньшего количества сообщений заметно

снижается задержка при установлении соединения (экономится время, требуемое на отправку пакетов от клиента к серверу и обратно). Современные браузеры широко используют сокращённый Handshake (но не обязательно с версиями до 1.3). Сессии могут сохраняться на серверах в течение нескольких минут, нескольких часов, а возможно - дольше.

Сейчас используются и другие способы возобновления сессий. Например, [RFC 5077](#) вводит понятие тикета сессии (TLS Session Ticket) - это расширение, в составе которого сервер передаёт клиенту зашифрованное представление *серверного контекста* TLS. Клиент может восстановить сессию, передав в составе расширения ClientHello сохранённый тикет.

Общие замечания

После того как клиент и сервер установили соединение (по полной или сокращённой схеме), они обмениваются TLS-записями с типом Application Data, каждая такая запись содержит блок данных, зашифрованный симметричным шифром, а также код аутентификации сообщения. Код аутентификации защищает сообщение от изменения. Клиент и сервер самостоятельно ведут учёт полученных и переданных блоков.

TLS имеет ряд важных, и достаточно общих, технических особенностей: во-первых, этот протокол никак не скрывает факт установления соединения (как указано выше, системы DPI могут уверенно детектировать начало сеанса TLS при помощи простых правил-фильтров); во-вторых, в TLS возможны утечки метаданных: число переданных блоков, различные предупреждения, передаваемые в открытом виде - всё это позволяет пассивному наблюдателю сделать некоторые выводы о характере передаваемой информации и, в случае веба, о типе совершаемых пользователем на веб-ресурсе действий. TLS лишь более или менее надёжно защищает от перехвата и подмены саму передаваемую информацию.

Повторное проведение Handshake

Во время работы в рамках TLS-сессии клиент и сервер могут столкнуться с необходимостью повторно провести Handshake. Такой случай предусмотрен протоколом. Хрестоматийная ситуация, когда это может потребоваться, следующая: после того как установлено TLS-соединение, клиент, использующий HTTP, запрашивает документ по некоторому URL, однако этот URL требует дополнительной авторизации с использованием клиентского сертификата. Клиентский сертификат может быть передан только в составе сообщения Handshake, поэтому сервер отправляет клиенту сообщение HelloRequest, требующее повторного обмена сообщениями Handshake (уже с клиентским сертификатом). Клиент может инициировать новую сессию в любой момент, передав сообщение ClientHello (в TLS

1.3 эта возможность исключена). Так как узлы уже согласовали TLS-соединение, обмен сообщениями повторного Handshake будет проводиться в защищённом виде. В примере с клиентским сертификатом это означает, что сертификат не будет виден прослушивающей канал стороне.

Предупреждения (Alert)

Спецификация TLS предусматривает обмен сообщениями с типом Alert. Это предупреждения и сообщения об ошибках. Например, в случае, если сервер не смог корректно разобрать сообщение ClientHello, он отвечает сообщением Alert, содержащим код ошибки Parse Error. Сообщения Alert иногда могут быть использованы в составе атак на TLS-узлы: так, при помощи отправки этих сообщений можно "подвешивать" сессию, когда перехватывающему устройству требуется время на, например, вычисление ключа.

После того, как узлы согласовали криптографический контекст и обменялись сообщениями ChangeCipherSpec - сообщения Alert передаются в зашифрованном виде. То есть, с этого момента содержание предупреждения не может быть прочитано прослушивающей трафик стороной, но сам тип записи - Alert - всё равно передаётся в открытом виде (*кроме TLS 1.3*), что, соответственно, может приводить к утечке информации о состоянии узлов и соединения.

Сеансовые ключи

Асимметричные криптосистемы (с открытым ключом) не подходят для быстрой передачи потоков данных. Поэтому TLS использует внутри сессии симметричные шифры. Обычно это блочные шифры, работающие в режиме, сходном с потоковым (GCM и др.). Есть несколько основных способов выработки общего секрета, который служит основой для вычисления сеансовых ключей. О симметричных сеансовых ключах узлы договариваются в процессе установления соединения (Handshake). Главные шаги протокола, связанные с получением общего секрета, рассмотрены выше. Осталось обсудить некоторые подробности. Прежде всего: *симметричных ключей используется пара* - один ключ для передаваемых сообщений, второй - для принимаемых. Серверному ключу для передаваемых сообщений (Write) соответствует клиентский ключ для принимаемых (Read), и наоборот. Обратите внимание, что выше примером служат сообщения протокола версии до 1.3. **Схема получения секретов и управления ими в TLS 1.3 существенно переработана и отличается от предыдущих версий фундаментальным образом.** Так, в TLS 1.3 используются несколько уровней ключей (для Handshake, для защиты трафика), каждый из уровней включает пару значений. Ход первоначального соединения для TLS 1.3 рассмотрен подробно в специальном разделе, но прежде чем к нему перейти - необходимо дополнительно разобраться с базовыми принципами и протоколами.

Основа ключей в TLS - общий секрет Master Secret - генерируется из нескольких составляющих: так называемый Premaster Secret, ClientRandom и ServerRandom. Эти составляющие изменяются от сессии к сессии. В **TLS 1.2** и раньше, Premaster Secret согласуется в рамках Handshake. Это либо последовательность случайных байтов, зашифрованная открытым ключом сервера (RSA, сейчас этот метод не должен применяться), либо значение, полученное в результате обмена по протоколу Диффи-Хеллмана (основной способ). В **TLS 1.3** RSA-обмен прямо запрещён, а для получения симметричных секретов используется протокол Диффи-Хеллмана. Более того, в рамках внедрения криптосистем с постквантовой стойкостью для TLS 1.3 добавлен ещё один метод: "инкапсуляция" (передача) секрета при помощи криптосистемы ML-KEM. Данный метод подразумевает передачу открытого ключа клиентом, и концептуально сходен с зашифрованием секрета RSA, только такое зашифрование тут происходит со стороны сервера в сторону клиента, а сам алгоритм существенно отличается в математической реализации.

Master Secret - это массив из 48 байтов, получаемый в результате применения клиентом и сервером функции, выдающей псевдослучайные значения ("псевдослучайной функции"). На вход этой функции подаются только что описанные параметры (Premaster Secret, ClientRandom и ServerRandom). Используемая функция определена спецификацией. То есть, рассматриваемые как последовательности байтов значения Premaster Secret, ClientRandom и ServerRandom поступают на вход функции (как аргументы), а результатом вычисления является последовательность из 48 байтов, которая считается псевдослучайной. "Псевдослучайность" тут означает, что не обладающая дополнительными данными сторона не может при помощи практических вычислений отличить эту последовательность от действительно случайной. Псевдослучайная функция (PRF) TLS 1.2 построена на базе хеш-функции SHA-256 (либо может быть использована более "мощная" хеш-функция, указанная в составе шифронабора). Предыдущие версии TLS используют конструкцию на базе сочетания MD5 (которая давно не считается криптографически стойкой) и SHA-1 (которую перестали считать достаточно стойкой в 2015 году). При этом способ использования MD5 и SHA-1 для генерации сеансового ключа не приводит к возникновению уязвимостей, связанных с известными сейчас (2025) недостатками этих функций.

RFC 5246 определяет Master Secret для TLS 1.2 следующим образом:

```
master_secret = PRF(pre_master_secret, "master secret",  
                    ClientHello.random + ServerHello.random)[0..47];
```

Master Secret - *это ещё не сеансовый ключ*. Дело в том, что разные шифры требуют разных ключей и дополнительных данных (например, инициализирующих векторов). Эти ключи и данные вычисляются на основе Master Secret с использованием той же псевдослучайной функции. Делается это следующим способом (TLS 1.2):

1. для выбранного шифронабора определяется количество байтов, необходимое для получения ключа и инициализирующей информации;
2. нужное число байтов (Key Block) генерируется на основе Master Secret, ServerRandom, ClientRandom при помощи последовательных вызовов всё той же псевдослучайной функции;
3. полученный Key Block разбивается на подмножества, нужные для инициализации и работы шифра, а также для вычисления кода аутентификации сообщений (MAC). При этом могут различаться ключи и векторы инициализации сервера и клиента (но полный набор криптографических параметров известен обоим узлам).

Логика только что описанной схемы в базовых чертах применима и к TLS 1.3, однако в новой версии механизм формирования симметричных ключей сложнее, в частности, введено понятие "ключевого контекста", которое включает в себя значения и самих сообщений Handshake. Подробнее этот механизм рассмотрен в специальном разделе, в рамках описания установления соединения TLS 1.3.

Зашифрование полезной нагрузки осуществляется на уровне TLS-записей.

Большинство методов получения исходных данных для сеансового ключа так или иначе оперируют публичным ключом сервера. На практике этот ключ использует почти 100% TLS-сессий. Публичный ключ передаётся сервером в сертификате. Помимо рассмотренных выше вариантов RSA и Диффи-Хеллмана, возможны экзотические схемы получения общего секрета:

1. **PSK - pre-shared key**. Схема основана на использовании общего секретного ключа и симметричной криптосистемы, при условии, что ключ был согласован заранее;
2. **временный ключ RSA** - исторический метод, предполагавший создание сеансового ключа RSA. Сейчас данный метод не используется, однако его поддержка послужила основой для атаки FREAK, опубликованной в 2014 году;
3. **SRP** - протокол SRP (Secure Remote Password), [RFC 5054](#). Протокол, позволяющий сгенерировать общий симметричный секретный ключ достаточной стойкости на основе известного клиенту и серверу пароля, без раскрытия этого пароля через незащищённый канал;
4. **Анонимный DH** - анонимный вариант протокола Диффи-Хеллмана, в котором не используется подпись на серверных параметрах. Такая схема подвержена атаке типа "человек посередине", встречается крайне редко (дополнительная защита от атак может быть обеспечена при помощи сохранения отпечатка ключа);

5. **DH с сертификатом** - вариант DH, в котором параметры (модуль, генератор и открытый ключ сервера) определены в серверном сертификате. Этот метод является историческим, требует специального сертификата и на практике не встречается. Основное его отличие от используемых сейчас вариантов (нередко называемых "эфемерным Диффи-Хеллманом", хотя подобного термина *следует избегать*) состоит в том, что серверный открытый ключ протокола Диффи-Хеллмана оказывается зафиксирован: именно он входит в сертификат и подписывается удостоверяющим центром. Это означает, что схема не обладает прогрессивной секретностью. Сервер всякий раз использует один и тот же секретный ключ Диффи-Хеллмана, а раскрытие этого ключа позволяет расшифровать ранее записанный трафик TLS.

Протокол Диффи-Хеллмана

Чтобы зашифровывать полезные данные, сторонам TLS-соединения нужно получить совпадающий набор секретных ключей для используемых шифров. Современный метод генерации общего сеансового секрета - это протокол Диффи-Хеллмана (DH). Использование DH, прежде всего, позволяет добиться прогрессивной секретности (Perfect Forward Secrecy, PFS), когда раскрытие долговременного серверного ключа асимметричной криптосистемы (RSA или ECDSA) не приводит автоматически к раскрытию ранее записанного трафика.

Протокол Диффи-Хеллмана - важнейший элемент современных реализаций TLS. Этот протокол имеет огромное значение для множества других защищённых протоколов Интернета, для прикладной криптографии в целом. Протокол не является шифром или алгоритмом электронной подписи. На его основе можно построить и асимметричную криптосистему, и схему электронной подписи, но в TLS он используется с другой целью: этот протокол позволяет участникам обмена информацией договориться об общем секретном ключе через открытый канал связи. Задача Диффи-Хеллмана, лежащая в основе классической версии протокола, прямо связана с алгебраической задачей дискретного логарифмирования в конечной группе — на вычислительной сложности этой задачи и построена защита протокола от перехвата секретного ключа.

Основная идея протокола DH может быть сформулирована достаточно просто. Две стороны, желающие получить общий секрет, очевидно, могут сгенерировать собственные, локальные, секреты, например, то или иное число. Предположим, что у нас есть некоторая однонаправленная функция - то есть, можно легко вычислить значение функции для заданного аргумента, но обратить вычисления и узнать аргумент по значению - сложно. Тогда можно было бы устроить алгоритм таким образом, чтобы договаривающиеся об общем секрете стороны передавали по открытому каналу друг другу значения функции от секретных аргументов, проводили

над этими значениями и аргументами собственные вычисления и, в результате применения некоторых преобразований, могли бы каждая *прийти к одному и тому же значению*. Просматривающая же коммуникации третья сторона видела бы только значения однонаправленной функции, без возможности обратить её и узнать секрет. Успешная реализация такого протокола и есть схема Диффи-Хеллмана.

Важно отметить, что схема представляет собой протокол верхнего уровня, она не зависит от используемых математических примитивов, лишь бы они обладали нужными свойствами. Схема может быть реализована на базе самых разных математических систем, а не только, например, на группе точек эллиптической кривой. Самое интересное, что именно эта идея используется в самых современных криптосистемах, предлагаемых в качестве замены классическим в постквантовую эпоху. Обобщение схемы Диффи-Хеллмана позволяет строить криптосистемы, обладающие стойкостью к взлому с использованием (гипотетического) универсального квантового компьютера. Такие криптосистемы относятся к постквантовой криптографии. То есть, несмотря на то, что используемые сейчас варианты DH уязвимы для атак с помощью квантового компьютера, *ту же самую математическую схему* можно реализовать в варианте, обладающем постквантовой стойкостью, перенеся без изменений на другие математические объекты. Поэтому, иногда встречающиеся утверждения, что протокол DH не обладает постквантовой стойкостью, заметно искажают реальное положение дел: уязвимы лишь конкретные реализации протокола, но не он сам. Причём речь здесь идёт о *математической* реализации протокола, о самом алгоритме, а не о конкретном воплощении в программном коде.

Вернёмся к использованию DH в TLS. Сейчас (2025) распространены два варианта протокола. В классическом случае, DH работает в "обычной" конечной группе, задаваемой простым числом P , которое называется модулем. "Обычная" - довольно условное определение: имеется в виду, что классический DH работает в арифметике остатков, где числа берутся по модулю некоторого достаточно большого простого числа. Такие операции более привычны, чем второй распространённый вариант DH, который использует группу точек эллиптической кривой.

Под "группой" здесь понимается алгебраическая структура - множество с бинарной операцией, результат которой всегда лежит в этом же множестве, подчиняющейся некоторому набору аксиом (ассоциативность, наличие обратного элемента, нейтральный элемент). Конечная группа - это группа с конечным числом элементов. Для понимания принципов работы классического варианта DH не обязательно хорошо знать теорию групп - достаточно будет знакомства с арифметикой остатков.

Рассмотрим классический вариант DH подробнее. Здесь группа, в которой будут проводиться операции вычисления общего секрета TLS, это всего лишь множество остатков от деления на некоторое простое число, которое называют модулем, вместе

с привычным умножением целых чисел. Это обязательно достаточно большое простое число. "Большое" - означает, что разрядность его записи превышает 1000 бит, а современная рекомендация для классического варианта - не менее 2048 бит. В TLS 1.2 сервер передаёт это число (значение модуля) в составе сообщения ServerKeyExchange, если используется классический алгоритм DH.

Модуль полностью определяет используемую группу, поэтому каких-то дополнительных параметров передавать не нужно. На практике веб-серверы используют то или иное типовое значение модуля. В TLS 1.3 доступные параметры классического DH зафиксированы и передаются только идентификаторы, обозначающие ту или иную группу. Модуль не является секретным - он передаётся в открытом виде. Таким образом, известны группы, используемые большинством веб-серверов, поддерживающих DH (классический). Вообще говоря, уникальные группы DH рекомендуется генерировать при начальной настройке TLS, это несложно сделать стандартными утилитами. Тем не менее, не так давно наиболее распространённая типовая группа имела разрядность в 1024-бита, что не слишком много. Вместе с задающим группу модулем, сервер передаёт другой открытый параметр - генератор (это тоже число - см. подробности ниже), - а также свою часть обмена Диффи-Хеллмана - открытый ключ (принцип его вычисления рассмотрен ниже).

Второй вариант протокола Диффи-Хеллмана - "эллиптический", он основан на эллиптических кривых. (Элементарный обзор некоторых основ практического математического аппарата, который стоит за криптографией на эллиптических кривых, дан в "[Приложении А](#)".) Математические операции протокола DH в обоих случаях эквивалентны, отличие кроется в используемых группах: эллиптический вариант работает в группе точек эллиптической кривой. Такая группа может быть построена при помощи заданной на кривой операции, называемой сложением точек. Следует понимать, что здесь важно не название, а свойства операции, с таким же успехом её можно было назвать умножением точек или просто "блэбом". Собственно, различают два эквивалентных подхода к обозначению групповых операций: мультипликативный (умножение) и аддитивный (сложение). Обычно, применительно к TLS (и прикладной криптографии), в случае классического протокола DH - используют мультипликативный вариант и говорят об умножении (числа тут действительно умножаются, в привычном школьном смысле) и возведении в степень (экспонента); в случае эллиптического - аддитивный, и говорят о сложении точек и нахождении кратного (умножение на скаляр). Это чисто терминологическое отличие, никак не связанное со стойкостью и другими особенностями протокола.

Итак, операция сложения точек на кривой ставит в соответствие двум точкам - третью, называемую их суммой. На этой операции строится алгоритм удвоения точки и умножения на скаляр (целое положительное число, *не являющееся, соответственно, точкой на кривой*). Также вводится нейтральный элемент - в его

роли выступает "бесконечно удалённая точка". Эллиптическая кривая, в общем смысле, обладает непрерывностью, но в случае с протоколом Диффи-Хеллмана в TLS она рассматривается только в "целых" (или, если хотите, "дискретных") точках, множество которых задано определённым образом. Прикладная особенность группы точек эллиптической кривой состоит в том, что решение задачи дискретного логарифмирования (обратной задачи для основы протокола DH) здесь обычно сложнее, чем в "обычной" группе классического DH, что позволяет использовать ключи меньшей разрядности с сохранением стойкости. Впрочем, для понимания математического принципа протокола Диффи-Хеллмана детали работы с эллиптическими кривыми не требуются.

Протокол DH на "обычной" группе (классический) работает следующим образом. Для того чтобы получить общий секретный ключ, стороны сначала выбирают общие параметры Диффи-Хеллмана — это модуль, задающий группу (простое число), а также некоторый элемент этой группы, называемый генератором — соответственно: P и G (то есть, два целых числа, $G < P$). Параметры DH открыты и считаются известными третьей стороне. На следующем шаге каждая из сторон выбирает собственное секретное целое число a (и, соответственно, b) и вычисляет значение $A = G^a \bmod P$ (соответственно, $B = G^b \bmod P$). То есть, все операции проводятся по модулю P (это "взятие остатка" в школьном смысле: числа умножаются обычным способом, а от результата берётся остаток от деления на P), что, собственно, и отображает их результаты в элементы группы. Далее стороны обмениваются по открытому каналу значениями A , B и вычисляют $A^b \bmod P$ и $B^a \bmod P$, соответственно, каждая для себя. Полученные значения равны, так как, из свойства степеней, $A^b = (G^a)^b = G^{ab} = B^a = (G^b)^a = G^{ab}$. Таким образом, стороны получили общий секретный ключ $G^{ab} = G^{ba}$ (это, опять же, целое число $s < P$).

Если вернуться к описанной выше основной идее протокола, то операция возведения в степень (по модулю) выступает в роли однонаправленной функции, значения которой передаются по открытому каналу. Свойство (коммутативность) групповой операции (в данном случае, эквивалентное более привычному свойству степеней), позволяет сторонам, осуществляющим обмен значениями, прийти к одинаковому результату ($G^{ab} = G^{ba}$). В случае TLS, сервер передаёт в сторону клиента параметры Диффи-Хеллмана и свой открытый ключ (A), удостоверяя эти значения собственной электронной подписью (либо RSA, либо ECDSA). Подпись вычисляется от ключа, открытая часть которого указана в сертификате сервера. Прослушивающая канал сторона знает значения P , G , A и B . Но для того, чтобы определить значение секретного ключа, необходимо вычислить a или b , решив уравнение вида $A = G^x \bmod P$ относительно x . Стоящая за решением этого уравнения классическая задача и называется *задачей дискретного логарифмирования в конечной группе*. Эта задача разрешима, но, в общем случае, вычислительно трудна для чисел достаточно

большой разрядности (1024 бита и выше, для "классического" протокола DH), поэтому возведение в степень на практике оказывается однонаправленным.

Фундаментальная причина сложности данной операции сходна с общими проблемами деления (или вычитания, тут всё зависит от используемой терминологии) в группах: грубо говоря, деление представляет собой операцию *поиска* среди элементов группы такого, который при умножении на известный делитель давал бы делимое (поиск обратного элемента). Ситуация эквивалентна привычному делению из курса начальной школы: чтобы разделить 15 на 3 нужно найти такое число, которое при умножении на 3 давало бы 15. Если известны некоторые свойства группы, позволяющие построить её арифметическую структуру и ввести некоторую функцию сравнения, упорядочивания; или группу удастся рассмотреть в составе "большой" структуры, то операцию деления можно оптимизировать, используя тот или иной "быстрый" пошаговый алгоритм, отбрасывающий из перебора заведомо неподходящие элементы. Хорошо известным примером такой оптимизации является школьное "деление столбиком". Именно поэтому в случае с современными вариантами DH работает квантовый алгоритм Шора, использующий квантовое представление структуры группы (кольца, к которому, если говорить строго, конкретная группа принадлежит). Измерение этого квантового представления при помощи гипотетического квантового вычислителя с высокой вероятностью даёт результат, позволяющий уже классическим методом очень быстро выполнить дискретное логарифмирование.

Практическая применимость протокола Диффи-Хеллмана основывается на том, что знание секретного значения позволяет быстро вычислить нужную экспоненту.

Действительно, предположим, что требуется вычислить $A = G^m$. На первый взгляд, для этого необходимо умножить G на себя $m-1$ раз: $G * G * \dots * G$. Казалось бы, атакующая сторона тут оказывается в таком же положении, как и стороны, использующие протокол: атакующий может начать умножать G (это открытый параметр) и продолжать вычисления до тех пор, пока не получит искомое значение A (которое перехватил). Таким образом можно определить значение m . Однако реальная ситуация отличается. Дело в том, что, зная секретный параметр m , вычислить G можно гораздо быстрее. Простейший пример: предположим, что нам нужно вычислить G^5 , показатель 5 заранее известен. $G^5 = G * G * G * G * G$. Это четыре операции умножения, но с другой стороны, значение G^2 можно записать, и использовать несколько раз. Получаем: $G^2 * G^2 * G$, а это уже только три умножения (учитывая $G * G$). То есть, зная значение показателя, можно применить быстрые алгоритмы (см. алгоритмы Монтгомери и др.) и получить огромный выигрыш в числе операций по сравнению с атакующим, который должен проверить все показатели, что для разрядности в сотни бит потребует слишком много времени. Это фундаментальное свойство в теории криптосистем называется "удвоением значения" (потому что повсеместно используется именно удвоение, но выстроенное в

некоторое дерево). Чтобы протокол Диффи-Хеллмана можно было обобщить на ту или иную группу, требуется прежде найти способ перенести на эту группу "удвоение" (а это не всегда возможно).

Для протокола DH, работающего на эллиптической кривой, вместо модуля P задаётся сама кривая (с дополнительными параметрами); вместо возведения в степень - используется умножение на скаляр (то есть, вычисляется кратное для точки: если мы умножаем точку A на значение 5 , это означает, что на кривой вычисляется точка $5A = A+A+A+A+A$; подробности см. в "[Приложении А](#)"). В TLS используются именованные кривые, параметры которых заранее известны сторонам. Генератором, в эллиптическом случае, является точка кривой. Однако логика протокола от этих особенностей не зависит. Операции DH на эллиптической кривой просто будут записаны в аддитивной форме: $Ab = (Ga)b = Ba = (Gb)a = Gab$. Обратите внимание на скобки в предыдущем выражении: они указывают на "ассоциативность", в дополнение к плноценной коммутативности, обобщение которых и позволяет перенести схему Диффи-Хеллмана на другие математические объекты. Конечно, должен работать и описанный в предыдущем абзаце механизм ускорения вычислений при известном секрете. Для группы точек эллиптической кривой он работает, сводя основную часть вычислений к некоторому количеству удвоений точки.

Итак, практическая полезность DH строится на сложности задачи дискретного логарифмирования в конечной группе. Известно, что при наличии больших, но вполне доступных, вычислительных мощностей (не квантовых), для 1024-битной "классической" (не "эллиптической") группы можно уже сейчас предвычислить арифметическую структуру, потратив пару лет работы суперкомпьютера и сохранив результаты в специальных таблицах. После этого вычислять дискретный логарифм в конкретной группе можно достаточно быстро (за часы, а возможно, просто на лету), особенно, если вы используете специализированную многопроцессорную систему. Это означает, что можно расшифровать записанный ранее трафик TLS-сессий (а также других протоколов, использующих DH). Дело в том, что сеансовый ключ, если вы умеете отыскивать дискретный логарифм, элементарно вычисляется из ключа DH, который передаётся в открытом виде. Предвычислить нужную структуру можно только для заранее известной группы, поэтому для атакующего важно, чтобы TLS-серверы использовали типовые параметры. При этом, для тех, у кого ресурсов мало (кто не является специализированным агентством, например), группа остаётся вполне стойкой. (Для групп точек эллиптической кривой подобных алгоритмов быстрого дискретного логарифмирования на настоящий момент в открытой печати не опубликовано. С фундаментальной точки зрения это, скорее всего, обусловлено тем, что для произвольных эллиптических кривых не удаётся найти некоторых аналогов простых чисел, которые помогли бы *доступным, в плане вычислений, способом* вложить группу кривой в другое множество с дополнительной структурой, где

значение логарифма отыскать проще. Необходимо отметить, что это верно не для всякой эллиптической кривой и не для всяких параметров: в целом ряде случаев такое вложение можно вычислить на практике.)

Разновидность DH в группе точек эллиптической кривой называется ECDH. Это рекомендуемый сейчас вариант алгоритма. В современных реализациях TLS он получил большое распространение. Так как отыскание дискретного логарифма в группе точек эллиптической кривой вычислительно сложнее, можно использовать ключи с меньшей разрядностью. Высокую секретность, согласно современным представлениям, обеспечивает группа эллиптической кривой с разрядностью 256 и более бит. Также алгоритм позволяет безопасно использовать общую кривую, а не генерировать новую для каждого сервера. По крайней мере, современная ситуация такова, что рекомендовано использовать хорошо известные кривые из утверждённого списка (в TLS 1.2, 1.1 возможно использование собственных параметров, они будут передаваться в сообщениях Handshake; в TLS 1.3 - набор зафиксирован спецификацией). Распространённый случай – кривая P-256, предлагающая разрядность 256 бит. Если группу классического DH в TLS (до 1.3) легко поменять - модуль и генератор непосредственно передаются в сообщении сервера и могут быть любыми, - то для типовых эллиптических кривых всё сильно сложнее: все параметры здесь фиксированы заранее, клиент и сервер могут договориться только о выборе кривой из ограниченного списка.

В TLS версий до 1.3 параметры DH всегда задавал сервер. Эти параметры, как сказано выше, подписываются серверным ключом, что необходимо для того, чтобы параметры не могли быть заменены в момент передачи. Хотя сообщение Finished защищает весь обмен Handshake, активный атакующий мог бы подменить параметры DH на свои собственные, перехватив соединение и, в дальнейшем, сгенерировав для клиента и сервера разные, но корректные сообщения Finished - протокол DH сам по себе не защищён от атаки типа человек посередине (схема, сходная с только что описанной, применяется в атаке Logjam). Поэтому подпись на параметрах крайне важна. В TLS 1.3 протокол DH используется уже на начальном шаге установления соединения, поэтому параметры подписываются в составе прочих сообщений, а подпись передаётся сервером в сообщении CertificateVerify. Другое важное отличие 1.3 в том, что здесь параметры DH согласуются узлами, причём, инициатива принадлежит клиенту: он передаёт список поддерживаемых групп (а это и эллиптические, и "мультипликативные", классические варианты) в расширении сообщения ClientHello, так же, но в другом расширении, передаются и открытые ключи клиента. Сервер выбирает подходящие параметры.

Широко используются аббревиатуры ECDHE и DHE, то есть, варианты записи, оканчивающиеся буквой E. Эта буква обозначает то, что протокол DH используется для выработки временного сеансового ключа. E - от английского Ephemeral ("эффемерный"). То есть, E - подчёркивает то, что ключ не является постоянным

(однако, как уже отмечено выше, не следует использовать для обозначения этого свойства прямой русский перевод - "эфемерный": в русскоязычном варианте ключ называется "временным" или "сеансовым"). Почему потребовалось отдельно выделять "временные ключи"? Причина в том, что в TLS определены варианты соединений, использующих постоянный серверный открытый ключ DH, заданный в сертификате. Сейчас такие схемы на практике не встречаются, поэтому ключ, полученный в результате обмена сообщениями DH, всегда будет временным. **Это не означает, что данный ключ каким-либо образом автоматически исчезает после закрытия соединения:** во-первых, ключ может не удаляться из памяти сервера или направляться в долговременное хранилище, по тем или иным причинам; во-вторых, ключ можно восстановить из записи трафика, при условии, что третьей стороне известен (или она может вычислить) соответствующий секретный параметр Диффи-Хеллмана. Кроме того, ничто не препятствует серверу просто не обновлять "временный" ключ, каждый раз используя одно и то же значение при установлении соединения - то есть, сделав ключ статическим.

Пример серверного сообщения ServerKeyExchange, содержащего параметры DH (в "классическом" варианте) и подпись на этих параметрах. (Вариант на эллиптической кривой рассматривался в анализе дампа трафика выше.)

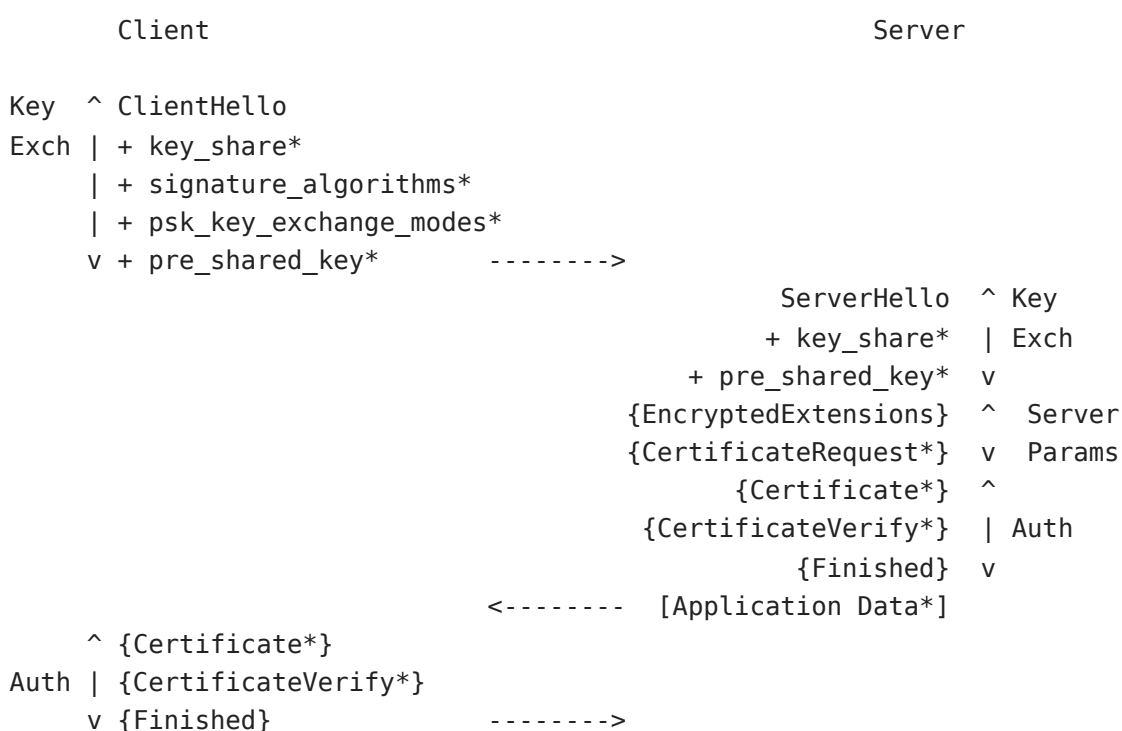
Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	16 03 01	; Handshake, TLS 1.0.
0003	03 0d	; Длина сообщения: 0x030d = 781 байт.
0005	0c	; Сообщение ServerKeyExchange.
0006	00 03 09	; Длина данных: 0x000309 = 777 байт.
0009	01 00	; Длина значения модуля P (из параметров DH):
0x0100 = 256 байт		(2048 бит).
0011	ff ff ff...	; Блок со значением модуля P (полностью не
приводится).		
[...]		
0257	00 01	; Длина блока со значением генератора G: 1 байт.
0259	02	; Значение генератора: 2.
0260	01 00	; Длина блока серверной части обмена DH: 0x0100 =
256 байт.		
0261	5c fb 83...	; Блок со значением серверной части DH (полностью
не приводится).		
[...]		
0517	01 00	; Длина блока подписи (RSA) на параметрах DH - 256
байт,		
		соответствует серверному ключу из сертификата в
2048 бит.		
0519	c4 bc 5d...	; Начало блока, содержащего значение подписи.

Итак, в TLS тем или иным способом, с использованием алгоритма Диффи-Хеллмана, согласуется набор симметричных секретов, на основе которых вычисляются секретные ключи для шифров, а именно - для алгоритмов преобразования передаваемых данных, реализующих фактическую защиту этих данных от раскрытия третьей стороне ("от чтения" и "от подмены"). Как отмечено выше, для защиты трафика в TLS используются только симметричные шифры ("симметричные" - означает, что знание ключа зашифрования позволяет легко расшифровать данные). В том числе, шифр используется и в ходе установления соединения в TLS 1.3: здесь зашифровывается существенная часть данных Handshake, что затрудняет извлечение метаинформации о соединении.

TLS 1.3: первоначальное соединение (Handshake)

В TLS 1.3 от предыдущих версий унаследованы только базовые принципы установления соединения: сохранились роли узлов (соединение инициирует клиент), не изменилась последовательность ClientHello - ServerHello, присутствует сообщение-сигнал Finished. Однако на этом сходство заканчивается. В TLS 1.3 сокращено и общее число сообщений Handshake, и число сообщений, передаваемых в открытом виде: узлы практически сразу переходят на зашифрованный обмен. Поэтому из схемы установления соединения удалён сигнал ChangeCipherSpec - он больше не требуется для реализации логики самого протокола (как ни странно, сам "сигнал" при этом остался и передаётся узлами, см. ниже). Изменилась роль сообщения CertificateVerify - при передаче со стороны сервера оно удостоверяет сообщения первой части Handshake.

Схема полного Handshake TLS 1.3 выглядит следующим образом:



[Application Data] <-----> [Application Data]

Установление соединения по полной схеме TLS 1.3
Источник: RFC 8446

Здесь символом '*' отмечены необязательные сообщения (передаются в зависимости от контекста), а в фигурных скобках даны сообщения Handshake, которые передаются в зашифрованном виде. Квадратные скобки обозначают данные полезной нагрузки, которые передаются в зашифрованном виде, но с использованием другого набора ключей шифрования. Обратите внимание: уже первый ответ сервера - содержит зашифрованные данные. В TLS предыдущих версий - практически все существенные Handshake-сообщения передаются в открытом виде.

В протоколе установления соединения TLS 1.3 выделяются три фазы: выработка начального значения общего криптографического секрета (ключей), определение параметров соединения, аутентификация (сервера и клиента).

К первой фазе относятся ClientHello и ServerHello (а также их расширения, обозначенные на схеме символом '+'). Результатом обмена сообщениями в первой фазе является получение первого секрета (в терминах спецификации исходный секрет называется `handshake_traffic_secret`). На основе этого секрета, используя дополнительно сами сообщения, переданные к этому моменту, стороны получают первый набор симметричных ключей. Эти ключи предназначены для зашифрования сообщений Handshake, начиная с серверного EncryptedExtensions. Например, серверный сертификат в TLS 1.3 передаётся в зашифрованном виде. Это означает, что сторона, прослушивающая канал связи, но не имеющая ключей, не сможет определить, что за сертификат был предоставлен сервером (и был ли он предоставлен вообще - истинная длина сообщений может быть скрыта при помощи дополнения их нулевыми байтами, это также определено спецификацией 1.3). Первая фаза завершается передачей ServerHello и обязательных расширений.

Вторая фаза включает серверные сообщения EncryptedExtensions и, при необходимости аутентификации клиента, CertificateRequest. Новое сообщение - EncryptedExtensions - добавлено в TLS 1.3.

В третьей фазе, представленной группами сообщений Certificate, CertificateVerify и Finished, происходит аутентификация сервера и клиента. Как и в предыдущих версиях TLS, аутентификация может быть полностью исключена (анонимный режим), однако типичный сценарий использования подразумевает, по крайней мере, аутентификацию сервера клиентом. При этом в TLS 1.3 сервер может перейти к отправке данных полезной нагрузки (Application Data на схеме) непосредственно после серверного Finished - это возможно потому, что к этому моменту сервером уже получен первый набор симметричных ключей.

В TLS 1.3 появился новый механизм опознавания используемой *версии протокола*. Поля, содержавшие номер версии в предыдущих вариантах, сохранены в статусе "исторических", их значения зафиксированы. Версия же 1.3 (её номер: 0x0304) передаётся в специальном расширении сообщений ClientHello/ServerHello. Данное расширение называется *supported_versions*, а его наличие является одним из признаков того, что клиент (или сервер) будут использовать версию TLS 1.3. Это важная особенность: **способ обозначения версий теперь перенесён на другой логический уровень**; старые поля с номерами версий остались на своих местах, однако больше не определяют номер версии и содержат значения, не относящиеся к TLS 1.3 (и, видимо, к следующим версиям). Со стороны клиента, в ClientHello, supported_versions включает список версий протокола, которые готов поддерживать клиент. Сервер передаёт в этом расширении номер выбранной версии. Данный механизм позволяет сохранить возможность использования предыдущих версий протокола. (Экспериментальные реализации TLS 1.3 при указании версии использовали специальный префикс 0x7F в supported_versions, это касается и сервера, и клиента.)

Сравним новую схему с TLS 1.2. Кроме уже упомянутого выше отсутствия сигнала ChangeCipherSpec, отсутствуют сообщения ClientKeyExchange, ServerKeyExchange и ServerHelloDone. При этом параметры, необходимые для построения криптографического контекста, из ClientKeyExchange и ServerKeyExchange перекочевали в расширения ClientHello и ServerHello: это key_share, supported_groups, а также signature_algorithms, psk_key_exchange_modes и pre_shared_key. В типичной ситуации, непосредственно для выработки криптографического контекста используются расширения supported_groups и key_share: первое содержит идентификаторы групп, используемых для протокола Диффи-Хеллмана; второе - открытый ключ или несколько открытых ключей (взятых в группах, перечисленных в supported_groups). Так как установление соединения TLS 1.3 проводится в защищённом режиме, расширения, содержащие криптографические параметры, необходимы. Более того, их наличие (вместе с supported_versions) - является необходимым признаком TLS 1.3. Криптографические операции TLS 1.3 рассмотрим подробно в специальном разделе.

Сообщение-сигнал ServerHelloDone (оно в предыдущих версиях имело нулевую длину и обозначало окончание первой партии серверных сообщений Handshake) заменено на серверное Finished.

Даже полный формат Handshake в 1.3 оказывается короче на одну итерацию: если взглянуть на [предыдущую версию](#), то там, после получения ответа сервера (заканчивающегося ServerHelloDone), клиент должен был отправить свою порцию сообщений, завершающуюся Finished, и дождаться ответного серверного Finished, только после этого переходить к передаче полезной нагрузки. В новой версии - серверное сообщение Finished приходит в первом же ответе сервера,

соответственно, клиент может сразу переходить к отправке полезной нагрузки (после своего Finished, конечно), не дожидаясь ещё одного пакета от сервера. Это экономит время, необходимое для доставки пакетов от клиента к серверу (и, вообще говоря, обратно). Такая же схема была предложена (и реализована) раньше под названием TLS False Start ([RFC 7918](#)), в версии 1.3 она, в доработанном виде, вошла в спецификацию самого протокола TLS. Для сервисов, устанавливающих тысячи TLS-соединений в секунду, уменьшение задержки в каждом сеансе на 30-100 мс, соответствующих времени передачи данных, очень существенно. Такая экономия ресурсов ускоряет работу сервиса *для всех клиентов*, а не только для поддерживающих TLS 1.3.

ClientHello 1.3

Среди нововведений версии 1.3 - запрет "пересогласования" соединения (renegotiation). Клиент больше не может отправить сообщение ClientHello внутри открытой TLS-сессии - это приведёт к прекращению сессии, так как является нарушением спецификации. Соответственно, отправка ClientHello возможна только при инициировании новой сессии. Зато на стороне сервера появилось сообщение HelloRetryRequest, которое может быть отправлено в сторону клиента и означает предложение повторного согласования сессии. При помощи HelloRetryRequest сервер может "мягко" сбрасывать несоответствующие требованиям запросы клиентов, ожидая, что в ответ клиент передаст скорректированное сообщение ClientHello.

Это новая для TLS концепция. В частности, HelloRetryRequest позволяет бороться с некоторыми видами активных атак при помощи cookie-сигнала, так как отправка cookie позволяет проверить, что клиент действительно отвечает по адресу, указанному в качестве адреса-источника запроса. Cookie в TLS 1.3 передаются в составе расширения сообщения HelloRetryRequest и имеют логическое сходство с известными куки-файлами в HTTP (или с менее известными cookie-сигналами из расширений TCP, где они используются для борьбы с SYN-флудом и другими видами флуда). Cookie представляет собой некоторый идентификатор (тикет), который сервер передаёт клиенту для того, чтобы клиент в следующем сообщении вернул данный тикет. Спецификация 1.3 предписывает использование конкретного cookie только в рамках одного TLS-соединения.

Схема согласования криптографического контекста в 1.3 делает шифронаборы, указываемые в ClientHello, несовместимыми с предыдущими версиями TLS. Однако клиент, готовый поддерживать предыдущие версии, может передавать в ClientHello и "новые", и "старые" варианты - формат идентификатора сохранился, это двухбайтовое значение. Понятно, что для установления соединения версии 1.3 - потребуется хотя бы один совместимый шифронабор.

Рекомендуемый список шифронаборов невелик: TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_CCM_SHA256, TLS_AES_128_CCM_8_SHA256. Здесь есть режим GCM, который широко используется с шифром AES. Есть режим CCM (Counter with CBC-MAC) - это режим счётчика (как и GCM), в котором аутентификация построена на коде CBC-MAC (разновидность кода аутентификации, по схеме построения совпадающая с режимом шифрования CBC). А также шифр ChaCha20, со схемой аутентификации Poly1305 (этот комплект криптопримитивов достаточно широко используется Google, например, он поддерживается браузером Chrome). Все упомянутые криптопримитивы давно используются в TLS на практике. (Упомянутые шифры, режимы их использования и схемы аутентификации - рассмотрены в специальных разделах ниже.)

TLS 1.3 разрешает исключительно аутентифицированное шифрование, которое не требует использования дополнительной хеш-функции для вычисления кода аутентификации сообщения (в схемах аутентифицированного шифрования некоторый аналог хеш-функции встроен в саму схему). Тем не менее, хеш-функция требуется для вычисления сессионных симметричных ключей. Поэтому в шифронабор TLS 1.3 входит указание на тип хеш-функции: как видно из приведённого в предыдущем абзаце списка, это SHA-256 и SHA-384.

Таким образом, если устанавливается TLS-соединение без использования криптосистем подписи в целях аутентификации, то указание на такие криптосистемы вовсе не передаётся узлами. Соответственно, оказывается невозможным использование сертификатов для аутентификации узлов (в первую очередь, сервера). Если сертификаты используются, то необходимо указание совместимой с ними криптосистемы.

Перечень поддерживаемых криптосистем электронной подписи передаётся в обособленном расширении ClientHello - SignatureAlgorithms. Аналогичное по назначению расширение уже было введено в спецификации TLS 1.2, однако в TLS 1.2 объявляемые в составе ClientHello шифронаборы тоже содержат указание на криптосистему электронной подписи, то есть, присутствует некоторое дублирование. В 1.3 введено ещё одно расширение, уточняющее перечень схем подписи по отношению к сертификатам - это расширение SignatureAlgorithmsCerts, определяющее криптосистемы подписей в сертификатах. (Возможно использование только единственного расширения SignatureAlgorithms, которое является обязательным.)

Спецификация указывает несколько криптосистем: два варианта подписей, базирующихся на задаче RSA, широко распространённую ECDSA и новую криптосистему EdDSA, которая тоже работает на эллиптической кривой. [RFC](#) для последней опубликован в начале 2017 года. EdDSA - довольно прогрессивная

криптосистема, построенная на задаче дискретного логарифмирования (схема подписи Шнорра, математически сходная с ECDSA) и работающая на эллиптических кривых в форме Эдвардса (откуда название). Кривые Эдвардса, при некоторых дополнительных условиях, оказываются очень эффективны при использовании в составе криптосистем электронной подписи.

Рассмотрим пример ClientHello TLS 1.3. Это сообщение браузера Chrome версии 68:

Смещение (десятичное)	Байты (шестнадцатеричная запись)	Комментарий
0000	16	; Заголовок TLS-записи, тип - Handshake.
0001	03 01	; Версия - используется значение 0x0301, то есть
		; TLS 1.0, для TLS 1.3 - версия
указывается в		; расширении SupportedVersions, а
данное поле		; имеет только "историческое
значение".		; Длина данных TLS-записи: 0x200 =
0003	02 00	; длина записи выравнивается под
512 байтов,		; байтов расширения Padding (см.
нужную границу при помощи		; не является необходимым.
ниже). Такое выравнивание		
0005	01	; Тип сообщения - ClientHello.
0006	00 01 FC	; Длина данных сообщения: 0x1FC =
508		
0009	03 03	; Номер версии ("историческое
значение": 0x0303 - TLS 1.2).		
0011	16 3D 4D CB 35	; Клиентское значение Random, 32
байта (полностью не приводится).		
[...]		
0043	20	; Длина поля SessionID: 0x20 = 32
байта (типовая длина).		
0044	93 13 EA E9 95	; Значение поля SessionID, 32 байта
(полностью не приводится).		
[...]		; В TLS 1.3 SessionID может
отсутствовать. Даже если данное поле		; используется, оно не служит
указанием на номер сессии - его роль		; чисто сигнальная; в частности,
сервер должен ответить ServerHello		; с тем же значением SessionID,
которое было передано клиентом (это		; строгое требование).

0076	00 22	; Длина списка шифронаборов: 0x22 =
34 байта, или 16 шифронаборов.		
0078	8A 8A	; Начало списка. Первый элемент
имеет значение 0x8A8A. Это так		
есть, специальное значение, которое		; называемое Grease-значение - то
сервером. Передача таких значений		; заведомо не поддерживается
серверы, работающие с ошибками, так		; браузером позволяет обнаружить
игнорировать неизвестные идентификаторы		; как спецификация предписывает
регламентируется TLS, это особенность		; шифронаборов. Наличие Grease не
Chrome/Chromium. Данные Grease имеют определённый		; браузеров линейки
шифонаборов, оба байта должны иметь одинаковое		; формат. Так, в случае
"полубайт" равен 0xA.		; значение, при этом младший
0080	13 03 13 01 13 02	; Три идентификатора шифронаборов,
совместимых с TLS 1.3. А именно:		
TLS_CHACHA20_POLY1305_SHA256;		; 0x1303 =
		; 0x1301 = TLS_AES_128_GCM_SHA256;
		; 0x1302 = TLS_AES_256_GCM_SHA384.
порядке предпочтения.		; Шифронаборы клиент указывает в
0086	CC A9 CC A8 C0 2B	; Далее следуют идентификаторы
шифонаборов, совместимые с другими		
C0 2F C0 2C C0 30		; версиями TLS. Их указание
позволяет попытаться установить		
C0 13 C0 14 00 9C		; соединение TLS 1.2 с серверами,
которые не поддерживают TLS 1.3.		
00 9D 00 2F 00 35		; Например, 0xCCA8 =
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256.		
00 0A		
0114	01 00	; Строго зафиксированное значение,
соответствующее длине и значению		
поле может иметь только значение		; алгоритмов сжатия: в TLS 1.3 это
ноль (сжатие спецификацией		; 0x01 (длина один байт), 0x00 -
		; не предусмотрено).
ClientHello. Некоторые расширения строго		; Расширения (Extensions)
SupportedVersions.		; необходимы - например,

0116 01 91 401 байт.	; Длина блока расширений: 0x191 =
0118 2A 2A 00 00 Grease для шифронаборов. Данное расширение старшими адресами; первые два байта -	; Расширение Grease, аналогично ; имеет нулевую длину (два байта со ; это тип расширения: 0x2A2A).
0122 FF 01 0xFF01, RenegotiationInfo. Это расширение ранних версиях оно используется для "пересогласования" TLS-соединения.) 00 01 00 значений обозначает, что сессия TLS 1.3 расширение просто будет	; Тип следующего расширения - ; не относится к TLS 1.3. (В более ; повышения безопасности ; Длина - один байт. ; Значение: 0 (эта комбинация ; только устанавливается; однако в ; проигнорировано сервером).
0127 00 00 0x0000). Расширение содержит имя сервера, обращение. (См. описание выше.) 00 10 00 0E 00 Значение 0 - символьное имя хоста. 00 0B байтов) 74 6C 73 31 33 2E ASCII: в данном случае - tls13.1d.pw. 31 64 2E 70 77	; Расширение ServerName (тип ; к которому производится ; Длина данных (0x10 = 16 байтов). ; Длина элемента списка. ; Тип записи (элемента списка). ; Длина строки с именем (0x0B = 11 ; Значение имени, записанное в
0147 00 17 00 00 0x0023), имеющие нулевую длину. 00 23 00 00 расширениями TLS 1.3.	; Два расширения (типы: 0x0017, ; Эти расширения не являются ; 0x0017 = ExtendedMasterSecret; ; 0x0023 = SessionTicket.
0155 00 0D расширение TLS 1.3 SignatureAlgorithms. поддерживаемых криптосистем перечень называется SignatureScheme). 00 14	; Расширение с типом 0x000D. Это ; В расширении содержится перечень ; электронной подписи (этот ; Длина данных (0x14 = 20).

00 12 криптосистем). Далее идёт список каждый из которых обозначает криптосистему. идентификаторы в порядке предпочтения): 04 03 08 04 04 01 05 03 08 05 05 01 08 06 06 01 02 01 0179 00 05 не является исключительным для TLS 1.3, версиями протокола. 00 05 01 00 00 00 00 запросить статус серверного сертификата. проверка того, не отозван ли сертификат. копия ответа респондера УЦ (OCSP), сертификата. Эту копию может вернуть сервер 0188 00 12 SignedCertificateTimestamp, тоже является предыдущих версиях протокола. привязывать серверные сертификаты к публичным работающих в рамках инициативы рассмотрение которой находится за рамками 00 00 является сигналом браузера о поддержке 0192 00 10 (Application Layer Protocol Negotiation). для быстрого согласования протокола, поверх TLS.	; Длина элемента списка (перечня ; двухбайтовых идентификаторов, ; Список (браузер передаёт ; ecdsa_secp256r1_sha256; ; rsa_pss_rsae_sha256; ; rsa_pkcs1_sha256; ; ecdsa_secp384r1_sha384; ; rsa_pss_rsae_sha384; ; rsa_pkcs1_sha384; ; rsa_pss_rsae_sha512; ; rsa_pkcs1_sha512; ; rsa_pkcs1_sha1. ; Расширение 0x0005 StatusRequest, ; оно используется и с другими ; Длина 5 байтов. ; Это расширение позволяет клиенту ; Под "статусом" подразумевается ; В данном случае запрашивается ; подтверждающего статус ; в составе расширений ServerHello. ; Расширение с типом 0x0012 - ; расширением, используемым и в ; Это расширение позволяет ; реестрам (логам) сертификатов, ; Certificate Transparency, ; данного описания. ; Нулевая длина: расширение ; технологии. ; 0x0010 - расширение ALPN ; Данное расширение предназначено ; который будет использоваться
--	---

00 0E	; Длина данных (0x0E = 14 байтов).
00 0C	; Длина списка с идентификаторами.
02	; Длина записи (первого)
идентификатора, два байта.	
68 32	; Идентификатор протокола HTTP2,
строка "h2".	
08	; Длина записи (второго)
идентификатора, восемь байтов.	
68 74 74 70 2F 31 2E 31	; Идентификатор протокола HTTP/1.1,
строка "http/1.1".	
0210 75 50	; Расширение с типом 0x7550 обычно
называется Channel_ID и используется	
это "проприетарное" расширение TLS	; Google Chrome. Условно говоря,
связано с тем протоколом, который позже	; с временным типом. Расширение
8471).	; стал Token Binding Protocol (RFC
00 00	; У этого расширения нулевая длина
данных.	
0214 00 0B	; Расширение 0x000B -
ECPointFormats. Это расширение определяет формат	; записи точек эллиптической
кривой, который поддерживает клиент.	
расширением TLS 1.3, но используется	; Это расширение не является
00 02	; с версией 1.2 (см. выше).
01 00	; Длина: два байта.
0x0100, единственно допустимый.	; Значение: несжатый формат -
0220 00 33	; Одно из важных расширений версии
TLS 1.3. 0x0033 - KeyShare.	
открытый параметр протокола DH клиента.	; Данное расширение содержит
установлении соединения, так как часть ответа сервера	; Параметр необходим при
симметричных ключей базируется на протоколе	; будет зашифрована, а генерация
00 2B	; Диффи-Хеллмана.
быть перечислено несколько открытых	; Длина данных. В расширении может
определённой группе, в которой работает DH.	; ключей, каждый соответствует
00 29	; Длина списка. В данном примере
передаётся два параметра.	
7A 7A	; Идентификатор группы первого
параметра. Указано Grease-значение 0x7A7A.	
00 01	; Длина данных параметра (один байт
- для Grease-значения это допустимо).	

00		; Сам "параметр" - нулевой байт.
00 1D		; Идентификатор группы второго
параметра. 0x001D соответствует X25519.		
байтовые ключи.		; Эта криптосистема использует 32-
00 20		; Длина данных ключа - 0x0020 = 32
байта.		
C4 EC		; Далее следует открытый параметр
DH для X25519 (полностью не приводится).		
[...]		
0267	00 2D	; Расширение с типом 0x002D
PSKKeyExchangeModes. Это расширение TLS 1.3,		
сервер используют режимы PSK, то есть,		; управляющее тем, какие клиент и
возобновления) соединения в условиях,		; режимы установления (обычно -
общий секрет (например, согласованный		; когда уже есть дополнительный
сессии).		; сторонами в предыдущей TLS-
кодом. В данном примере клиент передаёт		; Режимы обозначаются однобайтовым
данное расширение передавать не должен, это		; единственный вариант. (Сервер
00 02		; запрещено спецификацией.)
01		; Длина данных.
01		; Длина списка (один элемент).
(режим, включающий протокол Диффи-Хеллмана).		; Код режима: 01 - psk_dhe_ke
0273	00 2B	; Необходимое расширение TLS 1.3,
которое и делает данное сообщение		
1.3. 0x002B обозначает тип SupportedVersions,		; ClientHello - сообщением версии
TLS 1.3 список версий определяется строго		; то есть, поддерживаемые версии. В
указания на версии протокола - имеют лишь статус		; этим расширением: все прочие
00 0B		; "исторических".
Данные расширения представляют собой список		; Длина данных (0x0B = 11 байтов).
версий.		; двухбайтовых идентификаторов
0A		; Длина списка.
0A 0A		; Grease-значение версии.
7F 17		; 0x7F17 - обозначает версию draft-
23 TLS 1.3. То есть, это экспериментальная		; версия, использовавшаяся до того,
как вышла спецификация. Более того, данная		; версия имеет существенные отличия
от окончательной спецификации, и с TLS 1.3		; в версии RFC несовместима.

Окончательная версия TLS 1.3 имеет идентификатор
; 0x0304.
; Далее следуют прочие версии TLS,
которые поддерживает браузер Chrome.
03 03 ; TLS 1.2
03 02 ; TLS 1.1
03 01 ; TLS 1.0

0288 00 0A ; Другое важное (и необходимое)
расширение TLS 1.3 - SupportedGroups, тип 0x000A.
; В данном расширении клиент
передаёт идентификаторы групп, которые он поддерживает
; в качестве основы протокола
Диффи-Хеллмана. Список групп соответствует
; передаваемым открытым параметрам
DH (см. расширение KeyShare). Однако групп может
; быть указано больше, чем передано
ключей. Это означает, что сервер может
; инициировать новый раунд
установления TLS-соединения, запросив DH конкретно
; в какой-либо из поддерживаемых
групп.
00 0A ; Длина данных (0x0A = 10 байтов).
00 08 ; Длина списка (8 байтов). Далее
идут идентификаторы групп, которые поддерживает
; браузер Chrome (это только группы
точек эллиптических кривых).
7A 7A ; Grease-значение.
00 1D ; X25519 = 0x001D, открытый
параметр DH X25519 передан в этом же ClientHello,
; в KeyShare;
00 17 ; Secp256r1 = 0x0017;
00 18 ; Secp384r1 = 0x0018.
; Как и в других похожих случаях,
идентификаторы передаются в порядке предпочтения
; групп клиентом.

0301 4A 4A 00 01 00 ; Grease-расширение, имеющее длину
в один байт.
0306 00 15 ; Специальное расширение с типом
0x0015 - Padding, которое предназначено для
; выравнивания длины ClientHello.
Данные представляют собой нужное
; количество нулевых байтов.
00 D0 ; Длина расширения: 0x00D0 = 208
нулевых байтов, которые завершают сообщение.
00[...]00

Итак, сервер может определить, что данное ClientHello является сообщением версии 1.3 по наличию необходимого расширения SupportedVersions. С другой стороны, сообщение выглядит как ClientHello предыдущей версии, TLS 1.2, с тем отличием,

что содержит расширения, неизвестные для сервера, поддерживающего старую версию. Неизвестные расширения допускаются в TLS: клиент и сервер должны их игнорировать. Тем не менее, не следует делать вывод, что процедуры установления соединения (Handshake) в TLS 1.3 и TLS 1.2 совместимы - это не так. Формат ClientHello позволяет клиенту сразу же провести контролируемое понижение версии протокола и *выполнить установление соединения по схеме TLS 1.2, при условии поддержки на сервере*, но если сервер поддерживает версию 1.3, то уже следующий шаг протокола будет *радикально отличаться от 1.2* - и по составу сообщений, и по их структуре. А главное отличие состоит в моменте перехода на защищённый обмен данными - в TLS 1.3 алгоритм установления соединения полностью поменялся. Это очень важный момент, про который нельзя забывать: хоть номер версии 1.3 отличается от 1.2 всего лишь на единицу, различий между этими версиями несравнимо больше, чем, например, между 1.2 и 1.1.

Расширения ClientHello/ServerHello в TLS 1.3. Некоторые расширения, введённые в предыдущих версиях, сохранились и в 1.3, фактически, с теми же ролями. Однако, как уже отмечено в пояснениях к примеру ClientHello, есть три важнейших расширения, которые и превращают сессию в сессию версии 1.3:

SupportedVersions, SupportedGroups, KeyShare. Первое - определяет версию протокола. Второе и третье - необходимы для согласования криптографического контекста. Рассмотрим ряд других расширений (версии 1.3) подробнее.

PSKKeyExchangeModes и PreSharedKey. Эти расширения предназначены для создания сессий, использующих ранее распределённые между узлами (секретные) ключи. Определяют, соответственно, поддерживаемые режимы ("прямой" режим, или режим с дополнительным использованием протокола DH) и идентификатор ключа. Очевидно, что секретный ключ не может передаваться в открытом виде, поэтому расширение PreSharedKey содержит только данные, позволяющие другой стороне найти нужный ключ у себя. Кроме того, PreSharedKey определяет срок действия ключа и ряд других параметров (криптосистему и пр.).

EarlyData. TLS 1.3 позволяет узлам возобновить соединение по максимально быстрой схеме, когда полезные данные передаются клиентом в первой же итерации. Например, клиент может сразу отправить HTTP-запрос GET. Сигналом, что клиент действует именно таким способом, служит наличие расширений EarlyData и PreSharedKey. Второе нужно для того, чтобы сервер мог выбрать ранее согласованный сторонами секрет и сгенерировать ключ, необходимый для расшифрования поступившего от клиента запроса - полезные данные всегда передаются в защищённом виде, то есть, внутри записей Application Data.

Cookie. Данное расширение, во-первых, позволяет серверу запросить у клиента подтверждение намерения установить соединение; во-вторых - позволяет оптимизировать управление сессиями на стороне сервера. Cookie может

передаваться сервером вместе с `HelloRetryRequest` и содержит некоторую последовательность байтов, которую должен повторить клиент в своём следующем сообщении `ClientHello` (тоже в составе расширения `Cookie`). То есть, роль данного сообщения сходна со многими другими применениями различных "куки". Так, с его помощью сервер может существенно снизить затраты на обслуживание "зависших" сессий, - тем самым избежав ситуации отказа, - в случае одновременного открытия многих сессий злонамеренными узлами. В целом, так как TLS является протоколом уровня приложений и находится выше транспортного уровня, защита с помощью `Cookie` может оказаться не слишком эффективной. Например, в случае с TCP, если запросы от "завешивающих" сессию узлов доходят до TLS-сервера, это может означать, что где-то на более низких уровнях борьбы с атакой допущена ошибка. TCP-сессия существует независимо от TLS, поэтому, даже если серверу не требуется "держат" открытым состояние TLS-сессии, это не означает автоматически, что будет закрыта сессия TCP. Тем более, что занять сессию TCP атакующий мог ещё до того, как были переданы какие-либо данные уровня приложений. Тем не менее, ключевым отличием здесь является то, что объёмы вычислительных ресурсов, необходимых для обслуживания открытия TCP-сессии и TLS-сессии - несравнимы: TLS требует в тысячи раз больше (понятно, что TCP придётся обслуживать так или иначе). Соответственно, `Cookie` особенно полезны, когда используется безсессионный транспорт (например, UDP), так как в этом случае на транспортном уровне отсечь атаки с массовым открытием сессий невозможно (за отсутствием понятия сессии).

Кроме перечисленных выше расширений, в TLS 1.3 есть ряд других. Например, расширение `CertificateAuthorities` позволяет информировать другой узел о списке поддерживаемых доверенных удостоверяющих центров. Предусмотрено специальное расширение-сигнал для сервера о том, что клиент поддерживает аутентификацию клиента, а также ряд других расширений, которые мы не рассматриваем.

ServerHello 1.3

`ServerHello` - ответное сообщение сервера. На `ClientHello` сервер может ответить либо `ServerHello`, либо `HelloRetryRequest`. `ServerHello` в TLS 1.3 решает те же задачи, что и в версии 1.2, однако способ решения отличается. При помощи этого сообщения сервер выбирает шифронабор из предложенных клиентом, передаёт данные, необходимые для построения общего криптографического контекста. Так как в новой версии шифрование включается практически сразу, `ServerHello` обязательно содержит (в составе передаваемых расширений) параметры, позволяющие узлам незамедлительно получить общий секрет, который послужит прообразом симметричного ключа шифрования. Сервер вычисляет общий секрет сразу же, так как уже следующие за `ServerHello` сообщения должны передаваться в

зашифрованном виде. Это важное, и весьма существенное, отличие от предыдущих версий TLS.

Как уже описано выше, в TLS 1.3 используются симметричные ключи нескольких уровней (или "этапов"). Ключи, применяемые для защиты сообщений Handshake, отличаются от ключей, используемых для защиты, собственно, трафика. Однако ключи следующих этапов связаны с ключами этапов предыдущих. Вычисление каждого поколения использует дополнительные данные, поэтому только по секретным ключам Handshake восстановить ключи защиты трафика вычислительно сложно. Для определения общего секрета используется протокол Диффи-Хеллмана (DH), клиентские параметры которого сервер уже получил в ClientHello. Источником дополнительных входных значений служат также сообщения Handshake, полученные и подготовленные к передаче сервером. Серверные параметры DH - передаются в составе ServerHello: в этом состоит ещё одно архитектурное отличие от предыдущих версий, в которых серверные DH-параметры передавались в специальном сообщении ServerKeyExchange (см. выше). Перенос параметров внутрь ServerHello, конечно, обусловлен тем, что использование протокола Диффи-Хеллмана стало обязательным.

С параметрами DH, с другими способами получения общего секрета в TLS 1.3, связан следующий момент: клиент может указать несколько вариантов поддерживаемых групп, но сами параметры передать не для всех из них. То есть, в терминах ClientHello, расширение SupportedGroups содержит больший перечень групп, чем перечень параметров в KeyShare. Это обычная ситуация в случае браузеров: как можно видеть в примере ClientHello Chrome выше - данный браузер передаёт только один (рабочий) параметр в KeyShare, это X25519, при этом список поддерживаемых групп SupportedGroups включает три элемента. Если сервер смог найти среди поддерживаемых групп те, которые совместимы с серверными настройками, но для этих групп клиент не передал параметров DH, то сервер может ответить вместо ServerHello сообщением HelloRetryRequest. Предполагается, что клиент, в таком случае, ответит новым ClientHello, передав параметры в нужной серверу группе. (Подобное использование HelloRetryRequest относится и к другим случаям, когда параметров в совместимом ClientHello серверу недостаточно для установления соединения.)

Основной особенностью сообщения HelloRetryRequest является то, что это сообщение по структуре *полностью* повторяет ServerHello, имеет такой же код типа. Отличить два сообщения клиент может *только по значению поля Random*. Это довольно интересная особенность TLS - использование значения Random в качестве сигнала. Есть и другая особенность: при наличии HelloRetryRequest и Cookie-расширения - сервер может использовать другой механизм вычисления хеш-функции от переданных и принятых в рамках установления сессии сообщений, "выгрузив" состояние соединения на сторону клиента при помощи расширения Cookie (которое,

в данном случае, требуется). Несомненно, это ещё один пример из ряда "неочевидных решений", используемых в TLS. Тем не менее, его внедрение обусловлено заботой об оптимизации производительности протокола.

Рассмотрим пример сообщения HelloRetryRequest (а на следующем шаге - ServerHello TLS 1.3). Это сообщение, полученное от одного из серверов facebook.com; соответствующее ClientHello не содержало ни одного подходящего для сервера значения в списке ключей ClientKeyShare (заголовок TLS-записи не приводится):

Смещение (десятичное)	Байты (шестнадцатеричная запись)	Комментарий
0000	02	; Тип сообщения: несмотря на то, что это сообщение HelloRetryRequest, ; указан тип 0x02, соответствующий ServerHello.
0001	00 00 54	; Длина данных. 0x54 = 84 байта.
0004	03 03	; Версия (историческое значение), 0x0303 = TLS 1.2.
0006	CF 21 AD 74 E5 9A 61 11	; Поле Random. Так как это
сообщение HelloRetryRequest, данное поле	BE 1D 8C 02 1E 65 B8 91	; содержит фиксированное значение,
обозначающее тип сообщения.	C2 A2 11 16 7A BB 8C 5E	; Значение приведено полностью, это
32 байта, значение SHA-256	07 9E 09 E2 C8 A8 33 9C	; от строки "HelloRetryRequest".
0038	20	; Длина поля SessionID, 0x20 = 32 байта.
0039	10 20 30	; Значение SessionID. Сервер
возвращает копию значения, полученного	[...]	; в этом поле ClientHello
(полностью не приводится).		
0071	13 01	; Идентификатор выбранного сервером шифронабора:
		; 0x1301 = TLS_AES_128_GCM_SHA256.
0073	00	; Историческое значение, обозначающее отсутствие сжатия.
0074	00 0C	; Длина блока расширений
ServerHello. 0x0C = 12 байтов.		
0076	00 2B	; Расширение 0x002B -
SupportedVersions. Сервер передаёт значение версии		

		; протокола, которое было выбрано.
0078	00 02 03 04	; Длина записи версии (два байта) и
сама запись: 0x0304 = TLS 1.3.		
0082	00 33	; Расширение 0x0033 - KeyShare. Мы
рассматриваем сообщение HelloRetryRequest.		
расширение KeyShare, сервер передаёт только		; В этом сообщении, используя
Диффи-Хеллмана, для которой он ожидает		; идентификатор группы протокола
ClientHello.		; получить клиентский ключ в новом
0084	00 02 00 1D	; Длина записи (два байта) и
запись, обозначающая группу X25519 (0x001D).		

Итак, HelloRetryRequest повторяет структуру ServerHello, но последнее, тем не менее, отличается по интерпретации клиентом. Перейдём к ServerHello TLS 1.3. Здесь приводится ServerHello, полученное от сервера Google, обслуживающего веб-сервис под доменом gmail.com. Соответствующее ClientHello содержало ключи для всех трёх "эллиптических вариантов" DH, сервером выбрана группа X25519. Разбор сообщения (заголовок TLS-записи не приводится):

Смещение (десятичное)	Байты (шестнадцатеричная запись)	Комментарий
0000	02	; Тип - ServerHello = 0x02.
0001	00 00 76	; Длина данных: 0x000076 = 118
байтов.		
0004	03 03	; Историческая версия (актуальная -
передаётся в расширении SupportedVersions).		; 0x0303 = TLS 1.2.
0006	CD F3 1F DD 1B 59 A5 E5	; 32 байта Random (полностью не
приводится).		
значениями, так как в ServerHello роль данного поля		; Это байты со случайными
серверной части случайного набора данных, который		; состоит именно в передаче
сеансовых ключей.		; позже войдёт в состав основы
[...]		
0038	20	; Длина данных SessionID, 0x20 = 32
байта.		
служит только для передачи значения,		; Как описано выше, SessionID здесь

		; полученного в ClientHello.
0039	DE AD DE	; 32 байта значения SessionID,
полностью не приводятся.		
[...]		
0071	13 01	; Идентификатор выбранного сервером
шифронабора:		
		; 0x1301 = TLS_AES_128_GCM_SHA256.
0073	00	; Фиксированное нулевое значение,
один байт: нет сжатия.		
0074	00 2E	; Длина расширений ServerHello.
0x2E = 46 байтов.		
0076	00 33	; Расширение 0x0033 = KeyShare. В
этом расширении сервер передаёт открытый		
		; серверный ключ протокола Диффи-Хеллмана в выбранной группе.
0078	00 24	; Длина данных расширения, 0x24 =
36 байтов.		
0080	00 1D	; Идентификатор используемой
группы: 0x001D - это X25519.		
0082	00 20	; Длина записи ключа. Для X25519
запись ключа требует 32 байта (0x20).		
		; Значение ключа передаётся в
следующем поле.		
0084	38 9B E0 44	; 32 байта ключа X25519.
[...]		
0116	00 2B	; Расширение SupportedVersions =
0x002B.		
0118	00 02 03 04	; Длина (два байта) и значение:
0x0304 = TLS 1.3.		
поддерживаемых версий, в том числе, несколько		; Клиент мог предложить несколько
передаёт единственный номер, соответствующий		; draft-версий TLS 1.3. Сервер
бы сервер (теоретически) выбрал версию ниже		; выбранной версии. В случае, если
такой в предложенном клиентом списке), то данное		; TLS 1.3 (при условии наличия
На практике, сервер, поддерживающий TLS 1.3,		; расширение не использовалось бы.
должен пытаться согласовать именно 1.3.		; встретив совместимое ClientHello,

Попробуем поместить ServerHello версии 1.3 и ServerHello предыдущих версий в единый логический контекст, чтобы выявить основные различия. Сразу же можно заметить, что в версии 1.3 клиент предлагает серверу возможные параметры DH. В предыдущих версиях - инициатива была за сервером, клиент мог только ответить сообщением ClientKeyExchange. Благодаря изменению ролей, узлы уже после передачи ServerHello получают общий секрет. Передача случайных значений Random, списков шифронаборов - тут сохраняется логика предыдущих версий. В предыдущих версиях присутствовали и расширения, однако в 1.3 роль их стала значительно важнее, хотя бы потому, что именно наличие определённых расширений (SupportedVersions и др.) делает ServerHello сообщением версии 1.3.

Защищённые сообщения Handshake в TLS 1.3

ServerHello, при штатном ходе установления соединения, является единственным сообщением сервера, которое передаётся в открытом виде. Уже следующее сообщение - EncryptedExtensions - зашифровано. Зашифрованные сообщения в TLS 1.3 передаются строго внутри TLS-записей с типом Application Data (0x17). Это ещё одно важное отличие от предыдущих версий. Мы подробно рассмотрим формат защищённых записей ниже, в специальном подразделе, а сейчас отметим только один момент: "настоящий" тип записи - Handshake (0x16) - передаётся внутри защищённой записи, и доступен только после того, как эта запись успешно расшифрована.

Для использования шифров с целью защиты TLS-трафика, прежде всего, нужны симметричные ключи, а кроме них - так называемые "векторы инициализации", то есть, наборы байтов, которые необходимы для реализации режима использования шифров: дело в том, что в TLS шифры не применяются в "наивном режиме", когда блок открытого текста прямо зашифровывается, без дополнительных преобразований. (Этот момент подробно рассмотрен в разделе, посвящённом шифрам в TLS.) Используя общий секрет, полученный на начальном этапе, а также все переданные к настоящему моменту сообщения, узлы определяют текущие симметричные ключи. Предположим, что сервер выбрал шифронабор TLS_AES_128_GCM_SHA256 (0x1301). Тогда, для успешной защиты сообщений, потребуется пара симметричных ключей: будем обозначать их как sh_write, sh_read (от Server Handshake Write/Read). Длина этих ключей - 128 бит, соответствует разрядности ключей шифра. Обозначения write и read нужны потому, что первый ключ используется для *записи* сообщений в канал, то есть, для отправки их в сторону клиента, а второй - для чтения. На стороне клиента - ключи меняются местами. Схема использования шифров тут полностью аналогична [описанной выше](#). Кроме ключей, узлы должны определить векторы инициализации шифров. В данном случае, это начальные значения, необходимые для работы шифра в режиме

аутентифицированного шифрования (то есть, с встроенной в режим проверки целостности данных).

Сам алгоритм вычисления ключей и векторов инициализации в TLS 1.3 отличается от предыдущих версий. Для получения нужной "порции битов" используется функция, которая в спецификации называется HKDF-Expand-Label:

Для ключа:

```
key = HKDF-Expand-Label(Secret, "key", "", key_length)
```

Для вектора инициализации (iv):

```
write_iv = HKDF-Expand-Label(Secret, "iv", "", iv_length)
```

Функция возвращает нужное количество битов ключа или вектора инициализации - их длины, соответственно, передаются в `key_length` и `iv_length`. Рассмотрим другие аргументы:

`Secret` - это секрет, соответствующий текущему этапу протокола, он, например, отличается для установления соединения и для этапа обмена полезными данными;

`"key"/"iv"` - текстовые строки, обозначающие предназначение ключевой информации. Строки вводятся для того, чтобы результаты дополнительно различались, а разработчики, реализующие протокол, имели меньше шансов на ошибку. Подобные текстовые строки используются в TLS 1.3 и во всех других случаях генерации ключей.

Функция HKDF-Expand-Label устроена несколько сложнее, чем может показаться: внутри скрываются другие функции, которые ставят генерируемую информацию в соответствие с текущим контекстом. То есть, симметричные ключи в TLS 1.3 зависят от множества факторов, что, например, позволяет на уровне протокола защититься от некоторых логических ошибок в программной реализации.

Вернёмся к процессу установления соединения. Следом за `ServerHello` сервер обязательно передаёт сообщение **EncryptedExtensions**. Это логический блок, в который помещаются те расширения `ServerHello`, которые спецификация позволяет передавать в зашифрованном виде. Нередко сообщение `EncryptedExtensions` не содержит ни одного расширения, однако в таком случае необходимо передать пустое сообщение (то есть, состоящее только из заголовка).

Примеры `EncryptedExtensions` (обратите внимание, что здесь, по очевидным причинам, приводится расшифрованное сообщение; в реальных сессиях - это сообщение передаётся в защищённой TLS-записи с "историческим" типом `0x17`; такой тип используется потому, что в TLS 1.3 он назначается всем защищённым записям, а подлинный тип записи - содержится внутри защищённых данных):

Смещение (десятичное)	Байты (шестнадцатеричная запись)	Комментарий
(Пустое сообщение EncryptedExtensions.)		
0000	08	; Тип сообщения: 0x08 =
EncryptedExtensions.		
0001	00 00 02	; Длина: 2 байта.
0004	00 00	; Пустая строка (строка нулевой
длины, то есть только два байта,		; обозначающих нулевую длину).

Смещение (десятичное)	Байты (шестнадцатеричная запись)	Комментарий
(Сообщение EncryptedExtensions, содержащее два расширения.)		
0000	08	; Тип сообщения: 0x08 =
EncryptedExtensions.		
0001	00 00 16	; Длина данных: 0x16 = 22 байта.
0004	00 14	; Длина списка расширений: 0x14 =
20 байтов.		
0006	00 00	; Тип расширения: 0x0000 =
ServerName (передано пустое расширение).		
0008	00 00	
0010	00 0A	; Тип расширения: 0x000A =
SupportedGroups. Это расширение содержит		; перечень групп, поддерживаемых
сервером (см. описание выше,		; в ClientHello).
0012	00 0C	; Длина данных (0x0C = 12 байтов).
0014	00 0A	; Длина списка (0x0A = 10 байтов).
0016	00 1D 00 17 00 1E 00 19 00 18	; Перечень идентификаторов групп.

За EncryptedExtensions может последовать либо сообщение CertificateRequest (запрос сертификата клиента), либо Certificate - последнее содержит серверный TLS-сертификат. За исключением того, что эти сообщения зашифрованы, они аналогичны соответствующим сообщениям в предыдущих версиях. Передача сообщения Certificate сервером требует передачи и сообщения CertificateVerify.

В сообщении ClientHello клиент передаёт расширение SignatureAlgorithms, содержащее список поддерживаемых криптосистем электронной подписи. Одна из этих криптосистем используется сервером для вычисления значения CertificateVerify - сообщения, решающего сразу две важных задачи: во-первых, это сообщение

позволяет клиенту проверить, что у сервера действительно есть секретный ключ от представленного сертификата; во-вторых, оно удостоверяет содержание полученных к этому моменту на стороне сервера других сообщений Handshake (перечень и содержание отправленных клиентом сообщений клиент знает и так). Очевидно, что список поддерживаемых криптосистем подписи должен включать криптосистему, совместимую с сертификатом, так как вычисление подписи не от ключа сертификата - не имеет никакого смысла. То есть, CertificateVerify прежде всего следует рассматривать как основополагающую часть механизма аутентификации сервера клиентом. TLS-сертификат является публичным электронным документом, поэтому всякий может скопировать серверный сертификат и, позже, передавать его подключающимся клиентам. Однако наличие актуальной подписи лишает такой ход смысла.

Подпись CertificateVerify вычисляется от всего набора полученных (к моменту вычисления) сервером сообщений - откуда и возникает инструмент раннего удостоверения подлинности сессии: если между клиентом и сервером находится активный атакующий, который модифицирует сообщения, то клиент обнаружит расхождение подписи, так как проверяет её для своей копии *переданных* сообщений. Это существенное улучшение ситуации по сравнению с предыдущими версиями протокола, где подобное удостоверение подлинности становилось возможным только в самом конце, после получения сообщения Finished.

В предыдущих версиях TLS аналогичное по значению сообщение передавал только клиент, в случае своей аутентификации сервером. В отношении же самого сервера, в зависимости от используемой схемы генерации общего секрета, предполагалось, что только узел, располагающий секретным ключом, сможет расшифровать переданные клиентом данные, либо о наличии секретного ключа можно судить по подписи на сообщении ServerKeyExchange. При этом, подпись на ServerKeyExchange не защищает прочие сообщения Handshake. (В TLS 1.3 сообщения ServerKeyExchange нет вообще.)

Сообщение CertificateVerify имеет очевидную структуру: оно состоит из идентификатора криптосистемы подписи и самой подписи. Важнее посмотреть на то, как именно, и какие данные подписываются. Электронная подпись на практике вычисляется для значения хеш-функции от подписываемого блока данных. В случае CertificateVerify исходными данными служат все сообщения, полученные и переданные сервером к моменту генерирования подписи, а именно: сообщения, начиная от ClientHello и до, включительно, EncryptedExtensions или CertificateRequest; плюс - в цепочку добавляется сообщение Certificate. Обратите внимание, что в цепочку сообщений входят и HelloRetryRequest, если оно отправлялось сервером, и *первое* ClientHello. От объединения сообщений вычисляется значение хеш-функции, которое помещается в специальную структуру, и уже эта структура поступает на вход криптосистемы электронной подписи.

Клиент проверяет подпись `CertificateVerify` при помощи открытого ключа, содержащегося в серверном сертификате - этот ключ известен из сообщения `Certificate`.

Завершает набор серверных сообщений - серверное **Finished**. Это сообщение, как и в TLS предыдущих версий, содержит отпечаток всех известных серверу сообщений Handshake, которые предшествовали Finished. Finished является HMAC (кодом аутентификации с хеш-функцией), использующей секретный ключ. Этот ключ, в свою очередь, стороны вычисляют на основе имеющегося у них секрета. Обратите внимание, что Finished передаётся в зашифрованной TLS-записи и уже защищено кодом аутентификации используемого режима шифрования. Именно в таком режиме передаётся и Finished предыдущих версий протокола (см. выше). То есть, здесь логика хорошо совпадает, за исключением того, что в TLS 1.3 серверное Finished передаётся *на шаг раньше*.

В TLS 1.3 исключены сообщения-сигналы **ChangeCipherSpec** (CCS), которые в более ранних версиях обозначали момент перехода в защищённый режим обмена данными. Однако экспериментальное внедрение TLS 1.3 ещё на ранних стадиях (речь про draft-версии) показало, что в ряде случаев TLS-соединение без CCS распознаётся разными системами инспекции трафика (DPI) как подозрительное (или "некорректное"), и, соответственно, разрывается. К сожалению, использование подобных "практик", а также оборудования DPI (и DLP), спроектированного с ошибками, является нормой для корпоративных сетевых сред, поэтому в TLS пришлось внести соответствующие дополнения: сигнал CCS вернулся в качестве фиктивного сообщения, которое никак не учитывается узлами, но позволяет "обмануть" DPI, делая соединение возможным.

Клиентская часть сообщений Handshake завершается Finished. После того, как стороны проверили Finished и убедились, что значения корректны, сессия считается установленной, а узлы переходят на новое поколение ключей.

Управление сеансовыми ключами в TLS 1.3

В TLS 1.3 используется многоэтапный подход к управлению симметричными ключами. Его логика строится на следующих предложениях: 1) каждая фаза протокола должна использовать свой набор ключей; 2) значения ключей должны зависеть не только от общего секрета и параметров Random, но и от других свойств конкретной сессии; 3) каждый следующий набор ключей зависит от предыдущего и *дополнительной* информации.

Основу для общего секрета составляет значение, полученное в рамках начального этапа обмена: здесь может быть использован как протокол DH, так и, в новых реализациях, гибридные схемы с постквантовой стойкостью (ML-KEM+X25519), а в

качестве дополнения - секретное значение, которое узлы согласовали каким-то способом ранее (схемы PSK). Поколения ключей соответствуют ходу соединения, при этом вычисление каждого поколения ключей основано на соответствующем поколении общего секрета. Схема не очень сложная, представляет собой ступенчатый переход от секрета к секрету, с подмешиванием новой информации на каждом шаге. Подмешиваемая информация представляет собой текстовые строки и сообщения протокола, которые преобразуются при помощи специальной функции. В предыдущих версиях TLS ничего похожего не было. Однако механизм не является сколь-нибудь уникальным или особенным: есть немало криптографических протоколов, где применяется сходный подход, в качестве примера куда более сложного развития этой же идеи можно привести протокол Signal, который, как утверждается в документации, использует мессенджер WhatsApp (и ряд других приложений).

Рассмотрим упрощённую схему генерации ключей. Этапы идут сверху вниз, начиная от "пустого" значения, обозначенного 0 (в протоколе это *строка* нулевых байтов заданной длины). Центральная вертикальная линия соответствует движению ключевого материала, где к нему последовательно применяется функция генерации ключей (HKDF). Слева - показаны основные источники входных данных с ключевой информацией. Справа - промежуточные результаты (общие секреты), сообщения Handshake, используемые в качестве дополнительно подмешиваемой информации, и симметричные "этапные" секреты, которые получаются на каждом шаге. Подмешивание информации происходит через последовательное вычисление значений хеш-функций и объединения (конкатенации) входных сообщений. Пример: если на входе имеется пара сообщений (ClientHello, ServerHello), а текущий этап является этапом Handshake (II - на схеме; указан Handshake Secret), то байты сообщений, в том порядке, как они передавались на транспортном уровне, объединяются в одну строку, от этой строки вычисляется хеш-функция, к результату присоединяется строка символов "с hs traffic", которая обозначает этап, а получившийся новый набор байтов, вместе с общим секретом, служит аргументом для генерации набора ключей при помощи функции Derive-Secret(). В реальном протоколе отличаются некоторые детали, в частности, используются специальные "обёртки" вокруг хеш-функций, но алгоритмическая логика - точно такая, как описано.

0	
v	
PSK -> HKDF = Early Secret	; Самый ранний
этап. Здесь используется общий секрет,	
	; возможно,
полученный до установления соединения (PSK).	
+-----> [ClientHello]	; При отсутствии
такого секрета - используется пустая	
	; строка.

I)		v	; ClientHello -
служит дополнительной информацией,		binder_key,	; получают наборы
секретов binder_key и т.д.		client_early_traffic_secret,	
		early_exporter_master_secret.	
	v		
Derive-Secret(., "derived", "")			; Переход между
этапами сопровождается трансформацией секрета.			
	v		
(EC)DHE -> HKDF = Handshake Secret			; Следующий этап -
Handshake. Здесь в качестве источника			; ключевой
информации служит общий секрет, полученный			
+-----> [ClientHello...ServerHello]			; при помощи
протокола DH (слева). Дополнительная информация			; это сообщения
ClientHello, ServerHello. В результате -			
II)		v	; получают наборы
секретов для защиты сообщений на этапе		client_handshake_traffic_secret,	; Handshake.
		server_handshake_traffic_secret.	
	v		
Derive-Secret(., "derived", "")			
	v		
0 -> HKDF = Master Secret			
		+-----> [ClientHello...server, client Finished]	; Первый набор
ключей для защиты полезного трафика приложений			; получается по той
же схеме, но в качестве дополнительной			; информации служат
сообщения от ClientHello до серверного			
III)		client_application_traffic_secret_0,	; Finished. (Для
resumption_master_secret - до клиентского		server_application_traffic_secret_0,	; Finished, так как
данный секрет предназначен для будущих		exporter_master_secret,	; сессий.)
		resumption_master_secret.	
	.		
	.		
	.		

Упрощённая схема преобразования ключей в TLS 1.3.
(RFC 8446)

Обратите внимание, что на схеме отражён алгоритм генерации *симметричных секретов*, но это ещё не сами сеансовые ключи. Ключи, а также соответствующие

векторы инициализации, получаются на основе этих секретов в результате применения функции из семейства HKDF, а именно - HKDF-Expand-Label(). Узлы могут периодически заменять ключи защиты трафика приложений, выполняя ещё одну итерацию преобразования (этап III).

Сокращённый вариант установления соединения (Handshake) в TLS 1.3

При проектировании TLS 1.3 большое внимание уделялось снижению потерь времени в работе протокола. Основной вклад в задержку по времени вносит установление соединения. Поэтому предусмотрена сокращённая схема, которая оказывается даже быстрее сокращённой схемы предыдущих версий. Новая схема носит условное название 0-RTT (Zero Round-Trip Time - нулевая задержка приёма-передачи).

Для использования данной схемы требуется, чтобы стороны заранее согласовали общий секрет (как описано выше). Если такой секрет известен, то клиент может начать соединение отправкой ClientHello, с указанием общего секрета, за которым сразу же следуют данные полезной нагрузки. В случае HTTPS, такими данными будет, например, запрос HTTP GET - так как это самый распространённый сценарий в работе веб-сервисов. Другими словами, клиент сразу же, не дожидаясь ответа сервера, приступает к отправке запросов уровня приложения. В случае, если сервер успешно принял соединение, он отвечает сообщением ServerHello и другими сообщениями (Certificate и т.д.), заключает фазу открытия сессии сообщением Finished, и сразу же отправляет полезные данные, являющиеся ответом на запрос приложения клиента. В случае HTTPS, это будет HTTP-ответ на клиентский GET-запрос. Несложно заметить, что схема, по задержкам приёма-передачи, полностью эквивалентна обычной схеме HTTP: клиент отправляет запрос и получает ответ на него. При этом часть, относящаяся к TLS, можно считать некоторой дополнительной "обёрткой" над HTTP-схемой, не требующей выделенных сеансов приёма-передачи.

Клиентский запрос в схеме 0-RTT не обладает прогрессивной секретностью: он зашифрован не с выделенным сеансовым ключом, а с общим секретным ключом, полученным в другом сеансе, который должен был быть где-то сохранён. Соответственно, если ключ будет скомпрометирован, получившая его сторона сможет расшифровать клиентскую часть Handshake из записанного ранее трафика. Этот аспект 0-RTT относится только к клиентскому запросу, так как дальнейший трафик приложения защищается с использованием сессионных ключей, которые уже могут быть сгенерированы с обеспечением прогрессивной секретности. По этой и ряду других причин, схема 0-RTT в целом считается менее безопасной (это отмечено в спецификации TLS 1.3).

Защита данных расширения SNI

Расширение SNI (Server Name Indication) используется для передачи серверу логического имени ресурса, с которым планирует установить соединение клиент. Как видно из [разобранных выше сообщений](#) - SNI содержит строку символов, в которой имя записано. Эта строка передаётся в открытом виде, но при этом является одним из важнейших элементов, несущих метаинформацию о соединении. SNI оказывается необходимым в конфигурации, когда один и тот же IP-адрес разделяется несколькими ресурсами под несколькими именами хостов (например: `sr.example.com`, `web.example.com` и т.д. - все размещаются на IP-адресе `10.1.2.3`), но для каждого имени используется свой TLS-сертификат и разные серверные ключи. Сервер должен отправить клиенту серверный сертификат в самом начале соединения. Если таких сертификатов несколько и они различаются по именам, то, в случае простого ClientHello, без указания SNI, сервер не может определить, какой сертификат отправить. При этом процесс валидации сертификата клиентом требует, чтобы имя узла, которое ожидает получить клиент, совпало с именем в сертификате. (Оговорка про IP-адрес здесь важна потому, что если узел использует один IP-адрес для одного имени, то определить подходящий сертификат и ключ на стороне сервера можно по IP-адресу, с которым соединился клиент.)

SNI поддерживается всеми современными браузерами и повсеместно используется в вебе. Это означает, что на начальном этапе установления соединения клиент раскрывает третьей стороне, пассивно прослушивающей трафик, имя узла, с которым пытается соединиться. Получается, что полезный трафик защищён TLS, но прослушивающей стороне видны имена, к которым обращается клиент. Это хорошо известная особенность. Попытки побороть данную утечку предпринимались довольно давно, а особенно активно - в ходе разработки версии TLS 1.3. Однако ввести защиту SNI от прослушивания непосредственно в спецификацию TLS 1.3 не получилось. Это, в основном, связано с тем, что всякая защита SNI требует дополнительных вычислительных затрат: имя в SNI нужно для того, чтобы найти подходящий сертификат на стороне сервера; этот сертификат содержит ключ, необходимый для создания защищённого канала, поэтому защитить SNI тем же ключом - не получится; это означает, что нужны дополнительные ключи, которые требуют согласования и, следовательно, обмена сообщениями. Другими словами, сложно выстроить простую и надёжную схему. Тем не менее, экспериментальный механизм защиты SNI для TLS 1.3 всё же появился в 2019 году. На стороне клиента поддержку добавили в браузер Firefox. По результатам экспериментов, после длительных обсуждений, первоначально [предложенный вариант](#) фактически забраковали и полностью переделали, а новая спецификация находится в статусе экспериментального внедрения (2025).

Изначально механизм назывался ESNi (Encrypted SNI), однако позже, по мере развития спецификации и появления новых особенностей, название изменили на

ECH (Encrypted ClientHello), под которым технология и известна сейчас (2025). [Спецификация ECH](#) всё ещё находится в состоянии черновика. Старое название (ESNI) показывает, что защищать планировали именно и только поле SNI. Новое название наводит на мысли о том, что здесь уже прячется полноценное сообщение ClientHello. Именно так и обстоит дело.

В ESNI предполагали передавать зашифрованное имя сервера (SNI) в специальном расширении сообщения ClientHello. Новый вариант технологии отличается концептуально, поскольку построен на полноценном туннелировании TLS-соединения. Это достигается при помощи инкапсуляции секретного (внутреннего) сообщения ClientHello2 в открытое (внешнее) сообщение ClientHello1. Естественно, так как сведения об имени скрытого ресурса передаются во внутреннем ClientHello2, то схема полностью решает и задачу сокрытия SNI. Однако ECH предоставляет гораздо более высокий уровень секретности: технология позволяет клиенту защитить сам *процесс установления* TLS-соединения со скрытым сервисом.

Обратите внимание, что изначально TLS не пытается скрывать факт установления соединения между двумя узлами. ECH, на первый взгляд, укладывается в эту концепцию, поскольку защищённое ClientHello2 хоть и зашифровано, но сам факт его передачи в составе внешнего соединения не скрывается. При ближайшем же рассмотрении оказывается, что факт *наличия* защищённого ClientHello2 является свойством *внешнего* TLS-соединения, которое лишь выступает в роли средства прикрытия и полноценным TLS-соединением не является. Этот аспект особенно хорошо виден, если учитывать, что скрытый сервис может иметь *другой IP-адрес*, а тот узел, к которому обратился клиент с начальным сообщением ClientHello1, только туннелирует внутреннее TLS-соединение, содержание которого недоступно для просмотра, в том числе, и на проксирующем узле. Таким образом, ECH - это шаг в сторону реализации стеганографического подхода к созданию TLS-соединений.

Для установления соединения с защищённым сообщением ClientHello сторонам требуется согласовать секрет, который позволит сгенерировать общий симметричный ключ. В качестве основного (*но не единственного!*) транспорта для публикации необходимых ECH-ключей используется доменная система имён (DNS). DNS здесь работает следующим образом: в специальной DNS-записи, связанной с точкой входа скрытых сервисов, публикуется набор криптографических параметров (в том числе, открытых ключей), которые поддерживает скрытый сервер. Клиент, - это может быть браузер, - извлекает из DNS ключи, вычисляет секрет и, на основе этого секрета, симметричный ключ, который используется при зашифровании дополнительного сообщения ClientHello. Зашифрованные таким образом данные клиент передаёт серверу в составе начального сообщения ClientHello, внутри специального расширения. То есть, полезная нагрузка ECH - передаётся в формате расширения ClientHello. Сервер, располагая соответствующими секретными ключами, может вычислить симметричный ключ и расшифровать внутреннее сообщение ClientHello

(ClientHello2). После того, как определены параметры скрытого ClientHello2, туннелирующий узел передаёт это сообщение скрытому сервису, который уже выполняет оставшиеся шаги алгоритма начального установления TLS-соединения.

Таким образом, в ECH туннелирующий узел лишь обеспечивает обмен данными между клиентом и скрытым сервисом на уровне потокового соединения, а не на уровне TLS. Это ключевой аспект технологии - TLS-соединение устанавливается со скрытым сервисом, а входной узел-туннель, - который, формально, выглядит как TLS-узел, в реальности только *маскирует факт установления TLS-соединения*.

Фактически, за вычетом некоторых логических отличий, можно считать, что клиент устанавливает со скрытым сервисом TLS-соединение через туннель, обеспечиваемый внешним, входным узлом. Важно понимать, что при этом никакого внешнего TLS-соединения не существует, оно лишь имитируется на начальном этапе. Туннелирование осуществляется на транспортном уровне (входной узел просто копирует пакеты в обе стороны, никак их не изменяя), а полезная нагрузка соединения зашифровывается с теми ключами, которые клиент согласовал непосредственно со скрытым сервисом. Тем не менее, так как используется TLS 1.3, в котором уже начальная фаза установления соединения защищена от прослушивания, а также из-за семантических свойств зашифрованных данных, для третьей стороны скрытое TLS-соединение не отличается от "обычного". Подлинное имя сервера оказывается защищено от просмотра третьей стороной, а необходимые дополнительные ключи - публикуются в DNS, то есть, логически не зависят от транспорта конкретной TLS-сессии. Реализовать такую схему оказалось возможным лишь благодаря архитектурным особенностям именно TLS 1.3.

Исторически, в подобных схемах сокрытия соединения открытая, - внешняя, - часть сообщений клиента может содержать маскирующее имя сервера (эта идея высказывалась в черновике [Fake SNI](#)), либо не содержать имени сервера вообще. Естественно, запрос к DNS, необходимый для получения ключей, может послужить каналом утечки, аналогичным открытому полю SNI. Предполагается, что, во-первых, клиент использует защищённый способ доступа к DNS (например, DNS-over-TLS); во-вторых, для дополнительной маскировки ключи могут публиковаться под DNS-именем, которое отличается от скрытого имени сервера (естественно, такое "имя прикрытия" должно быть заранее известно клиенту); в-третьих, ключи вообще могут быть распределены способом, отличным от DNS, например, непосредственно в составе дистрибутива браузера (или другой программы-клиента).

Спецификация ECH предусматривает выделение для хранения ключей отдельного типа DNS-записи. Кроме ключей и других криптографических параметров, соответствующая DNS-запись (она называется HTTPS/SVCB и уже введена в DNS) сможет включать и IP-адреса сервера. Это означает, что клиенту достаточно будет одного запроса в DNS для извлечения и ключей, и адреса сервера.

Асимметричные криптосистемы, электронная подпись

Для организации защищённого соединения в TLS используются асимметричные криптосистемы, криптосистемы электронной подписи. На практике, сейчас это RSA и ECDSA, *крайне редко* - EdDSA в различных вариантах (EdDSA в этом описании не рассматривается). Как описано выше, RSA может применяться в исторической схеме для передачи сеансового секрета клиентом на сервер (такая схема всё ещё встречается). ECDSA - служит в TLS только для получения подписи, то есть, аутентификации. Предполагается, что в ближайшем будущем RSA окажется полностью вытеснена эллиптическими системами, а в качестве второго направления добавятся постквантовые алгоритмы (то есть, криптосистемы, обладающие стойкостью в предположении атаки на квантовом компьютере).

Криптосистема RSA

RSA - самая массовая криптосистема с открытым ключом (асимметричная криптосистема) в Интернете за всю историю. Повсеместно используется для подписи в составе TLS-сертификатов. Долгое время RSA являлась, фактически, единственной криптосистемой TLS-сертификатов. Лишь в 2014 году наряду с RSA в сертификатах стала заметна доля [ECDSA](#). Логика построения RSA напоминает протокол Диффи-Хеллмана, однако отличаются свойства однонаправленной функции. В RSA однонаправленная функция имеет "лазейку" или "чёрный ход", знание которого позволяет легко обратить функцию, вычислить аргумент по значению. RSA основана на возведении в степень по модулю (то есть, "взятие остатка"), а стойкость этой криптосистемы связана с задачей разложения числа на простые множители. Можно считать, что секретом в RSA является именно разложение "ключа" на простые множители. Слово "ключ" здесь дано в кавычках, так как на практике ключ содержит дополнительный параметр, но этот параметр всегда можно легко вычислить, зная разложение. (Задачу разложения числа на простые множители принято называть "задачей факторизации".)

Параметрами криптосистемы RSA являются модуль (это всегда составное число) и открытая (часто её называют "шифрующей") экспонента - то есть, показатель степени, в которую возводится числовое значение сообщения. Сообщение, в рамках RSA, это целое число, записываемое в виде последовательности байтов. Модуль в RSA является составным числом, произведением двух простых: $N = pq$, где p и q - достаточно большие *простые числа*, именно их и нужно держать в секрете. (Строго говоря, успехи криптоанализа RSA накладывают на p и q немало дополнительных требований, которые мы не рассматриваем, так как для понимания принципа работы криптосистемы это не важно.) Знание разложения $N = pq$ позволяет построить "лазейку", с помощью которой можно обратить однонаправленную функцию, лежащую в основе RSA. "Лазейка" состоит в том, что для "шифрующей экспоненты" вычисляется обратный элемент - "расшифровывающая экспонента". Быстро

вычислить этот обратный элемент позволяет знание разложения модуля ключа на простые множители.

Итак, помимо модуля, для использования RSA с целью генерирования электронной подписи и расшифрования сообщений, необходимо сгенерировать секретный ключ. Секретным ключом как раз и является экспонента, обратная к шифрующей (расшифровывающая). В случае TLS, сервер публикует модуль N и шифрующую экспоненту E , а в секрете сохраняется соответствующая E расшифровывающая экспонента d , такая, что $d = E^{-1}$ по модулю $(p-1)*(q-1)$; или $d * E = 1 \pmod{(p-1)*(q-1)}$. В состав представления секретного ключа обычно входит и разложение модуля - числа p и q ; это позволяет сильно ускорить вычислительные операции; конечно, зная открытую и секретную экспоненты - несложно факторизовать и модуль, но данная операция потребует дополнительных затрат.

Таким образом, *открытый* (публичный) ключ RSA состоит из модуля N и экспоненты E , а закрытый - из N, p, q, d - модуля, разложения модуля (p, q) и секретной экспоненты. Так как шифрующая экспонента - это публичный параметр, то повсеместно используются типовые значения, например, 65537 (или 0x010001). При этом соответствующая секретная (расшифровывающая) экспонента будет отличаться для разных значений модуля N . Открытый ключ обычно публикуется в составе сертификата TLS (сертификаты подробнее рассмотрены в [специальном разделе](#)) и служит для проверки подписей, а для их вычисления - необходимо знать соответствующий секретный ключ.

RSA позволяет зашифровать сообщение, выступая в роли асимметричного шифра. Учебный пример зашифрования состоит в возведении сообщения, представленного в виде числа, в шифрующую степень и вычислении остатка по модулю N (заданному в составе ключа). $C = m^E \pmod N$, где C - шифротекст, а m - открытый текст.

Соответственно, зная расшифровывающую экспоненту, можно расшифровать C : $m = C^d \pmod N$. Это соотношение можно проиллюстрировать следующим **нестрогим (!)** рассуждением, используя тот факт, что $e = d^{-1} \pmod{(p-1)(q-1)}$, поэтому: $C^d = (m^e)^d = m^{e*d} = m^{e*e^{-1}} = m^1 = m$.

Если более строго, то нужно воспользоваться функцией Эйлера и теоремой Эйлера (или Лагранжа, или малой теоремой Ферма). В рамках данной статьи не рассматривается несложный математический аппарат RSA и не приводится доказательство корректности операций, только формулы. А именно: пусть $N = p*q$, а *сообщение m взаимно просто с N* . Зашифруем сообщение $C = m^E$; тогда, $C^d = (m^E)^d$; т.к. $d * E = 1 \pmod{(p-1)*(q-1)}$, получаем $d * E - 1 = k * ((p-1)*(q-1))$, для некоторого k ; подстановка: $C^d = (m^E)^d = m^{(k*((p-1)*(q-1)) + 1)} = (m^{(p-1)*(q-1)})^k * m$; т.к. $(m^{(p-1)*(q-1)}) = 1 \pmod N$, получаем $C^d = (m^{(p-1)*(q-1)})^k * m = (1)^k * m = m$. (Полное строгое

доказательство, использующее малую теорему Ферма, нетрудно найти в литературе.)

Обратите внимание, что из-за возможности проведения эффективной атаки, RSA нельзя использовать подобным способом, непосредственно зашифровывая сообщение. Допустимым вариантом является, например, зашифрование RSA случайного секрета, дополненного специальным образом до подходящей разрядности, с последующим вычислением ключа симметричного шифра при помощи хеш-функции от переданного секрета - именно эта схема входит в спецификации TLS (однако использование RSA для обмена сессионными секретами в настоящее время считается устаревшим методом).

Генерация ключей RSA начинается с генерации пары простых чисел, которые составят модуль. Знание p и q позволяет легко определить обратную к шифрующей экспоненту. Однако задача разложения числа на простые множители является вычислительно сложной. Фактически, для достаточно больших чисел (2048 бит и более), сейчас не известно алгоритмов, позволяющих находить разложение произвольных чисел за "разумное время" с использованием любых технологически мыслимых вычислительных мощностей. Существует, - пока сугубо теоретическая, но отлично обоснованная, - возможность быстрой факторизации при помощи алгоритма Шора на квантовом компьютере достаточной разрядности (под вопросом находится сама возможность создания подходящего квантового компьютера). "Квантовая атака" касается не только RSA, но также и описанных выше вариантов DH, и криптосистемы электронной подписи ECDSA. Так как применимость квантового компьютера зависит от его доступной разрядности - от количества рабочих кубитов, - то, например, ECDSA будет взломана на таком квантовом компьютере раньше RSA, так как ECDSA использует ключи существенно меньшей разрядности (типичные значения: 256 бит против 4096 у RSA).

При использовании для создания и проверки электронной подписи, RSA работает следующим образом. Сторона, генерирующая подпись, вычисляет значение хеш-функции от подписываемого сообщения M , а именно - $H = \text{Hash}(M)$, приводит это значение к разрядности используемого модуля N и вычисляет значение подписи при помощи секретной (расшифровывающей) экспоненты: $S = H^d \bmod N$. То есть, с арифметической точки зрения, происходит следующее: значение H , рассматриваемое как двоичная запись натурального числа, возводится в степень с показателем d , а для полученного значения вычисляется остаток от деления на N . На практике используются некоторые существенные оптимизации, а процесс вычисления называется "возведением в степень по модулю числа N ".

Для проверки подписи требуется знать открытую часть ключа: модуль и шифрующую экспоненту E . Значение подписи возводится в степень e , а получившееся значение сравнивается с вычисленным значением H хеш-функции сообщения: $S^E \bmod N ==$

Н. Если значения совпали, то подпись верна. Для вычисления корректной подписи (подделки), соответствующей заданному сообщению, третьей стороне необходимо знать секретную экспоненту d . Защиту обеспечивает следующий факт: для быстрого определения d по известным параметрам E и N потребуется найти разложение N на простые множители, а это вычислительно трудно.

Механизм электронной подписи RSA (как и ECDSA) используется для удостоверения параметров протокола Диффи-Хеллмана, переданных сервером в сообщении `ServerKeyExchange`. Подпись S , передаваемая в составе сообщения TLS, является целым числом, имеющим разрядность, соответствующую разрядности модуля ключа. Модуль ключа N содержится в сертификате сервера. Так, если модуль имеет разрядность 2048 бит, то для записи подписи потребуется 256 байтов ($256 \cdot 8 = 2048$). Подпись, как отмечено выше, вычисляется от значения хеш-функции для объединения параметров DH (в них входит и серверная часть), кроме того, в TLS 1.2 к параметрам добавляются значения `ServerRandom` и `ClientRandom`. В процессе вычисления значения подписи параметры преобразуются к формату, требуемому алгоритмом применения RSA.

При прямой передаче сеансового секрета на сервер RSA применяется в TLS самым хрестоматийным образом: открытый ключ извлекается из сертификата сервера, сгенерированный секрет преобразуется к установленному формату, а результат возводится в степень, заданную шифрующей экспонентой. Получившееся зашифрованное сообщение передается в `ClientKeyExchange` (как описано выше). Сервер, которому известно значение секретной экспоненты, расшифровывает полученное сообщение. Предполагается, что если третья сторона вмешалась в ход установления соединения, подменив сервер, то этой стороне не удастся корректно расшифровать переданный клиентом секрет, соответственно, стороны не смогут прийти к общему набору сеансовых ключей. Интересно, что для обхода данного ограничения перехватывающей стороне не требуется строго сам секретный ключ, значение которого нигде не участвует, а достаточно, чтобы была возможность расшифровать данные, например, отправив запрос серверу. Такая схема иногда используется при делегировании полномочий по инспекции TLS-трафика третьей стороне, без передачи самого секретного ключа (однако в отношении RSA схема вовсе не является полностью безопасной: можно заметить, что операции с секретной экспонентой эквивалентны генерированию RSA-подписи). Данный метод получения сеансового секрета использовать не рекомендуется.

Другое историческое применение RSA, как асимметричного шифра в TLS, возможно при выборе сервером варианта протокола установления соединения, использующего временные ключи RSA. В таком случае сервер передает модуль и значение открытой экспоненты в составе сообщения `ServerKeyExchange`. Клиент использует полученный открытый ключ RSA для того, чтобы зашифровать случайные байты `ClientKeyExchange`, которые послужат для генерации симметричных ключей. Эта

экзотическая схема была актуальна для "экспортных" вариантов протокола, когда от реализации требовалось, чтобы для защиты сессии не применялись ключи RSA длиннее 512 бит: сервер использовал "длинный" ключ из сертификата для подписывания "короткого" временного ключа из ServerKeyExchange. Сейчас RSA в TLS не следует использовать в качестве асимметричного шифра, это небезопасно (тем более, с коротким ключом).

Важно учитывать, что математическая эквивалентность операций получения электронной подписи и расшифрования в RSA является особенностью данной криптосистемы, не более того. Операции подписывания и зашифрования/расшифрования - это разные операции, их совпадение в RSA не означает, что другие криптосистемы электронной подписи могут прямо служить шифрами (хотя построить шифр на базе такой криптосистемы обычно можно).

Для RSA предложено большое количество теоретико-числовых атак. Часть этих атак можно эффективно применить к дефектам и ошибкам в той или иной реализации RSA, что делает атаки не только практическими, но и весьма быстрыми. Криптосистема RSA считается устаревшей в целом, что, впрочем, не мешает ей оставаться очень популярной.

Рассмотрим фрагмент трафика установления TLS-соединения, в котором узлы договорились использовать шифронабор с передачей сеансового секрета с помощью шифрования RSA (а именно: TLS_RSA_WITH_AES_128_CBC_SHA). Сертификат, представленный сервером, содержит открытый ключ длиной 2048 бит. Этот ключ используется клиентом для зашифрования. О том, что клиент прислал именно зашифрованный RSA секрет - сервер знает потому, что RSA указана в шифронаборе, соответственно, рассматриваемое сообщение ClientKeyExchange не содержит никакого указания на то, что используется именно RSA, с тем или иным ключом.

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	16 03 03	; Handshake, TLS 1.2.
0003	01 06	; Длина: 0x0106 = 262 байта.
0005	10	; Сообщение ClientKeyExchange (0x10).
0006	00 01 02	; Длина: 0x000102 = 258 байт.
0009	01 00	; Длина блока PreMasterSecret: 0x0100 = 256 байт. Длина
		соответствует разрядности ключа из сертификата - 2048 бит.
0011	54 ...	; Далее идёт зашифрованный блок из 256 байтов.
		Из них 48 байтов (46+2) - это PreMasterSecret, остальные байты
		содержат дополнение, необходимое для корректного использования RSA.

Криптосистема ECDSA

ECDSA является эллиптическим вариантом криптосистемы электронной подписи DSA. Последняя (DSA) - сейчас практически не используется. Интересно, что она, в свою очередь, является вариантом схемы подписи Эль-Гамала, которая представляет собой алгоритм, построенный на базе задачи дискретного логарифмирования, и, таким образом, имеющий прямое родство с классическим протоколом Диффи-Хеллмана.

Идею схемы ECDSA можно изложить следующим образом: предложим такую пару алгоритмов для генерации значения подписи и проверки этого значения, которые будут "сходиться" к одному значению только в том случае, если стороны использовали ключи из связанной однонаправленной функции пары (это, впрочем, очевидная схема для ЭЦП). В качестве однонаправленной функции используется умножение точки эллиптической кривой на скаляр (это та же операция, что и в ECDH). При этом подписи "рандомизируются", для чего служит дополнительный параметр - ещё одна точка кривой. При успешной проверке, результат, полученный проверяющей стороной, совпадёт с этой дополнительной точкой - откуда и возникает, достаточно условное, представление о том, что схемы сходятся "в одной точке". ECDSA существенно отличается от RSA, как по составу и свойствам параметров, ключей, так и по алгоритмам генерации подписи и её проверки. ECDSA, вообще говоря, несколько сложнее. Ключевым является тот факт, что обе основные части криптосистемы ECDSA - алгоритм вычисления подписи и алгоритм проверки, - представляют собой две части единого базового соотношения ("уравнения криптосистемы").

Итак, прежде чем использовать ECDSA, стороны должны согласовать общие параметры: выбрать кривую E (она традиционно задаётся парой чисел); выбрать конечное поле, над которым будут производиться операции на кривой (кривая, как и в ECDH, используется в целочисленной форме; "конечное поле" - упрощённо можно считать, что это тоже выбор одного простого числа или степени простого числа); выбрать соответствующую общую точку на кривой - генератор G . Точка - это пара значений (x, y) , которые далее называются координатами. (x, y) - элементы соответствующего поля (упрощённо - натуральные числа, меньше некоторого заданного; остатки от деления на модуль). Как и в случае с "эллиптическим" вариантом DH, на кривой задана операция сложения точек - это та же самая операция, которая используется и в ECDH (см. выше). На этой операции строится алгоритм удвоения точки и умножения на скаляр. Именно умножение на скаляр является основой криптосистемы (опять же, аналогично ECDH). Так как тут традиционно используется аддитивная запись для операции сложения точек, то речь идёт именно об умножении, однако в мультипликативном случае ему соответствовало бы возведение в степень, отсюда и возникает "логарифмирование". Если у нас есть точка N на кривой, то умножение на скаляр $[d]N$ - тоже даёт точку (B

$= [d]N$) на кривой. Задача вычисления d по известным N , B и есть задача дискретного логарифмирования в группе точек эллиптической кривой, уже знакомая из описания протокола Диффи-Хеллмана.

Открытый ключ в ECDSA - это точка Q на кривой, полученная умножением выбранного генератора G (тоже точка) на целое положительное число d : $Q = [d]G$. Соответственно, d - является секретным ключом. Схема в этой части напоминает RSA, только в данном случае в состав открытого ключа не входит некий модуль (но модуль, как элемент арифметики остатков, используется при задании параметров криптосистемы). Генератор G является публичным значением, но для того, чтобы определить d , потребуется отыскать соответствующий логарифм, что нереализуемо на практике (при условии правильно выбранных параметров криптосистемы). Открытый ключ, таким образом, представляет собой пару чисел (x_Q, y_Q) - так как точке соответствует две координаты.

Важно понимать, что эти координаты - это просто пара элементов подлежащего конечного поля, удовлетворяющих уравнению кривой. То есть, можно рассматривать кривую с операцией умножения точки на скаляр, как некоторую структуру, заданную поверх *пар* элементов выбранного поля (упрощённо - пар больших натуральных чисел, которые меньше значения модуля). При этом *умножение точки кривой на скаляр* - это операция *в группе точек кривой, а не в поле*. А именно: *умножению точки на скаляр d (натуральное число) соответствует $(d-1)$ операций сложения точки с самой собой, каждая из которых ставит в соответствие двум парам элементов поля, возможно, совпадающим, - третью пару, так как одна точка характеризуется именно парой элементов поля*. На практике, так как используется конечное поле, его элементы всегда можно пронумеровать натуральными числами, что позволяет эффективно проводить компьютерные вычисления. *Но следует учитывать, что операции в поле и на кривой не будут совпадать с привычными операциями над натуральными числами (которые поля не образуют)*. Дополнительные сведения о соответствующем математическом аппарате приведены в [Приложении А](#).

Для вычисления подписи в ECDSA используется хеш-функция, позволяющая получить "сжатое" представление сообщения M : $H = \text{Hash}(M)$. Значение H приводится к разрядности выбранных параметров. В описании ниже - n обозначает порядок генератора G в группе кривой, то есть $[n]G = 0$ (0 - нейтральный элемент группы точек кривой). Схема существенным образом базируется на "рандомизации" - каждая подпись использует случайный параметр k , который не должен повторяться. (Существует так называемый "детерминированный вариант" ECDSA, но мы его здесь не рассматриваем, тем более, что все его особенности являются глубоко техническими, а принципы верхнего уровня полностью совпадают с "обычной" ECDSA.) Уникальность k является фундаментальным требованием для обеспечения стойкости, поскольку повторное использование позволяет вычислить секретный

ключ. Это, пожалуй, самая часто упоминаемая в литературе особенность ECDSA, но, тем не менее, ошибки тут всё равно встречаются. Именно на некорректном повторном использовании k основывался нашумевший взлом системы защиты ПО Sony Playstation в 2010 году.

Вычисление подписи (на входе: значение хеш-функции от сообщения - H , которое приводится к разрядности n ; секретный ключ - d) выполняется следующим образом ([RFC 6979](#)):

- 1) выбирается уникальное (псевдо)случайное число k (соответствующее интервалу $[1, n-1]$);
- 2) вычисляется точка $[k]G$ (где G - это точка-генератор, заданная в параметрах криптосистемы). Данная точка на кривой является точкой, задающей конкретное значение подписи, связывающей алгоритм проверки подписи с алгоритмом вычисления подписи. Точке $[k]G$ соответствуют координаты (x, y) ;
- 3) для x вычисляется значение $r = x \bmod n$ (то есть, остаток от деления x на порядок генератора G ; r - это первая часть значения подписи; если $r = 0$, что маловероятно, то алгоритм возвращается к шагу 1);
- 4) вычисляется значение $s = k^{-1}(H + rd) \bmod n$. k^{-1} - это обратный к k элемент (относительно умножения) по модулю n , его несложно вычислить; (если $s = 0$, что маловероятно, то возвращаемся к шагу 1)
- 5) пара (r, s) - и есть значение электронной подписи.

Так как от выбора k зависит стойкость реализации ECDSA, для генерации этого параметра предложены различные безопасные схемы, в том числе, детерминированные алгоритмы, работающие наподобие счётчика и, таким образом, сводящие к минимуму вероятность повтора k .

Для проверки подписи необходимо знать параметры криптосистемы, а также открытый ключ Q . Перед проверкой *необходимо* убедиться, что переданные значения соответствуют криптосистеме (Q является точкой соответствующей кривой, (r, s) не равны нулю, и так далее). Проверка выполняется по следующему алгоритму (на входе - значение хеш-функции H от сообщения, пара (r, s) , открытый ключ Q):

- 1) вычисляется значение $v = s^{-1} \bmod n$;
- 2) вычисляется значение $u_1 = Hv \bmod n$;
- 3) вычисляется значение $u_2 = rv \bmod n$;
- 4) определяется *точка В на кривой* $= [u_1]G + [u_2]Q$. Здесь G - генератор, заданный в параметрах, а Q - открытый ключ. Умножение - обозначает умножение точки кривой на скаляр; сложение - сложение точек на кривой;
- 5) если для координаты x точки B выполняется $r = x \bmod n$, то подпись считается действительной.

Доказательство корректности. Согласно шагу 4 алгоритма вычисления значения подписи: $s = k^{-1}(H + rd) \bmod n$. Выполним подстановки (все сравнения по модулю n , вместо знака $\equiv$ для простоты используем знак равенства):

- $k = s^{-1}(H + rd) = s^{-1}H + s^{-1}rd = (Hv) + (rv)d = u_1 + u_2d$;
- мы нашли, что $u_1 + u_2d = k$; здесь k - случайный параметр подписи, а d - секретный ключ, соответствующий Q ;
- преобразования следуют из соотношений, заданных в алгоритме проверки и связывают открытый ключ, случайный параметр k и значение подписи, относительно значения H (хеш сообщения);
- $Q = [d]G$, что позволяет выполнить следующую замену: $[u_1]G + [u_2]Q = [u_1]G + [u_2d]G = [(u_1 + u_2d)]G = [k]G$ (соответствует исходной точке, определённой на шаге 2 алгоритма вычисления подписи);
- таким образом, если подпись действительно получена от ключа Q , то $r = x \bmod n$ (ведь x - это координата, соответствующая точке $[k]G$).

ECDSA используется как в TLS-сертификатах, так и для удостоверения серверных параметров DH. При этом ECDSA нельзя использовать для зашифрования, а только для получения и проверки электронной подписи.

Рассмотрим фрагмент трафика TLS-сессии - это сообщение ServerKeyExchange, в котором передаются параметры протокола Диффи-Хеллмана на эллиптических кривых (ECDH), подписанные ECDSA.

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	16 03 03	; Handshake, TLS 1.2.
0003	00 95	; Длина: 0x0095 = 149 байтов.
0005	0c	; ServerKeyExchange.
0006	00 00 91	; Длина: 0x000091 = 145 байт.
0009	03 00 17	; Серверные параметры ECDH: 0x03 - префикс именованной кривой; 0x0017 - обозначение кривой secp256r1.
0012	41	; Длина блока серверной части DH: 0x41 = 65 байт.
0013	04 f3 52...	; Значение серверной части DH - это точка на кривой secp256r1.
[...]		
0078	06 03	; Тип подписи и хеш-функции. 0x06 - хеш-функция SHA-512; 0x03 - тип подписи ECDSA.
0080	00 48	; Длина блока значения подписи: 0x0048 - 72 байта;
0082	30	; Далее идёт значение подписи. Значение представляет собой пару целых чисел (r, s) . Числа имеют разрядность, соответствующую разрядности группы кривой (в данном случае: 256

бит, то есть, 32		; байта на одно число). Пара кодируется в
соответствии с ASN.1, в		; двоичном представлении. Таким образом, для записи
двух чисел		; потребуется $32 * 2 = 64$ байта. Остальные байты
содержат служебные		; значения кодирования ASN.1. Так, 0x30 -
обозначает, что будет		; передана последовательность (SEQUENCE). Следующий
байт - длина		; последовательности в октетах.
0083	46	; Длина последовательности ASN.1: $0x46 = 70$ байт
(октетов).		
0084	02	; Префикс, обозначающий в ASN.1 тип INTEGER
(целое).		
0085	21	; Длина записи значения INTEGER: $0x21 = 33$ байта
(октета) (32+1).		
0086	00 98 8b...	; Далее идёт значение первого элемента подписи, 33
байта.		
		; Так как само число 32-байтовое, первый байт
содержит		; значение 00.
[...]		
0119	02 21	; Аналогичные предыдущему числу поля ASN.1: INTEGER
(0x02),		
		; 33 (0x21) октета.
0121	00 99 26...	; Значение второго элемента подписи (33 байта,
первый - 00).		

Параметры криптосистем

Все описанные криптосистемы подразумевают, что стороны согласовали некоторые параметры, определяющие контекст применения:

- для "классического" варианта протокола Диффи-Хеллмана это модуль, задающий группу, а также генератор в ней;
- для ECDH (на эллиптической кривой) - параметры кривой, включающие поле, над которым кривая используется, и, опять же, генератор;
- для RSA - шифрующая экспонента и модуль;
- для ECDSA - параметры кривой и генератор.

Наибольшее число параметров требуется для эллиптических криптосистем. В TLS для них принято использовать типовые кривые, обозначенные зафиксированными именами, но при установлении соединения в версиях ниже 1.3 можно указать и любую другую кривую. В TLS 1.3 - параметры зафиксированы строго. Очевидно, и сервер, и клиент должны соответствующую кривую поддерживать. Каждому типовому

набору соответствует сама кривая и поле, над которым проводятся операции. Реестр *имён и индексов* ведёт IANA, названия определены в [RFC 4492](#), [RFC 7027](#) и др. Конкретные параметры - определены в сопутствующих документах NIST, ANSI и др. В 2016 году соответствующий реестр IANA переименован в "Реестр поддерживаемых групп" ("Supported Groups Registry"), что лучше соответствует математической действительности, потому что используемые в TLS криптосистемы работают в группах, просто, в ряде случаев, это группы точек эллиптической кривой.

С именованием кривых существует некоторая исторически сложившаяся путаница. Например, кривая, распространённая в сертификатах, используемых для HTTPS, называется prime256v1, она же - secp256r1, она же - NIST P-256; а параметры этой кривой определены, в частности, в рекомендации NIST 186-4.

Типичные параметры ECDSA, на примере NIST P-256 ([источник: SafeCurves](#), там же представлены сводные данные по всем другим распространённым кривым):

-- Уравнение кривой:

$$y^2 = x^3 - 3x + 41058363725152142129326129780047268409114441015993725554835256314039467401291$$

Кривые определяются параметрами a, b уравнения: $y^2 = x^3 + ax + b$

Соответственно:

```
a = -3
b = 41058363725152142129326129780047268409114441015993725554835256314039467401291
```

-- Поле; значение задающего модуля P (простое число):

$$P = 115792089210356248762697446949407573530086143415290314195533631308867097853951$$

$$= 0 \times ffffffff0000000010000000000000000000000000ffffffffff =$$

$$2^{256} - 2^{224} + 2^{192} + 2^{96} - 1.$$

-- Генератор G. Точка на кривой, заданная парой координат (x, y) :

```
(48439561293906451759052585252797914202762949526041747995844080717082404635286,
36134250956749795798585127919587881956611106672985015071877198253568414405109)
= (0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5)
```

-- Порядок генератора G ; n (простое число):

115792089210356248762697446949407573529996955224135760342422259061068512044369
 = 0xffffffff00000000ffffffffffffbce6faada7179e84f3b9cac2fc632551
 = $2^{256} - 2^{224} + 2^{192} - 89188191075325690597107910205041859247$

Сертификаты

TLS-сертификаты (SSL-сертификаты - устаревшее название, являющееся синонимом) играют важную роль в современной инфраструктуре TLS. У TLS-сертификата одно предназначение: *привязать открытый ключ к некоторому сетевому имени*. Например, сопоставить ключ 0xA3FEF7...DA3107 имени example.com. TLS-сертификаты не содержат никакой секретной информации. Они не являются непосредственными носителями ключей шифрования передаваемых данных в TLS (за исключением открытого серверного ключа RSA, который может быть использован для зашифрования сеансового секрета в устаревших режимах работы TLS).

Открытый ключ, который содержится в серверном сертификате, принадлежит искомому серверу, с которым клиент планирует установить соединение. Однако открытый ключ может быть подменён третьей стороной на другой, имитирующий легитимный ключ сервера. Для того, чтобы можно было обнаружить подобную подмену, служит инфраструктура удостоверяющих центров, которая даёт клиенту механизм проверки принадлежности ключа. То есть клиент соглашается верить некоторой третьей стороне (не злоумышленнику, а удостоверяющему центру - УЦ), что эта третья сторона проверила соответствие ключа и сетевого имени (в вебе это обычно домен). Результат такой проверки подтверждается подписью УЦ, которую он ставит на сертификате сервера. Для подписи сейчас используются криптосистемы RSA и ECDSA.

Предполагается, что клиент TLS имеет в своём распоряжении некоторый набор сертификатов удостоверяющих центров (часть из этих сертификатов является корневыми) и может проверить подписи на предоставляемых сервером сертификатах, выстроив цепочку доверия, ведущую от серверного ключа к одному из доверенных корней. На практике, в браузеры встроены многие десятки корней, соответствующих различным УЦ. Для выстраивания цепочки доверия определяющее значение имеет именно корневой ключ, а не сам сертификат, который его содержит. Тем не менее, в сертификате содержатся имена, которые позволяют отличить один ключ от другого и сопоставить отдельные сертификаты друг другу.

В SSL/TLS используются сертификаты формата X.509, это очень старый формат, изначально разрабатывавшийся для телеграфа, но позже адаптированный для использования в Интернете, в частности, в вебе.

Сертификат представляет собой электронный документ, имеющий определённую спецификацией структуру. Так, каждый сертификат содержит поля Issuer и Subject. Поле Issuer определяет имя стороны, выпустившей данный сертификат (поставившей на нём подпись), а поле Subject - имя стороны, для которой сертификат выпущен. В случае веба, в Issuer обычно указано имя из сертификата УЦ

(не обязательно корневого), а в Subject, для серверного сертификата, - доменное имя, адресующее сайт. В 2017 году в браузере Google Chrome появилось новшество: для валидации сертификата по имени сервера, это имя должно быть указано не в Subject, а в расширении сертификата под названием SAN (Subject Alternative Name). Постепенно, такой порядок указания имён стал обязательным для веба, поэтому сейчас (2025) TLS-сертификат сервера, чтобы считаться валидным, должен содержать имя (или IP-адрес) в расширении SAN, а поле Subject для серверных сертификатов вообще игнорируется и может не заполняться.

TLS-клиент получает сертификаты с сервера (в большинстве случаев - это цепочка сертификатов, но может быть и единственный серверный сертификат) в сообщении Certificate (см. выше), выстраивает их в иерархию, где каждый сертификат удостоверяет следующий, проверяет подписи и соответствие имени из серверного сертификата имени сервера, с которым планирует установить соединение. Проверка имени является *важнейшим аспектом*, так как злоумышленник может выпустить доверенный сертификат для своего имени сервера, но предъявлять его под именем другого сервера (перехватив сеанс тем или иным способом). В таком случае полностью валидный сертификат злоумышленника будет отличаться от легитимного только указанным именем и ключом. Например, сертификат выпущен для example.com, а предъявляется для test.ru. (Такое несоответствие также является одной из самых распространённых ошибок настройки TLS-серверов.)

Сертификат выпускается на определённый срок, который указан в самом сертификате. Сертификат может быть отозван, по какой-то причине, раньше окончания срока действия. Браузеры и другие TLS-клиенты должны проверять статус отзыва сертификатов, для этого существует ряд механизмов. Однако в повседневной реальности проверка отзыва сертификатов, фактически, не работает. Более того, популярны стали сертификаты, имеющие очень короткий срок действия, и для таких сертификатов допускается отсутствие механизмов проверки статуса (отзыва). Считается, что короткоживущие сертификаты и так быстро перестают быть валидными. Эти моменты нужно учитывать при анализе рисков.

Посмотрим на пример TLS-сертификата - это расшифровка полей сертификата, выпущенного для dxdt.ru УЦ WoSign, полученная утилитой openssl x509:

Version: 3 (0x2)

; Версия спецификации, обычно - 3

Serial Number: 58:04:6c:41:4b:a3:02:a0:a7:c1:13:a3:07:53:97:90

; Серийный номер сертификата. На генерацию серийных номеров наложены определённые требования. Так, номера не должны быть предсказуемыми, не должно быть двух различных сертификатов одного УЦ с одинаковым серийным номером.

Signature Algorithm: sha256WithRSAEncryption

; Алгоритм подписи: это RSA-сертификат, с RSA-подписью, использующей хеш-функцию

SHA-256.

Issuer: C=CN, O=WoSign CA Limited, CN=WoSign CA Free SSL Certificate G2
; Имя стороны, выпустившей сертификат: WoSign. Это простое текстовое поле, определённого формата. В сертификате имеет значение каждый байт, это касается и имени в Issuer.

Validity

Not Before: May 5 23:50:37 2015 GMT

Not After : May 6 00:50:37 2018 GMT

; Срок действия сертификата. Не ранее - не позднее.

Subject: CN=dxdt.ru

; Имя, для которого выпущен сертификат - в нашем случае это dxdt.ru. Обратите внимание, что если не указано дополнительных имён DNS (см. ниже), сертификат будет валиден только для dxdt.ru, для www.dxdt.ru такой сертификат не подойдёт. Возможен выпуск сертификатов для имён с маской подстановки: *.dxdt.ru. В этом случае сертификат будет валиден для любого имени третьего уровня в dxdt.ru, например для tls.dxdt.ru или static.dxdt.ru, однако для dxdt.ru - сертификат окажется невалидным.

Subject Public Key Info:

; Данные открытого ключа сервера.

Public Key Algorithm: rsaEncryption

; Алгоритм - используемая криптосистема (RSA).

Public-Key: (2048 bit)

; Сам ключ (разрядность 2048 бит). Открытый ключ RSA (как описано выше) состоит из модуля и шифрующей экспоненты, обычно она выбирается из типовых значений, удобных для оптимизации вычислительных операций.

Modulus:

00:c4:c5:f6:49:02:50:3b:d2:9d:b6:d6:cd:19:80:
0f:66:a0:c9:eb:6b:b5:05:c5:52:e3:d9:09:25:a0:
2e:86:d8:e5:20:0c:39:66:bb:03:57:3e:23:a5:f7:
98:2d:99:16:f6:70:d4:87:c6:76:86:79:78:6a:4f:
bd:61:6a:89:23:7d:8c:dd:fc:8b:8f:d6:91:a5:33:
ed:df:35:b0:ee:cb:e7:43:1e:5a:8c:7e:e9:49:c3:
82:2a:95:9f:f1:a6:86:61:22:97:81:df:2b:55:b8:
f4:ad:0e:c1:f6:d2:c6:66:31:d3:57:a0:51:1e:5f:
7e:ce:d3:ba:27:21:cd:16:66:e5:22:e5:64:45:46:
64:8f:6a:f1:6c:2d:f2:8e:28:c1:72:27:3b:bf:8a:
10:56:c0:16:94:ec:60:c1:70:c0:c2:3d:28:8b:5c:
4c:44:e2:ac:67:1a:e7:93:ec:ec:1f:9b:a1:30:f6:
71:95:77:8d:8d:d4:d4:43:4e:1f:5e:5b:13:a9:48:
1d:c2:a3:87:26:e6:65:8a:ff:75:fb:84:c9:67:e3:
88:51:28:b2:2c:45:3a:7f:37:68:dd:da:14:73:c7:
71:b7:1c:ea:ea:d3:08:b4:a8:6b:8d:df:f6:be:71:
07:f0:fe:5c:b4:4e:15:de:2e:d8:96:8b:15:96:83:
b4:c3

; Модуль (2048 бит).
Exponent: 65537 (0x10001)
; Экспонента.

; Далее идут расширения формата, играющие важную роль в современной инфраструктуре TLS-сертификатов. Ряд из этих расширений имеет прямое отношение к работе TLS.

X509v3 extensions:

X509v3 Key Usage:

Digital Signature, Key Encipherment

; Указание на допустимое использование ключа из сертификата: в данном случае ключ может быть использован для создания подписи и для зашифрования других ключей. Первый вариант необходим при использовании сеансового ключа по алгоритму Диффи-Хеллмана; второй - для передачи клиентом зашифрованного секрета. Значения из этого поля проверяются клиентом, например, браузером.

X509v3 Extended Key Usage:

TLS Web Client Authentication, TLS Web Server Authentication

; Дополнительные параметры использования ключа: допускается применение для аутентификации веб-клиента и веб-сервера (традиционное использование).

X509v3 Basic Constraints:

CA:FALSE

; Ограничения на использование ключа и сертификата: данный сертификат не является сертификатом УЦ, то есть от него не должны выпускаться другие сертификаты. Технически, никаких проблем с выпуском ещё одного сертификата, подписанного ключом данного, нет. Однако при выстраивании цепочки доверия, клиент (например, браузер), должен сверять статусы сертификатов и доверять только подписям, полученным от сертификатов, для которых в этом поле указано CA:TRUE. Интересно, что браузер Microsoft IE длительное время, до 2003 года, некорректно проверял статус сертификата (из-за ошибки в коде), что позволяло выпускать валидные, с точки зрения IE, сертификаты для любого имени, приобретая обычный серверный сертификат у того или иного УЦ.

X509v3 Subject Key Identifier:

37:93:60:97:E7:8A:3C:FF:C6:68:6A:DD:97:92:A5:32:59:87:8A:E4

X509v3 Authority Key Identifier:

keyid:D2:A7:16:20:7C:AF:D9:95:9E:EB:43:0A:19:F2:E0:B9:74:0E:A8:C7

; Специальные идентификаторы ключей, позволяющие клиенту быстрее найти соответствующие ключи в сертификатах в своих хранилищах. Это особенно актуально для поиска подходящего ключа УЦ, так как ускоряет построение цепочки валидации.

Authority Information Access:

OCSP - URI:<http://ocsp6.wosign.com/ca6/server1/free>

CA Issuers - URI:<http://aia6.wosign.com/ca6.server1.free.cer>

; Адреса: ответчика OCSP (Online Certificate Status Protocol) - это протокол онлайн-проверки статуса отзыва сертификата; и сертификата УЦ, от которого выпущен данный сертификат.

X509v3 CRL Distribution Points:

Full Name:

URI:http://crls6.wosign.com/ca6-server1-free.crl

; Существует метод отзыва сертификатов при помощи выпуска списков отозванных сертификатов (CRL - Certificate Revocation List). Данное поле содержит адрес, по которому список доступен.

X509v3 Subject Alternative Name:

DNS:dxdt.ru, DNS:www.dxdt.ru, DNS:tls.dxdt.ru

; Дополнительные имена. Это расширение позволяет указать набор имён и адресов, для которых валиден данный сертификат. Например, здесь, в дополнение к основному имени, указаны: www.dxdt.ru и tls.dxdt.ru.

X509v3 Certificate Policies:

Policy: 2.23.140.1.2.1

Policy: 1.3.6.1.4.1.36305.6.1.2.2.1

CPS: http://www.wosign.com/policy/

; Перечень политик, которым соответствует данный сертификат. Политики устанавливаются УЦ (в соответствии с некоторыми нормами, если УЦ желает, чтобы его корни были включены в распространённые браузеры), но являются чисто административным инструментом, напрямую не относящимся к криптографии, хотя и влияющим на используемые механизмы. Например, именно политика определяет, будет ли сертификат расцениваться как сертификат "Расширенной проверки" (EV).

Signature Algorithm: sha256WithRSAEncryption

09:20:65:da:92:9f:93:d2:96:55:80:51:8e:06:3a:37:c1:26:
33:af:da:2a:63:0e:da:c2:b2:c9:31:80:b3:de:9a:b2:48:ce:
3b:11:ee:a7:81:fc:9e:48:5d:a0:59:6f:b6:d0:e7:da:c9:86:
47:79:ef:ea:71:cd:72:2f:b1:7d:ac:84:88:84:c5:a8:19:47:
4d:6b:1c:e0:8f:81:b0:c9:13:95:c9:f7:27:7a:93:2b:2c:08:
ac:c8:69:2d:0d:4f:c6:4d:b7:18:96:4f:50:c0:0f:23:20:cf:
22:28:5d:fd:e5:89:dd:c9:1d:9c:c6:b3:54:4b:65:e5:8d:2e:
26:07:85:fb:d4:d8:23:ed:30:dc:60:33:da:9e:85:79:37:1d:
ec:04:87:ad:c2:00:7b:87:06:7e:ee:26:27:25:ef:4a:5c:8f:
64:0f:77:28:3f:69:61:37:36:f8:40:78:8d:7d:00:f3:2f:e7:
fb:5b:c6:52:c4:5e:e5:19:54:06:ad:3f:7d:48:19:59:82:ef:
46:96:3b:fc:71:3f:4f:99:94:d5:1d:c2:80:44:e6:af:3c:cf:
01:c2:4c:e7:e8:53:8b:56:ad:06:c7:c9:23:90:7e:93:1d:f5:
f6:2e:29:21:ae:13:23:da:7a:f9:04:ea:62:8c:e3:24:f1:05:
db:0d:e5:1f

; Значение подписи сертификата. Подпись вычисляется от значения хеш-функции, аргументом

которой является весь сертификат (за исключением, соответственно, подписи).

Генерируется подпись при помощи секретного ключа УЦ, соответствующего сертификату с именем, указанным в Issuer.

Итак, TLS-сертификаты являются важным элементом TLS, позволяющим проводить аутентификацию. Ключи из сертификатов используются при генерации общего секрета. В теории, возможно установление анонимного TLS-соединения без использования сертификатов (это не запрещено спецификацией, а серверное сообщение Certificate не является строго обязательным), однако массово применяемые на практике методы генерации сеансового ключа основываются на

использовании серверного сертификата (а точнее - на использовании открытого ключа из сертификата).

Основные современные претензии к TLS-сертификатам касаются сложившейся административной структуры УЦ. Так, пока что никакая распространённая технология не мешает УЦ выпустить "подменный" сертификат, позволяющий прозрачно перехватывать TLS-соединения, используя атаку типа "человек посередине" (см. ниже в разделе про перехват HTTPS). Но такие технологии разрабатываются и успешно внедряются, к ним относится инициатива Certificate Transparency (поддерживаемая Google), Certificate Pinning или Public Key Pinning, когда в браузерах сохраняются (и обновляются) заведомо корректные для данного узла сертификаты или открытые ключи, а также технология DANE, использующая DNS в качестве дополнительной системы контроля.

Шифры в TLS

Как рассказано выше, TLS использует для защиты информации шифронаборы. Шифронабор (Cipher Suite) позволяет обеспечить не только сокрытие информации, но и её целостность. В состав шифронабора входит симметричный шифр. С его помощью осуществляется зашифрование потока данных, передаваемых через TLS-сокет. Возможно использование TLS без шифрования, однако это не является распространённым случаем.

В терминологии TLS существуют зашифровывающие и расшифровывающие функции. Эти функции работают с криптографическим контекстом соединения, о котором договорились узлы. После того, как узлы обменялись сообщениями ChangeCipherSpec (см. выше), все TLS-сообщения зашифрованы, а в TLS 1.3 шифрование включается даже раньше. Зашифровывающая функция преобразует открытый текст, подготовленный для отправки в составе TLS-записи (строго говоря, после сжатия, но в современной реальности TLS сжатие не используется), в секретный текст (зашифрованный). Расшифровывающая функция выполняет обратное преобразование. Для того, чтобы зашифровывающие и расшифровывающие функции успешно работали на обоих узлах, требуется выработка общего криптографического контекста - это происходит в рамках обмена сообщениями Handshake. В TLS могут использоваться и потоковые, и блочные шифры, последние - в различных режимах. Обычно, зашифровывающие и расшифровывающие функции являются одним симметричным шифром и различаются только составом ключей. TLS 1.3 *существенно* ужесточает принципы включения шифронаборов в реализации, фактически, в действующей версии 1.3 допущены только два шифра: AES и ChaCha20.

Шифр - это некоторый набор обратимых преобразований данных, каждое из которых определяется параметром - ключом шифрования. Порядок применения этих

преобразований к входным сообщениям принято называть режимом шифрования. Простейшая криптосистема может включать лишь один шифр, то есть, грубо говоря, состоять из одного алгоритма зашифрования и (обратного к нему) алгоритма расшифрования. В симметричных системах, которые используются в TLS, знание шифрующего ключа позволяет легко получить расшифровывающий ключ, то есть расшифровать секретное сообщение. В используемых сейчас массовых симметричных криптосистемах шифрующий и расшифровывающий ключи просто совпадают, поэтому обычно говорят об одном и том же ключе. Другими словами, симметричные системы подразумевают, что обе стороны, обменивающиеся информацией по закрытому каналу, знают некий общий секрет - например, секретный симметричный криптографический ключ, который позволяет как зашифровывать, так и расшифровывать сообщения. Обратите ещё раз внимание на то, что в TLS используется набор из двух ключей (ключ "записи" и ключ "чтения"), но эти наборы симметричны, на стороне клиента или сервера - ключи просто меняются местами: серверная "запись" становится "чтением" клиента.

Симметричные криптосистемы - первые из придуманных человечеством. Историческими примерами симметричных криптосистем являются так называемые "наивные" шифры — например, шифры простой замены букв текста на естественном языке, где в качестве ключа может выступать та или иная перестановка алфавита. Современные шифры, вроде AES или 3DES, тоже являются симметричными. К симметричным относится и единственная криптосистема, обладающая абсолютной стойкостью - шифр Вернама, с равновероятным одноразовым ключом, равным по длине сообщению (одноразовый шифр-блокнот). Шифр Вернама не очень удобен, поэтому редко используется на практике - только в весьма специальных случаях, требующих повышенной надёжности. Однако основная идея этого шифра послужила прообразом самых защищённых режимов шифрования в TLS.

В основе логики построения современных шифров лежит задача сокрытия статистических свойств исходного открытого текста. То есть, шифр стремятся спроектировать таким образом, чтобы его вывод был *вычислительно неотличим* от результата (псевдо)случайной функции (точнее - перестановки, но это комбинаторные детали). Дело в том, что попытка скрывать лишь сами символы открытого текста не даёт никакой секретности. Игрушечным, но весьма показательным, примером является шифр простой замены, где буквы открытого текста заменяются на какие-то другие символы (например, на специально выдуманные "иероглифы"). Каждая буква неузнаваема, но прочитать такой шифротекст легко, если известно, на каком языке был записан открытый текст, так как статистические свойства открытого текста прямо наследуются: частотность "иероглифов" соответствует известной частотности букв в тексте на исходном языке. Эта классическая "дешифровка" многократно описана в художественной литературе. Соответственно, современные криптосистемы прежде всего направлены на сокрытие

статистических свойств открытого текста, на "размытие" статистики "до уровня случайности". (Важно отметить, что при правильном выборе процедуры генерации ключей, шифр простой замены может быть сделан чрезвычайно стойким, вплоть до невозможности взлома персонажами художественного произведения.)

Криптосистемы строятся на базе некоторых "примитивов" - своего рода строительных блоков, которые выполняют базовые операции над данными. К базовым операциям относятся подстановки, перемешивание (перестановки), сложение и умножение (выполняемые по модулю некоторого числа или ещё каким-то особым способом). Эти операции проводятся над входными данными, рассматриваемыми как набор символов. Обычно, символами являются биты и байты, но возможны и другие варианты: например, слова из четырёх битов или двухбайтовые символы. Для того чтобы составить самое простое представление о современных симметричных шифрах можно представить, что шифр - это некоторый "ящик", на входы которого (в режиме зашифрования) поступают открытый текст и ключ, внутри символы открытого текста перемешиваются по схеме, определяемой ключом, и выводятся из "ящика" в качестве шифротекста. Каждый бит открытого текста оказывает существенное влияние на весь шифротекст: благодаря "лавинному эффекту", изменение даже значения одного бита в открытом тексте (или в ключе зашифрования) приводит к массовым изменениям в соответствующем шифротексте. Для расшифрования используется (другой) "ящик", на входы которого поступают *шифротекст* и ключ, символы опять перемешиваются, в результате на выходе получается открытый текст: если ключи совпали, то этот открытый текст совпадёт с исходным открытым текстом. Далеко не всегда для зашифрования и расшифрования используется один и тот же "ящик": алгоритмы могут различаться. Более того, в современных схемах использования, шифр может применяться только в режиме зашифрования, каким бы странным этот момент ни показался на первый взгляд.

Ключом в массовых современных симметричных криптосистемах является целое число достаточно большой разрядности - 128 бит и более. Если криптосистема добротная, то есть, наилучшим алгоритмом атаки (взлома) для неё является полный перебор всего множества ключей, то 128 бит - более чем достаточная степень безопасности. Предположим, что ключ можно угадать с вероятностью $1/2$, перебрав половину пространства ключей. (Вероятность $1/2$ - это огромный показатель в разрезе криптографической защиты информации. Криптосистема, которую можно взломать с такой вероятностью, никакой стойкостью не обладает.) Тогда, в случае 128-битного ключа, придётся выполнить $(2^{128})/2 = 2^{127}$ операций. Это огромное количество вариантов, перебрать их за обозримое время невозможно. Проблема состоит в том, что для шифров обычно известны оптимизации, которые позволяют уменьшить число перебираемых вариантов. Например, снизить его для 128-битного ключа до 2^{46} . Перебрать 2^{46} вариантов для вычислительно незатратного шифра можно очень быстро, даже если использовать в качестве вычислителя кластер из

устаревших смартфонов или GPU-ферму с несколькими видеокартами. Наличие такой оптимизации является уязвимостью криптосистемы.

Шифры принято делить на блочные и потоковые. В настоящее время такое деление оказывается довольно условным и имеет, скорее, историческое значение. Блочные шифры отличаются от потоковых тем, что работают с блоками фиксированной длины и требуют разбиения потока байтов на такие блоки; в блочном шифре каждый байт обрабатывается в составе блока байтов (битов). Потоковые шифры, напротив, работают прямо с потоком битов (или байтов, потому что байт, вообще говоря, тоже можно считать блоком), без разбиения. Существуют режимы использования блочных шифров, которые позволяют рассматривать их, фактически, как потоковые.

Классический потоковый шифр TLS - RC4, который сейчас считается устаревшим и нестойким. Классические блочные шифры: DES и 3DES, ГОСТ 28147-89.

Особенностью блочных шифров является то, что они требуют использования дополнения (padding): в случае, если открытый текст не кратен размеру блока, его нужно дополнить до соответствующего числа битов. С этим аспектом связано немало уязвимостей в реализациях TLS.

Мы рассмотрим два шифра - AES и ChaCha20 (второй - несколько более подробно), эти шифры закрывают основную часть TLS-трафика в современном вебе.

Шифр AES

AES - является современным блочным шифром, повсеместно используемым в TLS. Строго говоря, AES - это название стандарта шифрования США, сам шифр называется Rijndael (произносится: "рай'н-да'л"). Rijndael выиграл конкурс, проводившийся NIST, и стал стандартом (AES) в 2002 году (сам шифр опубликован в 1998). Шифр стандартизован для использования с 128-, 192-, 256-битным ключом и 128-битным блоком.

AES строится из "раундов" (rounds) - повторных выполнений некоторого набора операций по трансформации блока данных; каждое повторение использует свой ключ. Набор ключей генерируется (разворачивается) из исходного ключа, с использованием специального алгоритма. Так, в случае 128-битного ключа (AES-128), на выходе алгоритма разворачивания ключа - 176 байт или 11 128-битных ключей для повторных трансформаций, составляющих шифр. Разворачивание ключа в AES использует циклические сдвиги ключей, подстановки и операции умножения в конечном поле. Процедура разворачивания ключа является составной частью многих шифров, её надёжность и "необратимость" оказывают огромное влияние на стойкость шифра в целом.

Шифр AES оперирует матрицами (таблицами) размером 4x4 элемента. Такая матрица как раз соответствует 16 байтам 128-битного блока: один элемент - один

байт. Матрица со значениями байтов называется состоянием шифра, одна трансформация (раунд) последовательно выполняет над матрицей и её элементами несколько преобразований, после чего к матрице применяется ключ данного раунда. Результат преобразований переходит в следующий раунд, где всё повторяется, но со следующим ключом.

Базовые преобразования в AES-128 (названия определены в стандарте):

- SubBytes - это подстановка байтов, использующая таблицу подстановок (S-box): байт В заменяется на значение из таблицы, адресуемое значением В. Таблица, соответственно, состоит из 256 элементов. Эта же таблица подстановок применяется и в алгоритме разворачивания ключа. Подобные табличные подстановки используются во многих шифрах (например, в российском ГОСТ 34.12-2015 "Кузнечик"). Они имеют огромное значение для обеспечения стойкости шифра, в частности, стойкости к так называемому "дифференциальному" криптоанализу. Подстановки приводят к тому, что изменение значения одного бита незамедлительно влияет на несколько других, соответственно разрядности подстановки;
- ShiftRows - сдвиг строк. Это преобразование заключается в циклическом сдвиге элементов строк матрицы состояния. При этом первая строка не сдвигается, а вторая, третья и четвёртая - сдвигаются, соответственно, на один, два, три элемента (справа налево). Данное преобразование перемешивает байты между столбцами, связывая шифр в единую схему - отсутствие ShiftRows привело бы к тому, что байты столбцов подвергались бы независимым преобразованиям MixColumns (следующее преобразование);
- MixColumns - перемешивание элементов матрицы состояния (столбцами). В MixColumns над каждым столбцом матрицы состояний, включающем, соответственно, четыре элемента, проводится линейное преобразование. Столбец умножается на заданное значение, оба операнда в данном умножении рассматриваются как элементы конечного поля. Аналогичные преобразования, приведённые к матричной форме, также используются в большом количестве других шифров: ChaCha, Camellia, Twofish и пр. А операция умножения в конечном поле является основой операции аутентификации в режиме шифрования GCM (режим отдельно рассмотрен ниже, в специальном разделе).

После того, как выполнены три преобразования матрицы состояний, к ней применяется ключ действующего раунда. Операция называется AddRoundKey. Значение ключа разбивается на четыре блока по четыре байта и каждый из них применяется к столбцам матрицы состояний при помощи операции XOR. Операция AddRoundKey также выполняется перед первым раундом.

Далее операции шифра повторяются, но в AddRoundKey суммирование проводится со следующим ключом. Последний раунд не включает MixColumns. Для разной

длины основного ключа AES предусматривает разное количество только что описанных трансформаций (раундов): для 128-битного ключа - 10 раундов; AES-192 - 12 раундов; AES-256 - 14.

Запись AES на псевдокоде выглядит следующим образом (из документа FIPS PUB 197):

```
Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
// Nb - число столбцов в матрице состояния;
// Nr - число раундов;
// in, out - входная и выходная матрицы состояний;
// w - поток ключей раундов.
begin

    byte state[4,Nb]
    state = in

    AddRoundKey(state, w[0, Nb-1])

    for round = 1 step 1 to Nr-1

        SubBytes(state)
        ShiftRows(state)
        MixColumns(state)
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])

    end for

    SubBytes(state)
    ShiftRows(state)
    AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])

    out = state

end
```

Преобразования шифра обратимы. Соответственно, можно построить алгоритм расшифрования, который будет восстанавливать открытый текст, при условии наличия ключа. Наличие режима расшифрования, как ни странно, требуется далеко не всегда. Так, современные режимы использования блочных шифров основаны только на зашифровании: шифр используется в качестве генератора последовательности байтов, так называемой "гаммы", которая играет роль длинного ключа, совпадающего по длине с сообщением. *При этом само наличие обратимости (то есть, возможности расшифровать) в схеме шифра является важнейшим фактором обеспечения стойкости даже в том случае, когда используется только зашифрование.*

Гарантированная "обратимость", при условии знания ключа, является причиной того, почему в AES и во многих других шифрах используются конечные поля, как

алгебраическая структура. Дело в том, что конечное поле даёт относительно простой способ задать строгую структуру с нужными свойствами на конечном наборе элементов. То есть, когда отображения внутри множества гарантированно будут взаимно однозначными (биекция), а поэтому можно получить для зафиксированного элемента A "равномерное" отображение элемента X в $X + A$ на той же структуре. В AES, в частности, это делается на шаге MixColumns, при помощи умножения. То есть, здесь свойства конечного поля используется для перемешивания значений, с некоторой гарантией отсутствия наложения результатов. Поэтому, наблюдая статистику на выходе преобразования, будет сложно различить входные значения (это прямо следует из того, что используемые операции биективны).

Если внимательно посмотреть на принципы построения операций AES, то окажется, что большая часть преобразований может быть предвычислена для всевозможных значений отдельных байтов (0-0xFF) и сохранена в таблицах, из которых значения будут извлекаться при работе алгоритма. Это основной алгебраический метод оптимизации AES. Он же помогает составить общее представление о логике построения данного шифра: фактически, AES сводится к извлечению некоторой последовательности элементов матрицы возможных значений и сложении их при помощи XOR (то есть, по модулю 2) с входным блоком данных, при этом сама последовательность определяется используемым ключом, который разворачивается в набор ключей раундов.

Итак, блочный шифр ставит в соответствие блоку байтов открытого текста фиксированной длины и ключу шифрования - блок зашифрованных данных, той же длины. AES является самым часто используемым сейчас в TLS блочным шифром, фактически, это единственный распространённый безопасный вариант. Другие блочные шифры, с которыми можно столкнуться в практике TLS: 3DES (развитие устаревшего и нестойкого шифра DES, стандарта, предшествовавшего AES) и ChaCha20.

Шифр ChaCha20

Рассмотренный выше шифр AES является блочным шифром. Ранее в TLS/SSL широко использовался потоковый шифр RC4. Из-за обнаруженных недостатков RC4 сейчас считается недостаточно стойким и не должен использоваться. Другой вариант блочного шифра, получивший широкое распространение, это 3DES - улучшенный шифр, построенный на базе DES. В качестве современной альтернативы AES активно продвигается шифр ChaCha20, разработанный Даниэлем Бернштейном. В частности, соответствующие шифронаборы использует Google в своих браузерах и на своих интернет-сервисах. Кроме того, сейчас ChaCha20 является единственным, кроме AES, вариантом шифра, который доступен в спецификации TLS 1.3.

В основе ChaCha20 (мы рассматриваем шифр в изложении [RFC 7539](#)) лежит конструкция, эквивалентная блочному шифру. Строго говоря, шифр называется ChaCha, а ChaCha20 - это версия, использующая 20 раундов. Согласно определению, этот шифр является потоковым, но можно рассматривать ChaCha20 и как блочный шифр с "встроенным" режимом счётчика (это будет очевидно из описания шифра ниже). По сравнению с AES, ChaCha20 оказывается несколько проще по алгоритмической структуре и составу операций. Он алгоритмически ближе к "лёгким" шифрам, предназначенным для микроконтроллеров и других "встроенных" применений (например, Speck), что позволяет получить достаточно быстрые, безопасные и легко портируемые чисто программные реализации на разном аппаратном обеспечении. AES, действительно, отличается тем, что реализация с высокой производительностью на архитектурах, не имеющих достаточного объёма памяти и ряда специальных команд, сталкивается с серьёзными трудностями. Однако основной причиной появления ChaCha20 в стандартах и программном обеспечении TLS является необходимость наличия какой-то сравнимой по качеству альтернативы AES, так как единственность шифра ставит под угрозу безопасность протокола: в случае, если в AES будут обнаружены критические дефекты, быстро перейти окажется не на что.

Операции, составляющие шифр ChaCha20, можно разбить на три части. Первая часть, базовая, это функция ChaCha для "четверть-раунда" (Quarter Round, далее QR). Название обусловлено тем, что в раундах шифра данная функция за один вызов преобразует четыре переменные состояния из шестнадцати ($16/4 = 4$), а в каждом раунде - данная функция вызывается восемь раз: по четыре раза в каждой из двух конфигураций (см. ниже).

Вторая часть - функция преобразования состояния (базового блока): эта функция (далее - BF, от Block Function) построена на последовательном преобразовании состояния шифра при помощи QR-функции. Начальное состояние формируется на основе ключа, вектора инициализации (уникального значения, nonce) и счётчика, а результатом является блок ключевого потока длиной в 64 байта.

Часть три - это, собственно, функция зашифрования: вызывает функцию зашифрования блока (BF) с заданным ключом и nonce, последовательно увеличивая значения счётчика, а вычисляемый таким образом ключевой поток (гамму) суммирует с открытым текстом при помощи XOR. На вход поступают: 256-битный ключ (существуют версии шифра с другой разрядностью ключей, но для TLS рекомендован именно 256-битный); вектор инициализации (96 бит); начальное значение 32-битного счётчика; поток открытого текста. На выходе - шифротекст. Шифр ChaCha представляет собой вариант режима счётчика с гаммированием, то есть, каждый бит открытого текста складывается по модулю два с соответствующим битом последовательности, которую выводит шифр. Поэтому расшифрование производится аналогично зашифрованию (вместо открытого текста - шифротекст).

Состояние шифра. Все операции шифра производятся над таблицей, задающей его состояние (это напоминает AES). Таблица состоит из 16 32-битных значений (переменных состояния), которые нумеруются естественным образом - 0..15:

```
00 01 02 03
04 05 06 07
08 09 10 11
12 13 14 15
```

Представление в виде таблицы из четырёх строк удобно по той причине, что элементарные преобразования шифра разделены на два типа: диагональные и вертикальные (по столбцу таблицы). *Начальное состояние шифра* определяется значениями, которые записываются в эту таблицу. Константы, заданные в описании шифра, - записываются в первую, верхнюю, строку таблицы; значения текущего ключа - записываются в две следующих строки ($256/32 = 8$); значение счётчика и попсо - последняя строка:

```
cccccccc cccccccc cccccccc cccccccc
kkkkkkkk kkkkkkkk kkkkkkkk kkkkkkkk
kkkkkkkk kkkkkkkk kkkkkkkk kkkkkkkk
bbbbbbbb nnnnnnnn nnnnnnnn nnnnnnnn
```

c - константы;
k - ключ;
b - счётчик блоков;
n - попсо.

Функция "четверть-раунда" - QR. Функция преобразует четыре входных 32-битных значения (a, b, c, d) при помощи простейших базовых операций: сложение по модулю 2^{32} , XOR, циклический битовый сдвиг. Используются следующие преобразования (+ - сложение; $\oplus$ - XOR; $\ll N$ - обозначает циклический битовый сдвиг влево на N битовых позиций):

- 1) $a = a + b$; $d = d \oplus a$; $d \ll 16$;
- 2) $c = c + d$; $b = b \oplus c$; $b \ll 12$;
- 3) $a = a + b$; $d = d \oplus a$; $d \ll 8$;
- 4) $c = c + d$; $b = b \oplus c$; $b \ll 7$;

То есть, биты аргументов перемешиваются, результат записывается в те же переменные. Обратите внимание, что данная функция не зависит ни от ключа, ни от счётчика - она предназначена только для перемешивания значений состояния шифра.

Пример реализации QR на псевдокоде:

```
func QR(a, b, c, d){
    a = a + b
    d = (d ^ a) << 16
    c = c + d
```

```

    b = (b ^ c) << 12
    a = a + b
    d = (d ^ a) << 8
    c = c + d
    b = b ^ c
    b = b << 7
    return a, b, c, d
}

```

Функция преобразования блока - BF. Данная функция вызывает QR(a, b, c, d), подставляя в качестве аргументов различные наборы переменных из таблицы состояния шифра. Четыре набора соответствуют вертикальному раунду, четыре - диагональному. Пример выбора столбца для четверти вертикального раунда:

```

00 [01] 02 03
04 [05] 06 07
08 [09] 10 11
12 [13] 14 15

```

Четверть диагонального раунда:

```

00 01 [02] 03
04 05 06 [07]
[08] 09 10 11
12 [13] 14 15

```

То есть, вертикальный раунд последовательно обрабатывает четыре столбца, перемешивая значения внутри них, а диагональный - обеспечивает перемешивание между столбцами (комбинаторно, схема аналогична AES). Диагональные раунды чередуются с вертикальными, всего выполняется 10 повторов, состоящих из вертикального и диагонального раунда, что и даёт в сумме 20 раундов. После выполнения 20 раундов, исходное (на момент вызова функции BF) состояние шифра суммируется (mod 2^{32}) с полученным, сумма - выводится как результат функции BF. Наборы (по номерам) переменных состояния шифра, используемые в каждом раунде:

Вертикальные:

```

{ 00, 04, 08, 12 }
{ 01, 05, 09, 13 }
{ 02, 06, 10, 14 }
{ 03, 07, 11, 15 }

```

Диагональные:

```

{ 00, 05, 10, 15 }
{ 01, 06, 11, 12 }
{ 02, 07, 08, 13 }
{ 03, 04, 09, 14 }

```

Пример реализации функции BF на псевдокоде:

```

Block(state){

var back_state = state

for i := 0; i < 10; i++ {
    state[0], state[4], state[8], state[12] = QR(state[0], state[4], state[8],
state[12])
    state[1], state[5], state[9], state[13] = QR(state[1], state[5], state[9],
state[13])
    state[2], state[6], state[10], state[14] = QR(state[2], state[6], state[10],
state[14])
    state[3], state[7], state[11], state[15] = QR(state[3], state[7], state[11],
state[15])

    state[0], state[5], state[10], state[15] = QR(state[0], state[5], state[10],
state[15])
    state[1], state[6], state[11], state[12] = QR(state[1], state[6], state[11],
state[12])
    state[2], state[7], state[8], state[13] = QR(state[2], state[7], state[8],
state[13])
    state[3], state[4], state[9], state[14] = QR(state[3], state[4], state[9],
state[14])
}

state = state + back_state

return state
}

```

Таким образом, BF представляет собой блочный шифр, с разрядностью блока 512 бит - соответствует разрядности записи состояния шифра ($16 * 32$). Реализация, собственно, потокового шифрования ChaCha20 выполняется с применением режима счётчика: последовательно вызывается функция BF, с действующим ключом, действующим значением nonce и значением счётчика, которое для следующего вызова увеличивается (обычно, на единицу). Шифр инициализируется начальным состоянием, и это состояние последовательно изменяется с каждым вызовом BF. Результат работы BF каждый раз выдаёт 64 байта ключевого потока (гаммы), эти байты суммируются функцией зашифрования (XOR) с открытым текстом. Так как длина открытого текста может не быть кратной 64 байтам, для преобразования остатка, аналогично другим реализациям гаммирования в режиме счётчика, используется только часть финального блока ключевого потока (< 64). Режимы шифрования, в том числе, режим счётчика, подробно описаны ниже.

В TLS шифр ChaCha20 используется в режиме аутентифицированного шифрования, основанного на алгоритме Poly1305, также разработанном Бернштейном. Схема в основных чертах напоминает аутентификацию [режима GCM](#), и также позволяет передавать часть данных в открытом виде (с гарантией целостности).

Режимы шифрования

Если длина открытого текста превышает длину блока, то открытый текст для зашифрования блочным шифром потребуется разделить на отдельные фрагменты, соответствующие разрядности блока шифра. Наивный способ использования блочного шифра "в лоб" состоит в прямом разбиении открытого текста на последовательность блоков и в непосредственном шифровании каждого блока в отдельности, чтобы получить шифротекст: $C = E(K, PT)$, где PT - очередной блок открытого текста сообщения; K - ключ; $E()$ - алгоритм блочного шифра; C - шифротекст. Способ использования шифра называют "режимом шифрования". Только что описанный наивный вариант использования блочного шифра называется режимом ECB - Electronic CodeBook.

На практике такой способ *применять нельзя*, так как он не обладает стойкостью. Дело в том, что идентичные сообщения будут порождать идентичные блоки шифротекста (при одном и том же ключе), что приводит к утечке информации об исходном потоке данных. В частности, ECB не скрывает статистические свойства потока. Особенно плохо дела обстоят, если в защищаемом потоке присутствуют структуры, кратные размеру блока - эти структуры будут прямо повторяться в зашифрованном виде, что может выдать очень много информации об исходном, открытом тексте. Широко известен пример зашифрования в режиме ECB большого файла с растровым изображением (обычно, это изображение маскота Linux - пингвина Тукса): из-за того, что большие области исходного изображения имеют одинаковые значения пикселей, - то есть, байтов данных, - узнать, что изображено, нетрудно и на зашифрованном файле: "портятся" только цвета, но не форма. Поэтому на практике в TLS сейчас используются другие режимы работы шифров.

Режим CBC. CBC обозначает Cipher Block Chaining - в данном режиме зашифрованный блок подаётся на вход следующего шага и суммируется со следующим блоком открытого текста при помощи XOR, результат операции XOR зашифровывается, и так далее. То есть, $C_n = E(K, PT_n \oplus C_{n-1})$. Блоки оказываются соединены в цепочку. В качестве первого значения используется вектор инициализации, заданный в параметрах криптосистемы. Схематично, данный режим можно представить следующим образом (данные на схеме преобразуются *сверху вниз*, от блоков открытого текста к блокам шифротекста; IV - вектор инициализации; PT_n - блоки открытого текста, представляющие открытый поток; X - операция XOR; E - шифр):

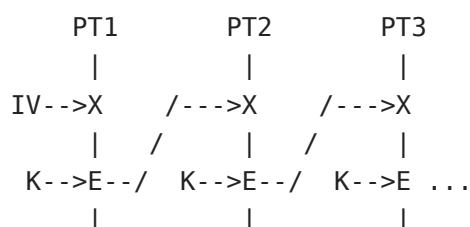


Схема зашифрования в режиме CBC.

Расшифрование использует обратный алгоритм шифра и обратную схему следования блоков. При этом, если для зашифрования следующего блока нужно знать результат зашифрования предыдущего, что диктует последовательную работу функции зашифрования, то для расшифрования произвольного блока достаточно знать значение шифротекста предыдущего: то есть, расшифрование может работать параллельно.

Режим CBC обладает важным свойством: *одинаковые блоки потока открытого текста не порождают одинаковые блоки шифротекста*. Схема позволяет преобразовать поток открытого текста большой длины в связанный поток зашифрованных блоков. Важное значение имеет вектор инициализации (это блок данных, соответствующий разрядности шифра): вектор может генерироваться случайно, либо в его роли может выступать некий счётчик. В общем случае, вектор не должен повторяться в паре с одним и тем же ключом и должен быть непредсказуем для третьей стороны. Скрывать вектор инициализации, вообще говоря, не требуется.

В TLS 1.0 режим CBC спроектирован так, что источником данных инициализирующего вектора для следующей TLS-записи является последний блок предыдущей. Это послужило основой для нашедшей атаке BEAST, которая служит примером обнаружения уязвимости именно на уровне протокола TLS. (Первый вектор инициализации генерируется в TLS 1.0 в рамках создания криптографического контекста и не передаётся между узлами, то есть, остаётся секретным, что, впрочем, не особенно увеличивает степень безопасности протокола.) В TLS 1.1 ситуацию скорректировали: исходные данные для вектора инициализации передаются вместе с зашифрованной записью и периодически изменяются (между записями), сам вектор для очередной итерации вычисляется по специальному алгоритму, который призван затруднить установление корреляции между значением вектора и другими параметрами, доступными для перебора и изменения третьей стороной. В TLS 1.2 вектор инициализации режима CBC передаётся в открытом виде, в самом начале зашифрованной TLS-записи.

Другим подходящим режимом работы блочного шифра является **режим счётчика (CTR)**. В этом режиме функция зашифрования применяется не к блоку открытого текста, а к значению некоторого счётчика. Блок открытого текста суммируется с полученным шифротекстом при помощи XOR. То есть: $C_n = E(K, L_n) \oplus P_n$, где L_n - блок счётчика, значение этого блока своё для каждой операции зашифрования (то есть, для каждого блока), обычно, оно просто увеличивается на единицу для каждого следующего блока. Начальное значение задаётся вектором инициализации. Счётчик не должен повторяться с одним и тем же ключом. Схема работы в режиме счётчика:

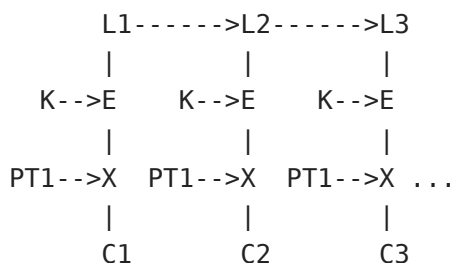


Схема зашифрования в режиме CTR.

Блочный шифр, таким образом, превращается в потоковый: функция зашифрования, будучи применённой к счётчику, порождает ключевой поток (гамму), с которой суммируется открытый текст. То есть, шифр здесь используется в качестве источника псевдослучайной последовательности, которая задаётся начальным значением счётчика L и ключом шифрования. Если предположить, что ключевая последовательность случайна, то получаем схему, которая эквивалентна шифру Вернама. Добротный блочный шифр выдаёт результат, который по статистическим характеристикам должен быть вычислительно неотличим от действительно случайной последовательности.

Для расшифрования в режиме счётчика применяется та же схема, только блоки шифротекста и открытого текста меняются местами. То есть, *для зашифрования* - вычисляется значение XOR от зашифрованного блока счётчика и блока открытого текста. Для расшифрования - XOR от результата зашифрования блока счётчика и *блока шифротекста*. Обратите внимание: сам шифр здесь *используется только для зашифрования*, обратная операция не требуется. Кроме того, в общем случае, не требуется и никакого дополнения последнего блока - лишние байты просто отбрасываются. Режим счётчика - очень эффективен. В TLS, впрочем, сейчас массово применяется развитие данного режима - режим GCM, включающий также и процедуру аутентификации данных.

Коды аутентификации сообщений

Во всех только что описанных режимах (ECB, CBC, CTR) третья сторона, активно вмешивающаяся в канал связи, может произвольным образом заменить фрагменты шифротекста, инициализирующие значения или шифротекст целиком. При этом изменённый блок шифротекста будет обработан на принимающей стороне и "расшифрован" в какое-то значение открытого текста. Это значение *не совпадёт* с исходным открытым текстом, так как блок был изменён во время передачи. Но на сам технический процесс расшифрования это не повлияет - просто, вместо исходного текста получим какой-то "шум". Это, как минимум, позволяет "сломать" поток на принимающей стороне, а как максимум - создаёт возможности для проведения различных атак "перебора". Чтобы защитить данные от изменения используются коды аутентификации сообщений - MAC.

В русскоязычной литературе MAC (Message authentication code - код аутентификации сообщения) называется также имитовставкой. Предназначение MAC - предоставить механизм проверки целостности сообщения, то есть, защитить данные от подмены. В общем случае, код аутентификации - это некоторое значение, вычисляемое для заданного сообщения и (обычно) секретного ключа; значение различается для разных сообщений. Чтобы вычислить корректное значение MAC - требуется знать секретный ключ. Таким образом, если сообщение было изменено третьей стороной, то проверка MAC позволит это выявить. Кроме того, корректное значение MAC позволяет утверждать, что данное сообщение было сгенерировано стороной, которой известен соответствующий ключ. Другими словами - предполагается, что подделка MAC без знания секретного ключа вычислительно недостижима. В TLS - MAC, в той или иной форме, содержится во всех зашифрованных записях. Способ вычисления кода аутентификации зависит от используемого шифронабора и режима шифрования.

НМАС - MAC, основанный на криптографической хеш-функции; используется, например, с шифрами TLS в режиме CBC. НМАС вычисляется независимо от функции зашифрования. Более того, как отмечено выше, в TLS MAC изначально использовался неверно: сперва для открытого текста вычислялся НМАС, а потом сообщение, с присоединённым кодом аутентификации, зашифровывалось. Именно так работает режим CBC в TLS. Внутри записи защищённые данные размещаются вместе с кодом аутентификации, и то, и другое - зашифровано. Архитектурная проблема состоит в том, что MAC можно проверить только после расшифрования сообщения и, если значение оказалось некорректным, реализация TLS должна сообщить об ошибке, из-за этого активный атакующий, манипулируя зашифрованным текстом, может использовать расшифровывающий узел в качестве криптографического оракула, который постепенно раскроет весь секретный текст. На этом основано несколько практических атак на TLS.

В TLS 1.0 (согласно [RFC 2246](#)) НМАС вычисляется на основе хеш-функции, согласованной в криптографическом контексте, например, SHA-1. Значение НМАС - это результат применения хеш-функции к сообщению, в которое "подмешано" значение ключа: $H(K \oplus \text{Pad}_1 + H(K \oplus \text{Pad}_2 + \text{text}))$, где H - хеш-функция, Pad_n - константы, K - секрет, text - исходное сообщение, "+" - конкатенация. Логика построения НМАС в TLS следующая:

$$\text{НМАС} = H_{\text{mac}}(\text{Key}, N + \text{Type} + \text{Ver} + \text{Len} + \text{PT});$$

Здесь:

$H_{\text{mac}}(\text{Key}, \text{Content})$ - хеш-функция, например SHA-1, используемая в режиме НМАС;

Key - секрет, согласованный сторонами для MAC в криптографическом контексте сессии;

N - порядковый номер TLS-записи;

Type и Ver - тип записи и версия протокола;

Len - длина данных записи;
PT - содержание записи;
+ - обозначает конкатенацию.

Таким образом, для защищённого трафика, код аутентификации HMAC зависит от всех полей, определяющих свойства TLS-записи, а не только от полезной нагрузки. При этом ряд полей (N, Type, Ver) - передаются в открытом виде. Обратите также внимание на то, что начальные TLS-записи, при установлении соединения, передаются в открытом, незащищённом MAC виде (поскольку криптографический контекст с полноценной функцией MAC ещё не согласован).

Под влиянием множества различных атак, концепция аутентификации защищаемых криптографическими методами данных эволюционировала. Современным подходом (применительно к блочным шифрам) считается использование шифра в режиме счётчика, объединяющем процесс зашифрования и вычисления MAC. При этом в область действия MAC попадают и данные сообщения, передаваемые в открытом виде. Такие режимы аутентифицированного шифрования называются AEAD - Authenticated Encryption with Associated Data (аутентифицированное шифрование со связанными данными). Фактически, массово в TLS сейчас используется единственный такой режим: GCM - Galois/Counter Mode: режим счётчика с аутентификацией Галуа. Этот режим повсеместно встречается вместе с шифром AES. Второй встречающийся вариант - Poly1305, который применяется в сервисах Google вместе с шифром ChaCha20 (см. выше). В TLS 1.3 разрешены только AEAD-режимы.

Аутентифицированное шифрование - GCM и CCM

Режим GCM (режим счётчика с аутентификацией Галуа) похож на обычный режим CTR тем, что порождает ключевой поток при помощи зашифрования блоков со значением счётчика. Полученные шифротексты складываются с открытым текстом при помощи XOR. GCM строится на двух специальных операциях: *incr* и *Mult*. Первая операция вычисляет значение следующего блока счётчика по предыдущему. Вторая (*Mult*) - выполняет умножение в заданном конечном поле (конечные поля, в инженерных приложениях, нередко называют полями Галуа, откуда и происходит обозначение режима).

Блок шифротекста вычисляется так: $C = E(K, L) \oplus PT$, здесь - K - ключ, L - счётчик, PT - блок открытого текста. Но GCM включает ещё и алгоритм аутентификации. Фактически, *incr* и *Mult* образуют два уровня GCM: первый уровень (*incr*) задаёт исходную гамму, ключевой поток, который позволяет зашифровать данные, второй уровень (*Mult*) - связывает получившиеся блоки шифротекста, действуя подобно хеш-функции, и позволяет построить код аутентификации (AuthTag в терминологии GCM). Схема работы следующая (IV - инициализирующий вектор; Ln - блоки счётчика; K - ключ шифрования; PTn - блоки открытого текста; Cn - блоки шифротекста; A -

связанные данные, передаются в открытом виде; E - шифр; incr, Mult_H - функции режима GCM, Mult_H - использует секретный ключ хеш-функции H; (Bits_num) - конкатенация записей полных длин PT и A в битах; X - операция XOR):

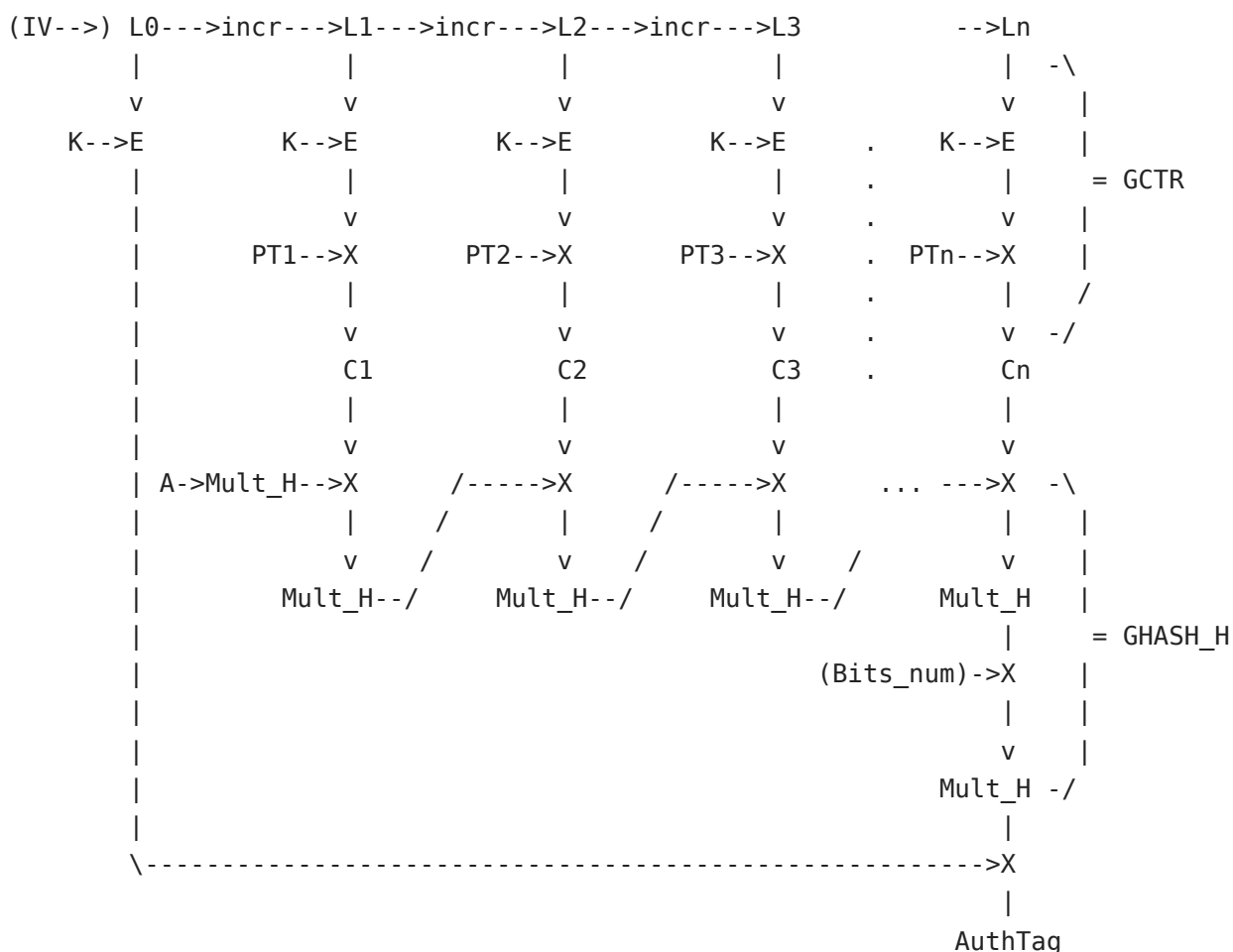


Схема зашифрования в режиме GCM.

Блоки открытого текста суммируются (XOR) с ключевым потоком, что даёт блоки шифротекста - это верхняя часть схемы, режим счётчика, GCTR. Получившиеся блоки шифротекста формируют входные данные функции Mult - ниже на схеме. Mult, в итоге, замыкает всю конструкцию, включая открытую часть сообщения (связанные данные A, в левой колонке на схеме), и вычисляет код аутентификации (AuthTag), который показан на нижней строке схемы. Обратите внимание, что в алгоритме вычисления AuthTag также *учитывается длина сообщения (Bits_num)*. Для работы Mult нужен секретный ключ H, который вычисляется путём зашифрования нулевого блока (блока, состоящего из байтов со значением 0) и, таким образом, зависит от секретного ключа шифра.

Итак, схема GCM достаточно сложна. Она состоит из двух базовых логических уровней, которые в правой части схемы обозначены как GCTR и GHASH. GCTR - это, фактически, режим счётчика; GHASH - это специальная хеш-функция с ключом, предназначенная для вычисления кода аутентификации, по устройству напоминает режим CBC (см. выше), но тут блоки как бы сжимаются в один, так сказать, "нарастающим итогом". Как указано выше, основа GCTR - операция incr (или inc_s в

стандарте NIST). Данная операция устроена достаточно просто: она увеличивает на единицу заданное количество (s) младших битов блока счётчика, рассматривая данные биты как двоичное представление целого неотрицательного числа (≥ 0), старшие биты остаются нетронутыми. GHASH основана на умножении значений (битовых блоков) в конечном поле (разрядность которого соответствует разрядности GCM - 128 бит).

Для работы GCM требуется инициализирующий вектор IV, этот вектор представляет собой число, используемое с данным ключом только один раз (то есть, это nonce) - строго говоря, стандарт требует, чтобы вероятность повторного использования на разных входных данных не превышала 2^{-32} . Таким образом, на входе - вектор инициализации IV, секретный ключ K, открытый текст PT, связанные данные A. На выходе - шифротекст C и код аутентификации AuthTag.

Перед расшифрованием должен быть проверен код аутентификации. Проверка возможна, так как алгоритм вычисления кода использует только шифротекст и связанные данные. Для проверки необходимо знать секретный ключ. Если проверка прошла успешно, то расшифрование данных производится аналогично режиму счётчика - генерируется ключевой поток и, при помощи XOR, вычисляется открытый текст. *При этом код аутентификации проверяется до попытки расшифрования.* Результатом работы функции расшифрования может быть либо открытый текст, либо символ FAIL ($\perp$), обозначающий невозможность расшифрования.

Режим GCM может быть использован с любым блочным шифром подходящей разрядности блока (128 бит, в действующем стандарте). Однако в современном Интернете этот режим в большинстве случаев встречается вместе с шифром AES, в составе шифронаборов AES_GCM. Необходимо ещё раз отметить, что в AES возможны разные длины ключей (128, 192, 256 бит), но разрядность блока всё равно остаётся равной 128 битам.

Режим шифрования CCM. В спецификацию TLS 1.3 включены шифронаборы, использующие AES в режиме CCM (полное название: Counter with Cipher Block Chaining - Message Authentication Code). Этот режим аутентифицированного шифрования решает те же задачи, что и режим GCM, однако существенно отличается от последнего алгоритмически. CCM используется не только в TLS, и не только в версии 1.3 протокола. Так же как и в GCM, в режиме CCM блочный шифр используется только в режиме зашифрования. CCM стандартизован лишь для блочных шифров, имеющих длину блока 128 бит.

Режим CCM представляет собой комбинацию из схемы аутентификации сообщений CMAC и режима счётчика. Используемая схема вычисления кода аутентификации построена аналогично режиму шифрования CBC (см. выше), отличие состоит в том, что промежуточные блоки шифротекста используются только для вычисления

следующего блока, а в качестве значения MAC - используется значение последнего блока. CCM предусматривает, что часть сообщения может передаваться в открытом виде. Открытая часть защищена от подмены данных кодом аутентификации. Как будет видно из схемы, приведённой ниже, основной особенностью CCM является то, что данный режим строго разделяет процессы вычисления кода аутентификации и зашифрования, причём код аутентификации вычисляется для открытого текста, а на следующем этапе зашифровывается. Для проверки целостности данных принимающей стороне придётся сперва расшифровать полученные данные. Это означает, что **внутри** режима шифрования CCM используется схема, в которой код аутентификации не защищает *уже зашифрованные* данные, а сам преобразуется функцией зашифрования (получаем аналог MAC-then-encrypt). В GCM код аутентификации вычисляется для зашифрованных данных (естественно, включая открытую часть AEAD), и проверка целостности сообщения возможна до запуска процедуры расшифрования. Подход GCM позволяет избавиться от некоторых векторов атак.

Режим CCM рекомендован для TLS 1.3, однако поддерживается далеко не везде. (Исторически, одной из основных причин включения CCM в спецификацию является его возможное использование устройствами, которые не имеют аппаратной поддержки функций GCM; другая важная причина - CCM присутствует в спецификациях протоколов беспроводной передачи данных WiFi.) При этом логическая схема CCM ещё сложнее, чем схема GCM.

Итак, рассмотрим режим CCM в соответствии с документом NIST SP 800-38C. Использование в TLS отличается в некоторых деталях (например, определение инициализирующих значений и пр.), однако логика построения - идентична. В абстракции CCM используются две функции: $F()$ и $Cntr()$. Функция $F()$ предназначена для преобразования исходных данных. $F()$ получает в качестве аргументов открытый текст (далее - PT), который будет зашифрован, дополнительные данные (A), передаваемые в открытом виде, а также инициализирующее значение (nonce, N). *Результатом этой функции является последовательность битовых блоков $B_0, B_1, B_2, \dots$, далее преобразуемая в соответствии со схемой для вычисления кода аутентификации.* Фактически, $F()$ просто собирает входные данные в один битовый массив, делит его на блоки нужной разрядности, вычисляет длины и параметры, которые записывает в нулевой (начальный, так как используется сетевой порядок) блок B_0 . Блок B_0 играет служебную роль и содержит значения, определяющие представление данных в полном сообщении. Например, в этом блоке, с помощью битовых флагов, определяется то, присутствуют ли дополнительные открытые данные в сообщении, какую длину имеет запись кода аутентификации и т.д. Важным моментом является то, что в B_0 записывается значение nonce, при этом блок B_0 , как видно из схемы, используется для инициализации схемы CMAC, то есть, в процессе

вычисления кода аутентификации. Это позволяет защитить не только сами данные, но и часть схемы их интерпретации.

Функция $Cntr()$ реализует работу счётчика и генерирует последовательность блоков $L_0, L_1, \dots$, которые поступают на вход, собственно, шифра, а после зашифрования - суммируются с блоками открытого текста при помощи XOR. Как и в режиме CTR блоки счётчика представляют собой битовые блоки с уникальными значениями, которые вычисляются на основе поппе (подходит и простейший вариант, где значение просто увеличивается на единицу). При этом первый (слева) байт каждого блока счётчика в CCM также содержит дополнительные флаги (аналогично B_0).

Работа CCM показана на схеме (A - дополнительные данные, передаваемые в открытом виде; PT, PT_n - открытый текст сообщения (зашифровывается), блоки открытого текста; N - инициализирующее значение; $B_0, \dots, B_n$ - блоки сообщения; $L_0, \dots, L_n$ - блоки счётчика; K - симметричный секретный ключ; E - функция зашифрования; X - XOR; Tag - код аутентификации; C_n - блоки шифротекста):

N, A, PT

$\{B_0, B_1, \dots, B_n\} = F(N, A, PT)$

$\{L_0, L_1, \dots, L_n\} = Cntr()$

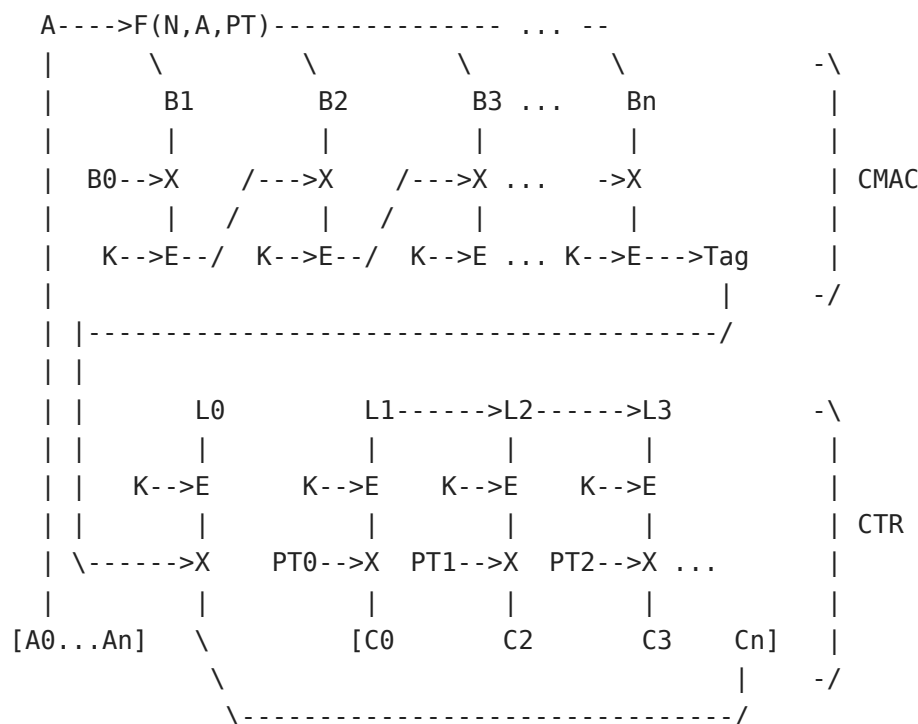


Схема зашифрования в режиме CCM.

Таким образом, на первом этапе битовые блоки (B_n), полученные из входного сообщения, преобразуются в код аутентификации. Преобразования совпадают с режимом CBC. Вычисленный код аутентификации (Tag) зашифровывается при помощи сложения с результатом зашифрования начального блока счётчика (L_0) и

присоединяется в конец шифротекста (Cn). Часть сообщения, подлежащая зашифрованию (PT), суммируется с зашифрованными значениями остальных блоков счётчика и присоединяется к дополнительным данным (A), которые разбиваются на блоки (в том числе, с использованием дополнения, которое требуется для выравнивания длины).

Защищённые записи TLS

Защищённая запись в TLS содержит полезные данные приложения, а также может содержать сообщения Handshake или Alert (в TLS 1.3). От других записей - защищённые отличаются кодом типа: 0x17 (23 в десятичной системе), ApplicationData. Полезные данные внутри записи зашифрованы действующим шифронабором, защищены отдельным кодом аутентификации (MAC) или AEAD-режимом. Часть параметров передаётся в открытом виде: тип записи, версия, длина данных и др. В TLS 1.3 подход к обработке защищённых записей сильно изменился. Так, открытый заголовок зафиксирован в качестве исторического: то есть, защищённая запись TLS 1.3 имеет значение поля версии 0x0303, что соответствует TLS 1.2, однако в самом протоколе 1.3 данное поле не используется. В TLS 1.3 все защищённые записи передаются с "историческим" кодом типа 0x17, при этом подлинный тип записи - содержится внутри защищённых данных.

Рассмотрим пример защищённой TLS-записи версии 1.3. Это запись, содержащая HTTP-запрос GET, адресованный хосту facebook.com и запрашивающий индексный документ. В TLS 1.3 структура защищённой записи, если рассматривать её в открытом виде, следующая: к байтам полезных данных приписан байт с кодом типа записи, следом за этим байтом могут идти нулевые байты дополнения. *То есть, если после успешного расшифрования защищённых данных последний байт оказывается нулевым, это означает, что использовано дополнение и все нулевые байты в конце нужно отбросить.* В случае, если дополнения нет - последний байт корректной записи не может быть нулевым, так как он содержит код типа записи.

Смещение (десятичное)	Байты (шестнадцатеричное представление)	Комментарий
0000	17	; Исторический заголовок. Код типа:
0x17 = 23 (Application Data).		; В TLS 1.3 этот тип строго
зафиксирован, но не используется для		; определения типа записи.
Например, запись Handshake в TLS 1.3		; имеет тип 0x16. Если такая запись
передаётся в защищённом виде,		; то в данном заголовке для неё всё
равно будет указано значение 0x17.		

0001	03 03	; Историческое значение версии:
0x0303 = TLS 1.2. Значение		; строго зафиксировано и не
является обозначением версии в TLS 1.3.		; Код версии TLS 1.3 - 0x0304, но в
заголовке записи он не указывается,		; так как узлам действующая версия
протокола известна из контекста,		; выработанного в процессе
установления соединения.		
0003	00 5D	; Длина данных. 0x005D = 93 байта.
(Длина указывается действующая.)		
0005	7E 6F B1 0F DB D3 7C E3	; Далее идут защищённые данные. В
данном случае, используется	8C 0F 7A 89 F6 0B 0D D1	; шифр AES в режиме GCM. При
зашифровании заголовков сообщения был	24 91 58 EC D0 B7 BE 85	; присоединён (слева) к защищаемым
данным в качестве "связанных данных",	10 9C AC CC 9A 70 08 99	; что соответствует и схеме AEAD
GCM и логике построения TLS-записи.	55 D2 5D 93 6D 3F 20 91	; Таким образом, данные заголовка
передаются в открытом виде, но защищены	26 F4 A2 02 27 74 2C 57	; кодом аутентификации.
больше, чем длина исходных данных (ниже).	AE B9 CC F2 6A E5 8B 5A	; Длина блока защищённых данных
результату зашифрования приписывается код	F3 14 CE AD C4 46 42 3D	; Это объясняется тем, что к
128 бит, что составляет 16 байтов.	52 8F 58 37 9E D8 EA 77	; аутентификации GCM, имеющий длину
	30 D2 0B B3 77 30 DE 36	
	FD 2C 79 FA EA 65 DF F0	
	22 6C A3 3B 08	
=== Исходные данные, в открытом виде:		
	47 45 54 20 2F 20 48 54 54 50 2F 31 2E 31 0D 0A	
; GET / HTTP/1.1\r\n	48 6F 73 74 3A 20 66 61 63 65 62 6F 6F 6B 2E 63 6F 6D 0D 0A	
; Host: facebook.com\r\n	55 73 65 72 2D 41 67 65 6E 74 3A 20 54 4C 53 2D 62 6F 74 2F	
; User-Agent: TLS-bot/	30 2E 32 20 54 4C 53 2D 31 2E 33	
; 0.2 TLS-1.3	0D 0A 0D 0A	
; \r\n\r\n	17	
; Тип записи (актуальный код) 0x17 = 23.	00 00 00 00 00	
; Дополнение: пять нулевых байтов.		

Длины ключей и сочетание криптосистем

С шифронаборами в TLS связано несколько криптографических примитивов, к которым применимо понятие разрядности. Возьмём в качестве примера шифронабор TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 - здесь есть разрядность ключа RSA (рекомендуемое значение - 2048 бит или более); разрядность симметричного ключа AES (указано 128 бит) и, наименее важная, разрядность хеш-функции (256 бит). Есть ряд рекомендаций по выбору разрядности ключей для криптосистем в TLS. Так, максимальная степень защиты должна применяться к сеансовым данным, потому что ради их защиты TLS и используется. Эти данные защищает симметричный шифр. 256 бит для AES считаются достаточными. Большие длины ключей для добротного симметричного алгоритма не имеют никакого смысла. В качестве симметричного шифра может служить не только AES, но и CAMELLIA, и даже корейский шифр ARIA. Возможно и использование шифров ГОСТ для TLS, если они поддерживаются программной реализацией (см. [Приложение В](#)).

Криптосистемы с открытым ключом служат в TLS для аутентификации. Заметьте, что задача выработки общего секрета - эквивалентна аутентификации узла, с которым устанавливается соединение. Действительно, при использовании слабого ключа аутентификации, атакующая сторона может перехватить сессию, выдав себя за сервер (например). Однако на практике это означает проведение сложной активной атаки, которая представляет собой менее вероятную угрозу, чем простое прослушивание канала, от которого защищает симметричный шифр. Естественно, перехвативший сессию атакующий уже не будет испытывать проблем со взломом симметричного шифра. Тем не менее, защита процедуры аутентификации нередко перемещается на второе по важности место, после защиты потока данных.

Хорошим примером, почему это происходит, является следующее наблюдение (подсказанное [З.Т. в комментариях на dxdt.ru](#)): если вы используете нестойкий алгоритм для аутентификации узла, то, в обозримой перспективе, например, через десять лет, этот алгоритм может быть взломан; взломавшая алгоритм сторона получает возможность подделывать сессии, но как атаковать сессию десятилетней давности? Никак. А вот если взломан шифр, защищающий передаваемый трафик, то становится возможным прочесть трафик, записанный десять лет назад. В этом может быть смысл, так как трафик, возможно, содержит до сих пор актуальные сведения.

Весьма полезной практикой является разумное выравнивание стойкости используемых криптосистем. Например, в подавляющем большинстве случаев не имеет смысла использовать RSA-ключ длиной в 4096-бит, если ваш TLS-сервер всё ещё поддерживает SSLv3, а в качестве симметричного шифра применяет DES с 56-битным ключом.

Основные направления атак на TLS

TLS имеет дефекты на уровне протокола, но гораздо больше дефектов обнаруживается в его реализациях. Например, до сих пор можно нередко встретить поддержку заведомо нестойких, устаревших шифронаборов, использующих ключи длиной менее 128 бит. В 2016 году началась активная миграция в сторону стойких сочетаний ключей и шифров в области HTTPS. Но проблема устаревших и слабых криптографических параметров остаётся актуальной для "мобильных приложений" (приложений, предназначенных для смартфонов) и многих видов встроенного программного обеспечения, которое не обновляется. Встречается даже поддержка SSLv2, который давно не является хоть сколь-нибудь защищённым на практике. Использование устаревших протоколов и шифронаборов означает, что атакующий может провести понижение уровня защиты до шифра, который сможет взломать на лету, после чего подделает сообщения Finished, став, тем самым, посредником в сессии. (Посредник в TLS может прослушивать весь трафик.) Однако современные браузеры не позволяют использовать заведомо нестойкие сочетания криптосистем и не поддерживают устаревшие версии SSL/TLS, что сводит поверхность атаки в вебе к минимуму. Тем не менее, неверно было бы считать, что проблем здесь нет: существует большое число устаревших клиентских устройств, которые всё ещё поддерживают старые протоколы, по историческим причинам среди них немало важных узлов, это, например, домашние WiFi-роутеры.

Выше уже упоминалась необходимость дополнения данных при использовании блочных шифров, которая приводит к появлению нежелательных криптографических оракулов (такой оракул помогает "угадывать" секретные тексты). Есть целый класс оракулов, - которые в англоязычной литературе называются Padding oracle, - породивший несколько нашумевших атак. Одно из первых использований оракула, связанного с дополнением данных, относится к 2003 году. Атака, сконструированная Сержем Воденэ (Serge Vaudenay), основывалась на том факте, что реализация протокола SSL возвращала разные сообщения об ошибках в зависимости от того, удалось ли после расшифровки записи обнаружить корректное дополнение, но не совпал код аутентификации ([MAC](#)), или корректного дополнения данных обнаружить не удалось. Именно последствия этой атаки были устранены только спустя семь лет в веб-сервере IIS. Не останавливаясь на только что упомянутой атаке, рассмотрим подробнее это важное направление.

Код аутентификации представляет собой некоторое число, записанное в виде последовательности байтов, которые прикрепляются к защищаемым TLS данным. Принимающая сторона вычисляет значение кода аутентификации для принятых данных и сравнивает его с прикреплённым кодом. Если значения совпали, то делается вывод о неизменности данных, если не совпали, то данные должны быть отброшены. Дополнение сообщения (padding) также входит в набор данных, для которого осуществляется вычисление кода аутентификации. В добротной системе -

внешняя сторона не должна иметь инструмента, который позволяет различить результаты обработки сообщений, кроме разделения на принятые и отвергнутые. Если же утекает дополнительная информация о внутреннем состоянии реализации криптосистемы, то оказывается возможным создание оракула, который позволит при помощи простого перебора раскрыть либо какую-то часть защищаемого сообщения, либо восстановить его полностью.

Рассмотрим гипотетическую схему, которая концептуально схожа с реализациями, послужившими источником уязвимостей TLS/SSL - POODLE, BEAST, SWEET32 и др. Пусть в нашей теоретической криптосистеме блочный шифр работает в режиме [CBC](#) и используется следующий алгоритм дополнения до полного блока: в конец блока дописываются байты со значением, равным количеству байтов в дополнении. То есть, если требуется дописать три байта, то все три байта имеют значение 03. Если блок дополняется пятью байтами, то со значением 05, и так далее. Дополнение вносится в открытый текст, перед зашифрованием. На стороне получателя проверяются значения всех байтов дополнения, начиная с самого последнего байта (мы не рассматриваем способ получения принимающей стороной информации о длине дополнения).

При расшифровании в режиме CBC выполняется операция XOR предыдущего блока шифротекста со значением, полученным в результате расшифрования текущего блока (само это значение ещё не является открытым текстом). На схеме это выглядит следующим образом:

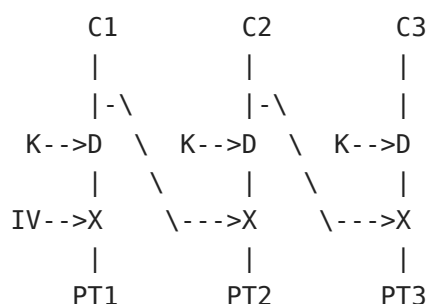


Схема расшифрования CBC.

Здесь $C_1, \dots, C_n$ - блоки шифротекста; PT_n - блоки открытого текста; IV - инициализирующий вектор; X - операция XOR; K - ключ расшифрования; D - алгоритм расшифрования (шифр). Дополнение содержится в последнем блоке, C_3 (но не весь этот блок - дополнение). Схема является обратной к приведённой в разделе "[Режимы шифрования](#)". Код аутентификации на схеме не показан - предположим, что код просто дописывается в конец сообщения.

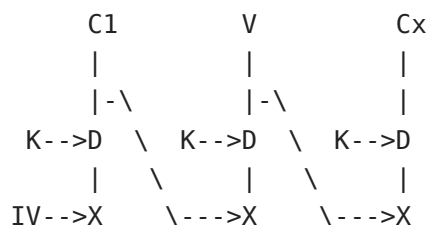
Схема работы нашей гипотетической криптосистемы. Если сообщение, состоящее из набора блоков $\{C_1, C_2, C_3\}$, где C_3 содержит дополнение, успешно расшифровано, - то есть, после преобразования получено корректное дополнение, - криптосистема

переходит к проверке кода аутентификации. Если в процессе расшифрования получено некорректное дополнение, то возвращается сообщение об ошибке расшифрования (присвоим этой ошибке обозначение PAD_ERROR), а код аутентификации не проверяется. (В случае, когда не совпал код аутентификации - возвращается *другое* сообщение об ошибке, пусть оно называется MAC_ERROR.) В данной схеме байты дополнения расположены в конце последнего блока.

Предположим, что число этих байтов известно третьей стороне, которая проводит активную атаку. Пусть теперь дополнение состоит из одного байта, имеющего, соответственно, значение 01. Тогда последний блок открытого текста имеет следующую структуру: {PT_n}[01], где {PT_n} - последовательность байтов открытого текста, а [01] - байт дополнения до полной длины блока.

Заметим, что атакующей стороне известны все блоки шифротекста (они могут быть пассивно перехвачены из канала связи). Мы допускаем активную атаку, поэтому возможна произвольная замена блоков. Заменяя блок C2 (см. схему ниже) на произвольный блок V, а блок C3 - на *некоторый полный блок шифротекста Sx*, и отслеживая сообщения об ошибках, выдаваемые криптосистемой, атакующая сторона может добиться ситуации, когда результат расшифрования блока Sx и последующей операции XOR с блоком V (подменный блок) даст значение 01 в последнем байте. Это значение является результатом операции XOR последнего байта блока V и последнего байта расшифрованного блока Sx. Повторное выполнение XOR с байтом блока V и значением 01, восстанавливает последний байт из расшифрованного Sx.

То есть, атакующий **узнал значение последнего байта из подставленного в конец цепочки блока Sx, который может являться произвольным блоком с защищёнными данными** - это ключевой момент атаки: значение последнего байта последнего блока (дополнение) и так было известно, однако замена блока на другой позволяет навязать криптосистеме задачу расшифрования блока, ни один байт которого не был известен атакующему. Пока значение последнего байта, полученного после расшифрования, не совпадало с ожидаемым значением дополнения - криптосистема выдавала ошибку PAD_ERROR, это позволяло атакующей стороне определить, что байт не угадан; как только значение совпало с 01 - криптосистема выдала ошибку MAC_ERROR, что явилось сигналом об успешном подборе. Такое поведение криптосистемы и является криптографическим оракулом. Так как целостность сообщения проверяется после того, как проведено расшифрование, становится возможной последовательная манипуляция блоками.



|
PT1
|
PT2
|
PT3

Подстановка блока шифротекста.

В описанной схеме, после того, как стал известен последний байт, можно, урезав фиктивное значение длины открытого текста, перейти к предпоследнему, и так далее: так как известно, какую маску нужно использовать, чтобы каждый следующий байт фиктивного дополнения соответствовал общему числу байтов в этом дополнении. Другими словами: атакующему теперь известно, какое значение имеет последний байт расшифрованного блока C_x , соответственно, он может так подобрать маску в виде подменного блока V , что результат XOR будет равен не 01, а 02, как требуется для дополнения из двух байтов (длина дополнения соответствует значению байта дополнения); соответственно, это позволяет перейти к подбору следующего байта (предпоследнего). Манипулируя длиной сообщений и значениями блоков, третья сторона может последовательно раскрыть любое количество байтов секретного текста за обозримое время: очевидно, что один байт будет гарантированно раскрыт за 256 попыток (максимально), а хорошая практическая вероятность около 1/2 достигается в 128 попыток.

Схема атаки, для случая с одним байтом дополнения:

$$\begin{aligned}
 (C_x) & [B1][B2][B3][B4][B5][B6][B7][B8] \\
 & \text{Dec}(K) \\
 & [A1][A2][A3][A4][A5][A6][A7][TT] \\
 & \text{XOR} \\
 (V) & [V1][V2][V3][V4][V5][V6][V7][V8] \\
 & = \\
 & [F1][F2][F3][F4][F5][F6][F7][PP] \\
 \\
 [TT] & = [PP] \text{ XOR } [V8]
 \end{aligned}$$

Если код аутентификации вообще не используется, то схема атаки не меняется. Однако при совпадении значения дополнения - расшифрованные данные окажутся мусорными, так как атакующая сторона проводит подмену блоков. Соответственно, приложение, использующее эти данные, может работать неверно. Тем не менее, факт отсутствия ошибки расшифрования послужит сигналом об успешном результате обработки очередного блока.

Обратите внимание, что если атакующий может выбрать открытый текст, то данная схема уже не зависит от того, проверяет ли сервер все байты дополнения, или только последний. Этот момент используется в построении атаки POODLE.

Рассмотрим теперь как работает конкретная атака на TLS/SSL, использующая уязвимость [POODLE](#) в SSLv3 (а также в некоторых реализациях TLS 1.0). Эта уязвимость построена на описанном только что "оракуле дополнения", однако схема

отличается: предполагается, что атакующий может не только переставлять блоки внутри защищённого сообщения, но и изменять часть открытого текста.

Пусть атака проводится на HTTPS-соединение, а атакующий может отправлять HTTP-запросы из браузера пользователя (например, заманив пользователя на специально подготовленную страницу, что несложно сделать, если уже есть возможность модификации трафика). Целью атакующего является получение содержимого файлов cookie, адресованных защищённому веб-ресурсу. В этом случае, POST-запросу, отправляемому на адрес защищённого HTTPS-ресурса, - работающего по протоколу SSLv3 с шифронабором, использующим режим CBC, - будет соответствовать несколько блоков шифротекста. Cookie-файл передаётся в этой же цепочке блоков, и, согласно протоколу HTTP, следует после адреса документа (path), на который отправляется запрос POST (тут возможны отличия в деталях, но расположение блока, который интересует атакующего, так или иначе, всегда известно точно). Идея состоит в том, чтобы, манипулируя длинной URL в POST-запросе, сдвигать значение Cookie так, чтобы его байты последовательно оказывались на последнем месте в одном из блоков цепочки.

В SSLv3 фрагмент записи, содержащий код аутентификации сообщения (MAC) предшествует дополнению. А само дополнение может представлять собой один полный блок, при этом последний байт дополнения содержит значение, равное числу байтов дополнения. Шифротексты последовательных сообщений, содержащих генерируемые клиентом HTTP-запросы, будут различаться, как того требует протокол. В рамках атаки, блок, содержащий cookie-файл, переставляется в конец записи, на место блока с дополнением. POODLE использует в качестве оракула только последний байт, то есть - байт, в котором записана длина дополнения. Сервер, при попытке обработать данные, будет возвращать ошибку SSL в случае, если значение последнего байта блока после расшифрования не совпало с корректным (ожидаемым) значением дополнения. Если значения совпали, то сервер корректно обрабатывает HTTP-запрос. Перебор возможен потому, что при каждом следующем запросе, с тем же открытым текстом, блоки шифротекста будут иметь другое значение (это одно из основных свойств TLS, рассмотренное выше). Для проверки следующего байта - атакующий модифицирует открытый текст, - то есть, HTTP-запрос, - таким образом, что значение cookie-файла смещается на один байт вправо (к концу записи), и на границе блока оказывается следующий байт cookie.

Видимое "снаружи" различное поведение программной реализации TLS в зависимости от того, были в TLS-записи обнаружены корректные дополнение и MAC или некорректные, - традиционный фундамент для атак, актуальный до сих пор. Эти атаки и привели к тому, что SSLv3 и более ранние версии перешли в разряд нестойких. Шифры, работающие в режиме GCM - а именно, AES, - не используют уязвимого дополнения блоков, соответственно, ликвидируют основу для

возникновения оракулов такого типа (но, конечно, не избавляют от возможных оракулов полностью).

Атака под названием [Sweet32](#) - относится к классу атак, использующих фундаментальные особенности протокола, а не уязвимости конкретных реализаций. Sweet32 так же требует работы в режиме CBC, но при этом зависит от шифра - он должен использовать 64-битные блоки (откуда название: $32=64/2$). Атака основана на достаточно простом, но поэтому весьма универсальном, свойстве: так как результат работы добротного шифра должен быть вычислительно неотличим от случайного результата, в криптосистеме в какой-то момент обязательно произойдёт коллизия - возникнут два блока с одинаковыми значениями. Из-за парадокса дней рождения, вероятность такой коллизии для шифров с малой разрядностью блока становится достаточно высокой на объёмах данных, которые, по современным меркам Интернета, не являются недостижимыми. Так, для 64-битного блока граница находится около 32 гигабайт. Sweet32 и подобные атаки не требуют наличия каких-то дефектов реализации или возникновения плохо предсказуемых ошибок в криптосистеме. Причина, по которой возможны такие атаки, архитектурная.

Рассмотрим блочный шифр с 64-битным блоком, работающий в режиме CBC, с одним и тем же значением ключа. Коллизия блоков шифротекста означает, что результатом работы шифра оказались два блока с одинаковым значением. Присвоим этим блокам номера i и k : $CT_i == CT_k$. Соответственно, так как ключ не изменялся, а шифр эквивалентен *перестановке битов*, два блока, поступившие на вход шифра, равны между собой: $IN_i == IN_k$ (обозначим эти блоки как IN , поскольку они, согласно режиму шифрования, не являются открытым текстом в контексте криптосистемы). Шифр работает в режиме CBC, это означает, что на вход шифра (блоки IN) поступали результаты суммирования предыдущего блока шифротекста (CT_{i-1} , CT_{k-1}) с соответствующими блоками открытого текста (PT_i , PT_k), таким образом: $CT_{i-1} \oplus PT_i == CT_{k-1} \oplus PT_k$, следовательно, по свойствам XOR: $PT_k \oplus PT_i == CT_{k-1} \oplus CT_{i-1}$. Оба блока CT_{k-1} и CT_{i-1} - известны, так как они передаются в открытом виде, поэтому прослушивающая канал сторона может вычислить значение XOR от двух блоков открытого текста. Это и составляет утечку. С вероятностью около $1/2$ коллизия блоков произойдёт на выборке в $2^{n/2} = 2^{32}$ блоков (n - разрядность блока, 64 бита).

Если атакующему известно значение XOR от двух блоков открытого текста (S), а также значение одного из этих блоков, то возможно вычислить второй ($PT_1 \oplus S == PT_2$). В случае с HTTPS некоторые фрагменты открытого текста, действительно, известны заранее, из свойств протокола HTTP и метаданных, например: URL, некоторые файлы Cookie, заголовки запроса, части веб-страницы (её можно получить с сервера) и так далее. Авторы практической атаки Sweet32 использовали для генерации трафика Javascript-код, отправлявший большое количество HTTP-

запросов с известной частью открытого текста. В качестве транспорта использовалось TLS-соединение с 64-битным 3DES (шифр, основанный на стандарте DES). Целью служили cookie-файлы, дописываемые браузером (это типовой метод демонстрации осуществимости атак на HTTPS). Первая коллизия была зафиксирована при количестве переданных блоков $2^{31.3}$, а полное раскрытие 16-байтного секрета потребовало 705 гигабайт трафика.

Данные объёмы трафика достаточно велики, несмотря на то что являются вполне достижимыми на практике. Кроме того, применительно к TLS, требуется, чтобы весь трафик передавался в рамках одного контекста сессии, так как замена ключей и векторов инициализации приведёт к тому, что атака не сработает из-за отсутствия коллизий. При этом, 705 гигабайт соответствует, приблизительно, $2^{36,46}$, что превышает порог $2^{n/2} == 2^{32}$ блоков, после которого должна была бы последовать замена ключей. Для шифров с большей разрядностью блока, например, AES-128, данная атака оказывается ограничена только тем фактом, что резко возрастает требуемое для достижения практической вероятности возникновения коллизии число блоков, что, естественно, не отменяет необходимости периодической замены ключей.

В описании атак, основанных на подмене блоков и байтов дополнения, отмечено, что сигналом о "правильной" интерпретации байтов дополнения могут служить различные коды ошибок, возвращаемые реализацией TLS, либо сам факт успешной обработки запроса протокола, находящегося уровнем выше TLS (HTTPS в случае с POODLE). Однако атакующий может извлечь полезную информацию и путём анализа времени обработки данных на стороне реализации TLS. На таком побочном канале утечки работает атака [Lucky13](#). Основная идея Lucky13 состоит в отправке на сервер специальным образом подготовленных TLS-записей, содержащих тщательно подобранные дефекты. Обработка этих записей на сервере и, в частности, вычисление кода аутентификации, приводит к тому, что сообщение об ошибке поступает с различной задержкой по времени. То есть, уязвимые реализации используют один универсальный код ошибки (следовательно, не содержат оракулов, подобных описанным выше), но время генерации этого кода коррелирует с данными открытого текста, расположенного внутри TLS-записей. Построив статистику времени обработки запросов, которые были специально подготовлены, можно вычислить открытый текст. То есть, в данном случае, криптографический оракул "отвечает" атакующей стороне посредством изменения времени выполнения тех или иных операций на сервере. Подобные побочные каналы утечки являются одними из самых сложных, как в плане выявления, так и в плане противодействия.

В 2015 году появилась атака [Logjam](#), связанная с использованием устаревших шифронаборов, а именно с протоколом Диффи-Хеллмана в так называемом "экспортном" варианте. "Экспортные" шифры относятся к первому периоду

криптовойн, к 90-м годам 20 века, когда был запрещён экспорт стойкой криптографии за пределы США, а вместо стойкой - экспортировались заведомо нестойкие криптосистемы. Logjam не использует оракулов, но позволяет провести понижение уровня секретности, навязав клиенту и серверу нестойкий вариант DH, на группе малой разрядности (512 бит). Это возможно из-за архитектурного недостатка протокола TLS: предложенные клиентом шифронаборы никак не удостоверяются, при этом клиент принимает любые параметры DH, переданные сервером, если только может их обработать. Используя сравнительно небольшие компьютерные мощности, для групп DH малой разрядности (1024 бита и меньше) возможно провести предварительные вычисления арифметической структуры, которые позволят в дальнейшем вычислять дискретный логарифм, фактически, в режиме реального времени. Соответственно, перехватывающий узел может раскрыть сеансовый ключ. (В распространённых браузерах проблема была исправлена летом 2015 года.)

Logjam использует следующую архитектурную особенность TLS: описанный выше [процесс установления соединения \(Handshake\) в TLS версий ниже 1.3](#) подразумевает, что фактическая аутентификация клиентом серверных параметров криптосистемы (выбранный шифронабор и т.д.) проводится лишь в самом конце. Основой такой аутентификации является сообщение Finished, которое передаётся в защищённом виде и должно подтверждать соответствие выбранных параметров состоянию сервера. При этом дополнительной аутентификации сообщений клиента, на начальном этапе, не предусмотрено. Атакующая сторона может осуществить атаку "Человек посередине" и, например, подменить в самом начале соединения клиентское сообщение ClientHello, содержащее идентификатор выбранного сервером шифронабора.

На первый взгляд, подобные подмены не могут достичь успеха: во-первых, сервер получает Finished от клиента, а это сообщение содержит значение хеш-функции от всех предыдущих сообщений и удостоверено кодом аутентификации; во-вторых, ответное сообщение ServerKeyExchange содержит подпись сервера, которая может быть проверена клиентом, а перехватывающий узел не сможет поменять сессионные параметры так или иначе. Но оказывается, что как раз здесь и скрывается возможность для атаки. Дело в том, что ServerKeyExchange никак не привязано к выбранному шифронабору, это сообщение содержит лишь параметры; в случае классического протокола Диффи-Хеллмана - это задающий группу модуль и открытый ключ сервера. Ключевой момент Logjam в том, что для клиента все параметры Диффи-Хеллмана в ServerKeyExchange выглядят одинаково. То есть, достаточно, чтобы сервер выбрал шифронабор с DHE (сеансовый вариант Диффи-Хеллмана) и передал любые подходящие для атаки параметры в ServerKeyExchange.

Пусть сервер поддерживает "экспортные" варианты протокола Диффи-Хеллмана: DHE_EXPORT. Предположим, что активно перехватывающий соединение узел

подменяет исходное сообщение клиента ClientHello и заменяет список шифронаборов на список, разрешающий только варианты с DHE_EXPORT. В ответ, помимо ServerHello, сервер отправляет ServerKeyExchange, содержащий параметры DHE в "экспортном" варианте, то есть, с модулем разрядностью в 512 бит. Параметры при этом подписаны серверным секретным ключом. Узел, проводящий атаку, заменяет в ServerHello выбранный шифронабор на любой вариант с DHE (уже не с "экспортным"), а на основании открытого ключа в ServerKeyExchange и параметров из ClientHello и ServerHello вычисляет сеансовый ключ. Очевидно, что, используя данный ключ, перехватывающий узел сможет подделать все прочие сообщения (кроме ServerKeyExchange). Клиент, получив в поддельном сообщении ServerHello указание на шифронабор с DHE, принимает параметры из ServerKeyExchange, так как это подлинные параметры, подписанные серверным ключом - единственная их особенность состоит в том, что используется модуль малой разрядности.

Для того, чтобы данная атака сработала, необходима поддержка DHE_EXPORT на сервере и поддержка групп DH малой разрядности клиентом. Ни то, ни другое - не является необходимым для нормальной работы TLS в современных условиях.

Другим примером атаки, использующей устаревшие экспортные ограничения против современных версий TLS, является [DROWN](#), опубликованная в 2015-2016 годах (CVE-2016-0800). DROWN является развитием атаки Блейхенбахера (Daniel Bleichenbacher) и использует соединения по морально устаревшему протоколу SSLv2 для расшифрования сеансовых ключей, которые передаются в сессиях TLS. Для того, чтобы атака стала возможной, необходимо наличие SSLv2-сервера, использующего общий с атакуемым TLS-сервером секретный ключ RSA. Такая ситуация вполне возможна: например, один и тот же TLS-сертификат может быть установлен и на веб-сервере, и на почтовом сервере, а это, в свою очередь, требует использования общего секретного ключа. Со стороны TLS-сервера не требуется наличие каких-то уязвимостей: атака работает против корректной реализации протокола TLS. Единственное существенное ограничение: в рамках атакуемой TLS-сессии клиент и сервер должны использовать для передачи сеансового секрета статический вариант с шифрованием RSA (как описано выше, такой способ не рекомендуется, но, тем не менее, до сих пор используется, в том числе, в банковских системах дистанционного обслуживания, где такой выбор обусловлен желанием инспектировать трафик при помощи относительно простых аппаратно-программных средств и методов).

Часть атаки DROWN в отношении TLS-сервера может быть реализована пассивно: то есть, только с использованием записанного трафика (а именно - TLS Handshake). Однако для завершения атаки потребуется активный доступ к связанному SSLv2-серверу - отправка запросов, имитирующих установление соединения SSLv2.

Атака основана на особенностях установления соединения SSLv2, в варианте, использующем те же самые "экспортные" шифронаборы, которые упоминались выше. Кроме того, ключевую роль играет вариант дополнения блока данных RSA, что совпадает с описанными в начале этого раздела криптографическими оракулами, возникающими при использовании симметричных шифров и дополнения (несмотря на то, что RSA - не является симметричным шифром). Установление соединения в SSLv2 существенно отличается от TLS (как протокол, SSLv2 представляет скорее исторический интерес, поэтому мы не рассматривали его в рамках настоящего описания; кроме того, формально, SSLv2 - проприетарная технология). В SSLv2, клиент в самом начале сеанса отправляет на сервер секрет, зашифрованный с использованием открытого RSA-ключа сервера. Для выполнения экспортных ограничений предусмотрен режим, когда часть сеансового секрета передается в открытом виде, и только часть, соответствующая по разрядности требуемому ограничению (40 бит), зашифрована RSA. Современные вычислительные мощности, в случае с анализом "экспортного" SSLv2, позволяют очень быстро перебрать 2^{40} вариантов (на практике, для DROWN, пространство перебора с целью расшифровки сессии TLS будет *другим*, авторы атаки приводят оценку приблизительно в 2^{50} операций, что, впрочем, не является препятствием: на специализированном кластере среднего уровня - вычисления заняли менее суток). Кроме того, формат сообщений SSLv2 предписывает при подготовке сообщения к зашифрованию RSA использовать дополнение определённого формата, особенности обработки этого дополнения на серверной стороне позволяют построить криптографический оракул, делающий возможным расшифрование секрета из соответствующего TLS-соединения.

Атака проводится в несколько этапов. На первом этапе атакующий пассивно записывает TLS-трафик: предмет записи - начало TLS-сессий, использующее статический обмен RSA для передачи сеансового секрета. Именно [TLS-сообщение](#), содержащее Premaster Secret, является целью атаки. Раскрыв информацию из данного сообщения, атакующий может вычислить сеансовые ключи и расшифровать весь записанный трафик сессии. На втором этапе - записанные сообщения с Premaster Secret преобразуются к виду, пригодному для отправки в качестве зашифрованного клиентского ключа в рамках SSLv2-сессии. На третьем этапе атакующий проводит попытки установления соединения с SSLv2-сервером, используя ответы сервера (требуется несколько десятков тысяч запросов) для расшифрования одного из подходящих Premaster Secret. Такое расшифрование состоит из набора шагов, в которых атакующий последовательно, на основе ответов сервера, модифицирует сообщение для следующего запроса.

Шифрование RSA - это всего лишь возведение открытого текста в степень со значением шифрующей экспоненты, выполненное по модулю ключа RSA ($C = m^E \bmod N$ - [алгоритм описан выше](#)). Дополнение, предписываемое используемым

форматом, требует размещения в начале сообщения двух байтов со значениями 00 02. Если результат расшифрования не совпал с данным шаблоном, то сервер может выдать сообщение об ошибке, либо другим способом раскрыть информацию о неудаче. На этом наблюдении и основана исходная атака Блейхенбахера: известное значение двух первых байтов позволяет установить интервал, в котором лежит открытый текст (открытый текст - это число, полученное в результате возведения в степень mod N). Далее, умножая шифротекст на некоторое выбранное значение, воспользовавшись тем, что $(C * K^E) = (m * K)^E \bmod N$, где m - открытый текст, и отправляя результат для расшифрования на сервер, атакующий может последовательно уменьшать интервал возможных значений шифротекста, с тем, чтобы в итоге получить единственное подходящее значение m. Использование в DROWN SSLv2 и экспортных ограничений, требующих короткого шифротекста (40 бит), позволяет существенно оптимизировать алгоритм Блейхенбахера, сократив число запросов к SSLv2-серверу до практически достижимых значений.

TLS и постквантовая криптография

Квантовый компьютер - это устройство, эффективно выполняющее алгоритмы квантовых вычислений. Термин "постквантовые", в контексте криптографии, обозначает стойкость к взлому с использованием квантовых алгоритмов на квантовом компьютере. Предполагается, что универсальный квантовый компьютер позволит реализовать алгоритмы, эффективное выполнение которых недоступно классическим компьютерам. Основные надежды связаны с алгоритмом Шора. Этот алгоритм, как уже рассказано выше, позволяет за обозримое время решать задачи, на сложности которых базируется стойкость и RSA, и ECDSA, и ECDH (и не только этих криптосистем). С прикладной точки зрения, появление квантовых компьютеров достаточной разрядности делает самые распространённые сейчас асимметричные криптосистемы нестойкими. Более того, можно будет расшифровать ранее записанный TLS-трафик, симметричные ключи для которого вычислялись только с использованием упомянутых криптосистем (данная угроза не распространяется на экзотические реализации TLS с PSK, где в качестве одного из источников секрета используется общий симметричный ключ, известный сторонам заранее, при условии, что стороны обменялись этим ключом по защищённому от "квантового перехвата" каналу).

Что касается используемых сейчас в TLS симметричных шифров, то известные квантовые алгоритмы на их стойкость в целом влияют не критически: считается, что удастся получить лишь квадратичный прирост эффективности полного перебора, а это означает, что 256-битный ключ симметричного шифра сохранит 128 бит постквантовой стойкости. 128 бит вполне достаточно. Однако это рассуждение относится только к перебору. Полный перебор универсален и работает против схемы с любым симметричным шифром, кроме абсолютно стойких. Но это вовсе не

означает, что не будут предложены квантовые алгоритмы, специально оптимизированные под конкретный шифр и существенно превосходящие перебор по эффективности. Тем не менее, так как универсального квантового алгоритма, быстро взламывающего базовые симметричные шифры TLS, не известно, то и проблем в части симметричных криптосистем в постквантовую эпоху для TLS (пока) не возникает. Главные проблемы относятся к асимметричным криптосистемам.

В TLS асимметричные криптосистемы используются на двух направлениях: аутентификация сторон соединения (электронная подпись, TLS-сертификаты) и получение общего симметричного секрета (варианты протокола Диффи-Хеллмана). Появление подходящих для практических атак квантовых компьютеров критически влияет на оба этих направления, однако практические механизмы влияния сильно отличаются. Так, взлом той или иной асимметричной криптосистемы электронной подписи позволяет атакующему подменять узлы и проводить активные атаки с перехватом сессий на начальном этапе, то есть, атаки посредника. А вот успешная атака в части симметричного секрета - позволит читать трафик сессий в пассивном режиме. Естественно, если рассматривать ситуацию с точки зрения массового использования, то пассивная атака оказывается гораздо более универсальной и, поэтому, опасной. Более того, как отмечено в разделе, посвящённом выбору параметров криптосистем, возможность успешно атаковать аутентификацию - бесполезна для ранее записанных сессий. Чего нельзя сказать про возможность расшифрования записанного трафика, защищённого симметричным шифром: здесь угроза квантовых алгоритмов гораздо выше и требует упреждающей реакции, поскольку необходимо обеспечить стойкость в течение некоторого времени (пока защищаемые данные не устареют). Поэтому первоочередные меры по защите TLS от "квантовых атак" направлены на внедрение постквантовых криптосистем для получения общего секрета. Если защитить симметричные ключи заранее, то появление квантового компьютера в будущем поставит под угрозу только достаточно старый трафик, который был записан до внедрения постквантовых криптосистем.

Стойкость (предполагаемая) к взлому на квантовом компьютере достигается благодаря тому, что постквантовые криптосистемы строят на математических конструкциях, не содержащих структур, для которых известны эффективные, в смысле атаки, квантовые реализации. Например, алгоритм Шора позволяет взломать RSA потому, что с помощью квантового компьютера возможно отыскать период достаточно простой функции (экспоненты). Знание периода позволяет быстро вычислить разложение модуля ключа на простые множители уже на классическом компьютере. Здесь эффективным квантовым воплощением базовой структуры как раз и является квантовая реализация упомянутой функции, благодаря которой всё пространство значений можно уложить в квантовые регистры и провести квантовое вычисление, определяющее период (на классическом компьютере - пришлось бы проверять значения функции для отдельных элементов, что потребовало бы

слишком много операций). Если же подходящих структур и методов преобразования не известно, то алгоритм считается стойким ко взлому на квантовом компьютере. Обратите внимание, что речи о *доказанном* отсутствии квантовых алгоритмов взлома пока что не идёт (доказывать *отсутствие* алгоритмов вообще непросто).

На настоящий момент (2025) постквантовые криптосистемы в TLS уже внедрены и массово используются в вебе. Было предложено много разных вариантов таких криптосистем, некоторые выбраны в качестве стойких по результатам [конкурса NIST](#).

В 2024 году процесс NIST позволил выбрать ряд алгоритмов в качестве стандартных, а именно: для обмена ключами - ML-KEM (FIPS 203, на этапе отбора криптосистема называлась CRYSTALS-Kyber, или просто Kyber), здесь ML обозначает Module-Lattice (модули и решётки); для подписи - ML-DSA (FIPS 204, основной, на этапе отбора - CRYSTALS-Dilithium, или просто Dilithium) и SLH-DSA (FIPS 205, дополнительный/резервный, на этапе отбора - Sphincs+; SLH это Stateless Hash-Based, то есть, "без записи состояния, на хеш-функциях").

На начальном этапе, постквантовые криптосистемы внедрены в TLS в дополнение к классическим. То есть часть общего симметричного секрета генерируется по постквантовому алгоритму. При этом эффективная разрядность полного секрета возрастает, но классическая часть всё ещё обеспечивает необходимый минимум. Это требуется для того, чтобы не потерять уже имеющуюся стойкость, если постквантовый алгоритм окажется ненадёжным - дело в том, что даже стойкость ко взлому на квантовом компьютере не гарантирует, что для конкретного алгоритма нет эффективных классических атак. Другими словами: классический протокол, например, ECDH, используется для генерирования общего секрета разрядностью в 256 бит, а постквантовый алгоритм - позволяет согласовать ещё 256 бит. Если использование квантового компьютера приводит к раскрытию ключа ECDH, то остаётся ещё 256 бит "постквантового секрета", если же постквантовая система оказалась уязвима для классической атаки, а квантового компьютера ещё нет, то остаётся 256 бит ECDH. Понятно, что в наихудшем варианте, - когда и квантовый компьютер раскрывает секрет ECDH, и постквантовая криптосистема оказалась уязвимой, - сессия будет взломана полностью, но такой вариант менее вероятен, чем ошибки в реализации новых, постквантовых, алгоритмов.

По только что описанной схеме один из криптографических протоколов с постквантовой стойкостью был [добавлен в браузер Chrome](#) уже в августе 2023 года, ещё до стандартизации NIST, как эксперимент. Речь о гибридной схеме передачи (инкапсуляции) секретного ключа X25519Kyber, которая, после стандартизации в 2024 году, превратилась в ML-KEM+X25519, уже без статуса экспериментальной. Именно ML-KEM+X25519 используется в Chrome, Firefox и других современных (2025) браузерах для защиты трафика TLS 1.3. Кроме ML-KEM+X25519 есть и реализации других гибридных вариантов, например, ML-KEM+ECDH/P-256.

Архитектура криптосистемы с постквантовой стойкостью и протокола версии 1.3 позволяет вписать гибридную схему в общий алгоритм установления соединения TLS так, что никаких других элементов затронуто не будет: то есть, в логике сообщений ничего не меняется, весь процесс установления TLS-соединения остаётся таким же, как и без постквантового "гибрида". Новая криптосистема влияет только на некоторые детали шагов получения общего симметричного секрета.

Благодаря поддержке на крупнейших источниках веб-трафика - Cloudflare и Google, - постквантовые гибридные криптосистемы уже в 2024 году защищали заметную часть интернет-трафика, как минимум, 15% (скорее всего - больше).

ML-KEM+X25519 в браузере Chrome (Chromium). Эта постквантовая гибридная схема использует криптосистему X25519 в качестве классической и [ML-KEM \(Kyber\)](#) - в качестве постквантовой. X25519, которая несколько раз упоминается выше, это хорошо известный вариант протокола Диффи-Хеллмана на эллиптической кривой, в данной криптосистеме сразу зафиксирована кривая (Curve25519) и ряд сопутствующих параметров, а открытые значения DH передаются в "упакованном" виде (открытый ключ здесь - точка на кривой, см. "[Приложение А](#)").

ML-KEM (вариант Kyber, ранее, в данном контексте, это Kyber768) - это асимметричная криптосистема, позволяющая реализовать зашифрование и расшифрование. В данном случае схема используется в режиме KEM (Key Encapsulation Mechanism - дословно: механизм инкапсуляции ключа), а верхнеуровневая логика совпадает с RSA-шифрованием. Криптосистема ML-KEM стандартизована NIST в нескольких вариантах, отличающихся оценками стойкости. В гибридной реализации Chrome используется вариант ML-KEM-768 (то есть, условно говоря, стойкость 768 бит), далее этот вариант здесь называется просто "ML-KEM". Гибридная криптосистема ML-KEM+X25519 внесена в реестр IANA, ей присвоен индекс 0x11EC. Криптосистема реализована только для TLS 1.3.

Криптосистема ML-KEM основана на задаче восстановления исходного набора значений (вектора) по набору, элементы которого переданы с "ошибками" (заданными отклонениями); секретным ключом, в каком-то смысле, здесь оказывается алгоритм "помехоустойчивого" кода, который позволяет исправить "ошибку". Концепция и происходит из теории помехоустойчивого кодирования, но при переносе в криптографию - детали поменялись. Базовая задача называется Module-LWE - то есть, "LWE для модулей", где LWE - Learning With Errors, задача восстановления значений по векторам с "ошибкой". По причине разнообразия терминологии и конкретных реализаций задач, данный класс криптосистем в формулировках оказывается сложнее и чем RSA, и чем ECDSA. Однако, чтобы составить общее представление о принципе использования ML-KEM для обмена ключами в TLS, подробное знакомство с математическими основами, как и в случае всё тех же RSA и ECDSA, не требуется.

Для прикладного воплощения теоретической идеи ML-KEM использует кольца многочленов - несколько упрощённо можно сказать, что операции сводятся к умножению и сложению многочленов особого вида. При этом из многочленов строятся матрицы и векторы (откуда, собственно, и появляются "модули" в названии базовой задачи). Такой математический аппарат должен обеспечить стойкость к атакам на гипотетическом квантовом компьютере, поскольку эффективных квантовых алгоритмов взлома на этом направлении пока не найдено. Естественно, предполагается, что схема обладает и классической стойкостью.

Итак, в TLS логика использования ML-KEM совпадает с описанным выше RSA-шифрованием для передачи сеансового секрета, однако стороны меняются местами и открытый ключ здесь присылает клиент. А именно: TLS-клиент передаёт в составе ClientHello (в расширении key_share, см. уточнение ниже) свой открытый ключ ML-KEM, сервер генерирует общий секрет, зашифровывает его ML-KEM, используя клиентский открытый ключ, и отправляет клиенту в составе ответного ServerHello (серверное key_share); клиент, которому известен соответствующий секретный ключ, вычисляет общий секрет на своей стороне. Для получения симметричного секрета используется заданная функция генерирования ключа (KDF), входящая в состав криптосистемы. Эта схема и называется KEM - симметричный ключ здесь как бы скрывается внутри (инкапсулируется) защищённого при помощи асимметричной криптосистемы сообщения (KEM - это просто обобщение для многих способов передачи ключа, в терминах KEM нетрудно переписать и алгоритм Диффи-Хеллмана). Обратите внимание, что сервер вычисляет симметричный сеансовый секрет TLS сразу, вместе с зашифрованным ML-KEM сообщением.

Так как криптосистема гибридная, то в соответствующем блоке key_share, с идентификатором 0x11EC, передаются в виде одного массива два ключа: 1184 байта ключа ML-KEM-768 и 32 байта открытой части DH X25519 (открытый ключ X25519). Значения объединяются простой конкатенацией: к байтам ключа ML-KEM присоединяются (справа) байты ключа X25519. Открытый ключ ML-KEM - это набор из трёх многочленов, где каждый многочлен представлен кортежем своих коэффициентов, а именно - 256 коэффициентов, которые превращаются в 384 байта записи для каждого многочлена; кроме того, в состав ключа входит дополнительный 256-битный параметр - ещё 32 байта ($384 \cdot 3 + 32 = 1184$). Открытый ключ ML-KEM существенно отличается по представлению, например, от RSA, однако очень близок по количеству байтов к длинным RSA-ключам.

На сервере данные разделяются (по длине записи) и используются для обеих криптосистем. А именно: для X25519 сервер вычисляет общий секрет DH и серверную открытую часть DH (B), точно так же, как это делалось бы в случае отдельной криптосистемы X25519; для ML-KEM – сервер генерирует общий секрет и оборачивает его в KEM (то есть, зашифровывает исходное секретное значение, используя открытый ключ, присланный клиентом – тем самые 1184 байта). Два

секрета сервер объединяет в один массив простой конкатенацией (ML-KEM+X25519, в эксперименте с draft-версией - было наоборот: X25519+Kyber768).

Есть важная особенность: для X25519 общий секрет - это результат умножения открытой части DH клиента (A) на секретный скаляр сервера d: $s = d \cdot A$; а для ML-KEM – сервер выбирает исходное значение, которое отправляет клиенту в зашифрованном виде. При этом внутри ML-KEM для вычисления секрета по исходному значению используется отдельная функция (KDF), подмешивающая ещё и значение открытого ключа, это необходимый шаг, но, с точки зрения именно логики получения секрета, он не так важен. Секрет, генерируемый в рамках ML-KEM-768 в TLS – это тоже 32 байта (256 бит). После завершения первого этапа, сервер получает общий симметричный секрет, представляющий собой объединение выдачи двух алгоритмов: 32 байта и ещё 32 байта. Кроме того, сервер получил открытую часть DH и зашифрованный симметричный секрет (это только часть, предназначенная для ML-KEM, результат X25519 сюда не попадает).

Серверное расширение `key_share` формируется похожим на клиентскую часть образом: 32 байта открытой серверной части X25519 присоединяются к байтам шифротекста с симметричным секретом, который зашифрован ML-KEM (длина шифротекста - 1088 байтов, всего 1120 байтов). Ответное `key_share` сервер отправляет клиенту в открытой части сообщений TLS, в `ServerHello`, после чего генерирует на основе общего секрета набор [симметричных сессионных ключей](#) и переходит к зашифрованному обмену данными.

Клиент, получив `key_share` ML-KEM+X25519, разделяет данные на шифротекст ML-KEM и открытую часть обмена DH X25519 (B). По значению B клиент вычисляет общий секрет X25519 (32 байта), который совпадает с серверным. Используя секретный ключ ML-KEM, клиент расшифровывает шифротекст и вычисляет секрет ML-KEM, который передал сервер. Оба полученных значения объединяются, результат должен совпасть с серверным. В ML-KEM есть вероятностный элемент: так как это схема, концептуально происходящая из кодов с коррекцией ошибок, имеется очень и очень малая вероятность, что “ошибка” всё же останется, а клиент и сервер получат разные значения секрета (параметры подобраны так, что, на практике, этой возможностью можно пренебречь). На основе объединённого секрета клиент вычисляет набор симметричных ключей, после чего может проверить подлинность и расшифровать следующие сообщения сервера.

Метаинформация о соединении

TLS, при помощи шифра, скрывает основное содержание коммуникации от стороны, прослушивающей канал. Однако с TLS-соединением связано достаточно много данных, которые позволяют косвенно судить об уникальных характеристиках конкретного сеанса связи и, таким образом, не только отличать один сеанс от

другого, но и извлекать некоторые сведения о закрытой части коммуникации. Информацию, связанную со свойствами TLS-соединения, но не раскрывающую напрямую секретные данные, принято называть "метаинформацией".

Рассмотрим TLS-сеанс между браузером и веб-сервером, соответствующий штатной (без использования дополнительных средств защиты информации) работе пользователя с некоторым веб-интерфейсом. Например, заполнение веб-формы на сайте. Версия протокола - до TLS 1.3. Уже в момент установления соединения, третья сторона, прослушивающая канал, получает информацию о следующих параметрах соединения:

1) Время установления соединения и его продолжительность. Это очевидные параметры, они определяются из состава переданных TLS-сообщений и общих характеристик трафика. Скрыть данную метаинформацию возможно, однако TLS тут никаких инструментов сокрытия не предлагает: протокол проектировался в модели, которая прямо допускает, что прослушивающая сторона может непосредственно видеть пакеты, формирующие весь трафик TLS-сессии. Это, конечно, обусловлено основным предназначением протокола. Дело в том, что вычислительные затраты на сокрытие факта соединения весьма и весьма значительны, а TLS - это протокол для массовых, быстрых интернет-сервисов, которые не готовы тратить ресурсы на качественную стеганографию. Тем не менее, давление современной "информационной реальности" таково, что для TLS разрабатывают решения, позволяющие, фактически, скрыть сведения о факте *полезного* соединения, обернув одно TLS-соединение в другое. Ключевым на этом направлении сейчас является технология ECH, которая [описана выше](#).

2) IP-адрес сервера и IP-адрес клиента. Известны из TCP-соединения, не зависят от TLS напрямую, однако протокол неявно подразумевает, что адреса клиента и сервера TCP-соединения - совпадают с адресами клиента и сервера на уровне TLS; в частности, типовая реализация протокола не предполагает проксирования TLS-данных (однако такое проксирование, опять же, может быть реализовано дополнительными средствами - см. предыдущий пункт). В целом ряде практических случаев утечка сведений об IP-адресе сервера не даёт никакой ценной информации: один и тот же адрес может соответствовать десяткам тысяч различных ресурсов, выбор которых производится по имени. При этом подобное "перемешивание" IP-адресов и имён является эффективным инструментом сокрытия того сервиса, с которым в действительности работает клиентское приложение: при должном подходе IP-адрес вообще может нести лишь информацию о *провайдере соединения*, но не о сервисе.

3) Сведения о том, устанавливал ли браузер соединение с данным TLS-сервером ранее. В TLS предусмотрены механизмы ускоренного установления соединения, а они подразумевают передачу дополнительных "тикетов" в сторону

сервера, эти тикеты передаются в открытом виде. Кроме того, отличается сам алгоритм повторного (ускоренного) соединения, что позволяет узнавать такие сессии. При этом, конечно, ничто не мешает клиенту не использовать ускоренные схемы.

4) Имя сервера. Символьное имя сервера (вида example.com) передаётся браузером в открытом виде на этапе установления соединения. Речь про поле SNI, которое описано выше. Методы защиты от утечки данного имени только разрабатываются, поэтому, в случае с вебом (HTTPS), можно считать, что имя сервера сейчас передаётся повсеместно.

5) Дополнительные имена, связанные с данным сервером. На этапе установления соединения, подразумевающего аутентификацию сервера (а это подавляющее большинство соединений HTTPS), сервер передаёт TLS-сертификат в открытом виде (за исключением TLS 1.3). Сертификат содержит поля с указанием различных имён, для которых данный сертификат валиден. Кроме того, открытый ключ из сертификата позволяет идентифицировать сервер.

6) Выбранные криптографические параметры TLS-соединения. Шифр, алгоритм электронной подписи и алгоритм вычисления кода аутентификации - передаются в открытом виде.

7) Примерный объём переданных данных, примерные действия, выполненные пользователем. Прослушивающая сторона может определить длину передаваемых TLS-записей, штатная работа протокола (до версии 1.3) не подразумевает каких-то эффективных средств маскировки длины сообщений, если они содержат достаточное количество байтов (маскировать длину данных необходимо дополнительно). Например, если пользователь загружал через веб-форму файл существенного объёма, то количество переданных байтов, очевидно, будет отличаться от сценария, когда пользователь просто закрыл веб-форму. Анализ длины переданных записей, порядка их следования, и сопоставление результата с особенностями веб-интерфейса, нередко позволяет точно определить, какие именно действия совершил пользователь.

Метаинформация приобретает особое значение в тех случаях, когда третья сторона может накапливать сведения о большом количестве TLS-соединений. Это относится как к ситуации, когда записывается трафик множества соединений одного и того же клиента (с разворачиванием по времени), так и к ситуации, когда ведётся одновременная запись сессий многих клиентов (разворачивание по клиентам).

Атаки со стороны сервисов

Для большого класса добротных, стойких криптосистем, до сих пор используемых в TLS на практике, возможно восстановление сеансового ключа из записанного трафика, при условии, что атакующая сторона имеет в своём распоряжении

секретный серверный ключ (ключ из пары, открытая часть которой указывается в сертификате сервера). Это означает, что, получив секретный ключ сервера (не сеансовый!), кто-то может расшифровать накопленные ранее записи TLS-сеансов между клиентом и сервером. Упомянутый ключ может быть раскрыт разными способами: например, его можно скопировать с сервера, если есть доступ, или он может просто оказаться нестойким (такое случается не так редко, как можно подумать).

Сгенерированный по протоколу Диффи-Хеллмана сеансовый ключ уже не является долговременным, по решаемым задачам, но также может стать известен злоумышленнику: либо в результате активной атаки на уровне канала (как Logjam), либо в результате получения доступа к серверу, где сеансовый ключ может сохраняться достаточно долго, и не обязательно в защищённом хранилище. Очевидно, что если атакующая сторона получила соответствующий сеансовый ключ, то она может раскрыть TLS-трафик. Секретный серверный ключ или TLS-сертификат для этого не требуются.

Возможность выпустить TLS-сертификат для атакуемого домена сама по себе никак не помогает расшифровать TLS-трафик, идущий в этот домен, даже если трафик записывается. Процедура выпуска сертификата не требует передачи секретного ключа в удостоверяющий центр (передаётся только открытый ключ), поэтому у удостоверяющего центра секретного серверного ключа тоже нет, за исключением распространённого сейчас случая, когда заказчик поручает генерацию такого ключа УЦ (что, впрочем, является административной, а не технической, проблемой).

Однако валидный TLS-сертификат, выпущенный для атакуемого домена, позволяет провести незаметную для пользователя атаку типа "человек посередине". Для этого требуется, чтобы между атакуемым пользователем и сервером существовал управляемый атакующим узел, активно перехватывающий трафик. Этот узел выдаёт себя пользователю за легитимный сервер, предъявляя тот самый валидный сертификат. Пассивное прослушивание канала не позволяет раскрыть TLS-трафик подобным образом – наличие сертификата или секретных ключей УЦ никак тут не помогает.

Перехват TLS-соединения может быть автоматизирован – существуют специальные узлы-прокси (SSL-прокси), которые выполняют такой перехват на лету, в том числе, генерируя нужные сертификаты. В такой прокси должен быть загружен сертификат и секретный ключ, позволяющие подписывать другие сертификаты (например, годится промежуточный сертификат УЦ, выпущенный для этих целей и соответствующий ему секретный ключ). Такой "человек посередине" не работает при наличии некоторых дополнительных мер: например, установление TLS-соединения требует взаимной аутентификации, и перехватывающему узлу недоступен клиентский секретный ключ (либо ключи УЦ, удостоверяющего клиентский ключ); или – пользовательский

браузер ведёт реестр отпечатков открытых ключей сервера, которым он доверяет; или – пользователь применяет дополнительные источники сведений о разрешённых ключах и сертификатах, которые недоступны для подмены на перехватывающем узле (таким источником может служить DNS, либо другая база данных).

На практике, для массовых сервисов, возникают другие технические направления атак, напрямую не связанные с какими-то особенностями TLS.

Так, массовые онлайн-сервисы используют TLS/SSL termination: то есть, пользовательский TLS-трафик в зашифрованном виде доходит только до пограничного прокси, где благополучно транслируется в открытый протокол, обычно это HTTP, соответствующий HTTPS, который дальше ходит по внутренним (в логическом, а не техническом смысле) сетям сервиса в открытом виде. Тотальный HTTPS, с ростом числа клиентов, быстро превращается в неподъёмную, плохо масштабируемую технологию, поэтому TLS/SSL termination является весьма распространённым решением. Если система инспекции трафика (DPI) находится внутри сетей сервиса, за таким пограничным прокси, то никакой TLS ей не мешает. Именно так получают доступ к данным платёжных систем и веб-почты специальные службы, располагающие, на законных основаниях, собственным оборудованием DPI за пограничными прокси.

Сеансовые ключи могут экспортироваться сервером наружу, в другие системы, которые, например, осуществляют балансировку нагрузки или "очистку" трафика. В некоторых случаях вообще используются услуги третьей стороны для организации TLS-соединения. Кроме того, внутренний трафик распределённых сервисов с лёгкостью ходит между узлами и дата-центрами по арендованным у крупных операторов каналам связи в открытом виде, такой трафик может прослушиваться, хотя для пользователя он выглядит как TLS-соединение.

Иными словами: TLS не предоставляет абсолютной защиты, но в подавляющем большинстве сценариев использования Интернета этот протокол делает обмен информацией хорошо защищённым; для повышения скрытности и безопасности, если такое требуется, TLS должен использоваться совместно с другими решениями.

Приложение А. Конечные поля и эллиптические кривые в криптографии

В этом приложении в вольной форме рассмотрены элементарные математические основы, на которых строятся эллиптические криптосистемы, упоминающиеся в основном тексте.

Для начала потребуется базовый инструментарий, прежде всего - *конечное* множество с дополнительной структурой, которая определяется парой операций над элементами множества, обладающих свойствами "обычных" сложения и умножения. Мы собираемся решать в этом множестве некоторые уравнения, поэтому важно, чтобы результат применения операций не выходил за пределы данного множества. Не вдаваясь в алгебраические детали, достаточно вспомнить про деление с остатком - оно как раз позволяет построить нужную структуру. Для примера, будем делить на 17: всякое натуральное число можно отождествить с остатком от его деления на 17. У нас получится множество из семнадцати элементов, соответствующих числам от 0 до 16. Можно сказать, что здесь сохранились привычные операции сложения и умножения. Это легко проверяется: $11 + 13 = 7$, где 7 - остаток, получившийся при делении 24 на 17; $6 * 9 = 3$, так как $54 = 3 * 17 + 3$. В нашей структуре можно вычитать и делить, то есть, по-научному, она образует поле, однако и вычитание, и деление - несколько отличаются от "обычных"; эти детали мы сейчас не станем рассматривать, но полем всё же получившийся объект называть будем. Точнее - конечным полем, потому что в нём конечное количество элементов.

Описанный только что пример вовсе не является каким-то чрезмерным упрощением: конкретные реализации ECDSA именно с такими полями ("остатками от деления") и работают, но в качестве основы берётся не 17 или 19, а очень большое простое число (это число, задающее поле, называют модулем; чтобы "по остаткам" получилось именно поле, модуль обязательно должен быть простым числом; в криптографии используются и другие поля, практические воплощения которых строятся "на многочленах", здесь самое заметное отличие будет в количестве элементов; мы, опять же, пропускаем эти детали, иначе можно уйти далеко в сторону).

Для нашей задачи принципиально важен только один момент: можно простым способом, позволяющим проводить эффективные вычисления, построить *конечное* множество, на котором действуют привычные операции сложения и умножения, пусть и с некоторыми ограничениями. Конечность здесь имеет прикладное значение - мы собираемся "считать на компьютере" и нам потребуется эффективный алгоритм, позволяющий за *конечное* число шагов построить любой элемент нашего множества из других его элементов. Вот теперь можно переходить к кривым.

Рассмотрим *формулу* $2x + 3 = y$. Можно ли найти в нашем множестве остатков от деления на 17, с введённой структурой сложения и умножения, пары элементов (x, y) , удовлетворяющие этой формуле? Да, это легко сделать: 1 и 5, 3 и 9, 10 и 6. *Формулу* можно выбрать посложнее. Эллиптической кривой соответствует $y^2 = x^3 + ax + b$, где a, b - коэффициенты, которые определяют конкретный экземпляр кривой; a и b - элементы выбранного *поля*, на которые накладываются дополнительные ограничения, которые мы пропустим. В формуле используется возведение в степень.

Под этой операцией понимается последовательное умножение элемента на себя, то есть ничего нового здесь не написано, это всё привычные операции, но результат берётся по модулю некоторого (простого) числа.

Теперь, взяв подходящую (см. выше: $y^2 = x^3 + ax + b$) формулу, мы можем выбрать в нашем множестве со структурой *пары* элементов, соответствующие формуле. Множество таких пар (если оно не пустое) и будет в нашем случае эллиптической кривой, каким бы странным это ни казалось. Да, это не совсем кривая в "бытовом" понимании, но это не важно: в контексте рассматриваемой криптосистемы важны именно пары значений, задаваемые *уравнением* кривой (мы называем его просто формулой). Обычно, таких пар в правильно выбранном поле более чем достаточно. Пары элементов (x, y) называют точками эллиптической кривой над конечным полем. Это терминологическое соглашение. Конечно, за соглашением кроется глубоко теоретическая математика, но она, тем не менее, напрямую в прикладной криптографии не используется, в том смысле, что вовсе не обязательно представлять себе эллиптическую кривую над конечным полем как некую "линию" (но можно попробовать, сделав тем самым шаг на территорию геометрии над конечными полями; вообще, с точки зрения чистой математики, эллиптическая кривая это объект гораздо более общий, чем обозначено в нашем изложении, да и строится она другими способами).

Пусть у нас есть ещё пара *формул*, которые позволяют по *кортежу* значений, состоящему из двух пар (x, y) и (u, v) , вычислить другую пару (k, l) . Важное свойство: новая пара (k, l) тоже удовлетворяет уравнению кривой. То есть, мы берём две точки, вспоминаем, что каждая из них - это пара элементов поля, связанных между собой уравнением кривой, и применяем к этим элементам формулу, которая сопоставляет двум элементам - третий. Обозначим пары элементов буквами **Q** и **P**, $(x, y) = \mathbf{Q}$, $(u, v) = \mathbf{P}$. Примем, что упомянутые формулы - это функции $F_1(\mathbf{Q}, \mathbf{P})$ и $F_2(\mathbf{Q}, \mathbf{P})$ от пар элементов нашего множества (поля). Тогда, если у нас на входе две точки $(x, y) = \mathbf{Q}$ и $(u, v) = \mathbf{P}$, то, применив функции, мы получаем третью точку $(F_1(\mathbf{Q}, \mathbf{P}), F_2(\mathbf{Q}, \mathbf{P}))$, то есть (k, l) , где k, l есть значения, элементы поля, вычисляемые по некоторым формулам. Ключевое свойство этих формул в том, что получившаяся пара элементов (k, l) удовлетворяет нашему уравнению кривой.

Только что описанные формулы переводят точки кривой в точки кривой, то есть задают некоторую операцию на множестве этих точек (или, эквивалентно, на выбранных парах элементов нашего поля). Обозначим эту операцию символом " $\circ$ ". Так как у нас пары элементов, при этом элементы пары могут быть равны, будет удобным рассматривать большее множество, являющееся произведением нашего поля на себя. То есть, один экземпляр поля - для записи левого элемента пары, один - для записи правого. Это, вообще говоря, лишь удобное обобщение: ничто не

мешает оставаться с одним экземпляром поля, но тогда нужно учитывать возможную "кратность" - например, точка $(0, 0)$ содержит один и тот же нуль *дважды*.

Итак, внутри нашего множества со структурой появилось подмножество пар элементов, соответствующих точкам кривой, и операция на этих точках. Операция $Q \circ P = R$ задаёт (k, l) - третью пару элементов, или третью точку, соответствующую двум другим. И k , и l - элементы нашего поля. Формулы, преобразующие элементы, должны быть устроены таким образом, чтобы соответствующая им операция на точках кривой обладала определёнными свойствами. Например, она коммутативна и ассоциативна; более того, множество пар с данной операцией образует *группу*, то есть операция однозначна, существует "нулевая", нейтральная точка, которая описана ниже, и обратные элементы (если подходить совсем строго, то для получения обратного элемента нужна вторая операция, но она у нас тоже есть). Именно поэтому говорят о группе точек эллиптической кривой. Применение операции к парам элементов, соответствующих точкам кривой, не выводит нас за пределы подмножества пар, которые мы называли точками. Данную операцию принято называть сложением точек, но это, опять же, всего лишь удобное терминологическое соглашение. (Заметьте, что всякая эллиптическая кривая образует такую группу точек - это базовое свойство эллиптических кривых.)

Итак, мы использовали пары элементов поля для построения группы точек эллиптической кривой над конечным полем. То есть, в основе этой группы - два экземпляра поля (потому что у нас пары значений).

Операцию сложения точек можно применять к одной и той же точке. (Строго говоря, сами формулы, задающие сложение, при этом *могут* быть разными, то есть, различаются формулы для двух различных точек и формулы для сложения одной точки с самой собой, для удвоения точки. Впрочем, можно так подобрать структуры, что формулы окажутся одинаковыми, но это технические детали.) Итак, можно складывать точку саму с собой. Вспомним, что наше исходное множество - конечное. Поэтому множество пар элементов, соответствующих структуре точек эллиптической кривой, тоже конечно. Это означает, что на каком-то шаге последовательное сложение точки с самой собой даст в результате эту же точку (мы как бы вернёмся в начало; можно было бы предположить, что "зацикливание" произойдёт на какой-то другой точке, а не на начальной, но это не так, ввиду свойств операции сложения точек).

Данное наблюдение непосредственно связано с особой "нулевой" точкой O , главное свойство которой совпадает со свойствами привычного нуля по сложению - сумма любой точки P с "нулевой" даёт эту же точку P : $P \circ O = O \circ P = P$. Возникает вопрос о координатах, то есть, о паре элементов поля, которые соответствуют точке O . Чтобы не усложнять изложение, примем, что "нулевой" точке соответствует *пустая* пара элементов нашего множества. Точки, которые в сумме дают точку O - называют

обратными (аналогом этого, например, для целых чисел по сложению, является число A и его "минус", то есть $-A$, так как $A + (-A) = 0$). Если говорить о парах элементов в практических применениях, то для взаимно обратных точек, сумма которых даст O , один элемент будет одинаковым, а два других - обратными. Например: (x, y) и $(x, -y)$. Если последовательно складывать какую-либо точку с собой, то в какой-то момент обязательно получим точку O .

Только что описанная структура является чисто алгебраическим воплощением метода секущих и касательных, которым обычно геометрически иллюстрируют механизм сложения точек на эллиптической кривой, в её вещественной интерпретации: секущая проходит через две точки, которые суммируются, а касательная - используется при удвоении точки. Очень известная картинка. При этом на иллюстрациях рисуют кривую, обладающую нужной непрерывностью. Уравнения секущих и касательных служат основой для построения формул сложения точек. Элементы, входящие в пары, называют координатами.

Вернёмся к сложению точки с ней самой. $P \circ P$ можно переписать как $[2]P$, отразив тем самым, что мы сложили две "копии" точки. Тогда $[5]P = P \circ P \circ P \circ P \circ P$, и так далее. Запись очень похожа на умножение, но это не умножение *точек* ("умножения" для точек у нас вообще нет - мы его не определили), поэтому говорят об умножении точки на скаляр. Скаляром здесь называют целое число, соответствующее количеству "копий" точки. Умножение на скаляр ($[n]P$) имеет свои свойства. Например, в записи $[5]P$ можно представить число 5 в виде суммы $3+2$, получим $[(3+2)]P$ - такая запись соответствует тому, что в "разложение" суммы точек как бы добавили скобки, вот так: $(P \circ P \circ P) \circ (P \circ P)$, или $[3]P \circ [2]P$. Здесь необходимо понимать, что используются два различных "сложения": сложение в целых числах "+", и сложение точек кривой (" $\circ$ "). (В алгебре такая структура называется модулем, да, тоже модулем - см. первый абзац, - и встречается сплошь и рядом, в том числе, под другими названиями.)

По свойствам, умножение на скаляр совпадает с возведением в степень, операции отличаются только способом записи. Именно поэтому в описаниях эллиптических криптосистем фигурируют "логарифмы" - так сложилось исторически. Возможность переписать $P \circ P \circ P \circ P$ как $[2]P \circ [2]P$, то есть, как удвоение точки $[2]P$ - даёт необходимый механизм, позволяющий быстрее умножать точки на скаляр, если известно значение скаляра. Только благодаря этому и работают на практике криптосистемы, основанные на дискретном логарифмировании в группе точек эллиптической кривой. Это весьма общее свойство, которое необходимо для построения многих других практически полезных криптосистем. Только что описанное умножение точки на скаляр - прямо используется в протоколе Диффи-Хеллмана на эллиптических кривых (см. [основной текст](#)). Стойкость протокола основана на том, что вычислительно сложной является обратная задача: по известным точкам $Q = [n]G$, найти n .

Приложение Б. Применение TLS для защиты других протоколов

Среди самых распространённых в Интернете сценариев использования TLS находится HTTPS - защищённая версия протокола HTTP, которая повсеместно применяется для работы с веб-сайтами, а с некоторых пор и в целом ряде других случаев. Например, в 2018 году статус [RFC 8484](#) получил протокол DNS-over-HTTPS (DoH), который, в полном соответствии с названием, позволяет отправлять запросы и получать ответы DNS через HTTPS.

DNS - система доменных имён - это фундаментальная часть Интернета, представляющая собой распределённую базу данных, которая хранит пары "ключ-значение" и предоставляет механизм извлечения этих записей. Самый простой случай использования DNS - это поиск IP-адреса, соответствующего доменному имени: ключом здесь является доменное имя (например, example.com), а значением - IP-адрес (например, 192.168.0.3). Мы не рассматриваем подробности устройства DNS в этом тексте, а остановимся только на способах применения TLS для защиты информации, передаваемой при работе с доменной системой имён.

Извлечение информации из DNS инициирует DNS-клиент. Например, это может быть служба доменных имён операционной системы (либо встроенный в прикладные библиотеки сервис). Обычно, системная служба является самой простой из возможных: получив запрос от приложения, эта служба либо отвечает данными из локальных таблиц, либо просто переправляет запрос внешнему сервису поиска (который называется "рекурсивным резолвером") и ожидает ответ от него; основная особенность здесь в том, что рекурсивный опрос служба самостоятельно не выполняет. Внешний рекурсивный резолвер, осуществляющий поиск в DNS, традиционно предоставляется провайдером интернет-доступа. Кроме того, распространение получили глобальные общедоступные DNS-сервисы, например 8.8.8.8 (Google) и 1.1.1.1 (Cloudflare).

В классическом варианте DNS - вся информация передаётся в открытом виде. Именно обмен информацией между DNS-клиентом и внешним рекурсивным резолвером защищается при помощи DNS-over-HTTPS. Эта технология предполагает, что запросы к DNS-серверу направляются по HTTPS, фактически аналогично тому, как это происходит в случае веб-узлов. Так как HTTPS использует TLS для защиты информации, запросы и ответы оказываются скрыты от прослушивания третьей стороной.

Применение TLS для защиты DNS позволяет, во-первых, провести аутентификацию DNS-сервера, во-вторых, защитить адресную информацию от подмены. Аутентификация может быть выполнена штатным для TLS способом, при помощи TLS-сертификатов. Защита от подмены является основным свойством протокола TLS. Таким образом, TLS переводит использование DNS на новый уровень, решая

ряд проблем безопасности. Например, в DNS не предусмотрено никаких механизмов, позволяющих убедиться, что на запросы по некоторому IP-адресу отвечает именно тот DNS-сервер, который предполагается (ни IP, ни UDP, ни TCP не предполагают механизмов проверки подлинности узлов). Закрытие содержания DNS-трафика от прослушивания при помощи шифра позволяет перекрыть канал утечки сведений о веб-узлах и других интернет-ресурсах, используемых клиентом. Не менее важной оказывается и защита от подмены: дело в том, что обычные DNS-запросы и DNS-ответы могут быть перехвачены транзитными узлами, а передаваемая в них информация - может быть изменена, например заменён IP-адрес, при сохранении прочих свойств пакета. Подобное изменение информации приведёт к тому, что соединение будет установлено не с тем узлом, которому в действительности соответствует DNS-имя.

Очевидно, что для DoH требуется поддержка на стороне DNS-сервера. Такая поддержка уже доступна во многих серверных приложениях. На стороне клиента - DNS-over-HTTPS уже поддерживается браузерами Mozilla Firefox, Chrome, Microsoft Edge и др. Случай с браузерами является показательным: обычно, прикладные программы используют для работы с DNS системные средства или специальные стандартные библиотеки, однако для DoH работа с DNS осуществляется встроенными инструментами самого браузера.

DoH если и выглядит разумной технологией, то только для защиты канала между прикладной программой и внешним сервисом DNS-резолвера. При этом сама доменная система имён содержит и не менее важное второе плечо: это обмен информацией между резолвером и серверами DNS, осуществляющими поддержку доменных зон разного уровня (рекурсивный поиск). Существует другой вариант протокола, защищающего DNS при помощи TLS: он так и называется - DNS-over-TLS ([RFC 7858](#), DoT). Этот вариант отличается от DoH тем, что здесь TLS используется "напрямую", то есть без дополнительной внутренней обёртки в виде HTTP. DoT предлагает все те же возможности защиты, что и DNS-over-HTTPS, но не требует реализации обработки HTTP-запросов и ответов: DNS-трафик непосредственно передаётся внутри защищённых TLS-записей, позволяя, фактически, надстроить над обычным DNS-сервером TLS-прокси. По этой причине DoT гораздо лучше подходит для обмена информацией между DNS-серверами при рекурсивном поиске. Понятно, что DoT может быть эффективно использован вместо DoH и для защиты запросов между прикладным DNS-клиентом и DNS-резолвером.

(Отдельно нужно отметить, что в DNS давно существует механизм криптографической защиты адресной информации - DNSSEC. Этот механизм позволяет удостовериться подлинность информации, полученной из DNS, при помощи электронной подписи. При этом DNSSEC не подразумевает аутентификации узлов, участвующих в обмене данными, но в соответствующей модели угроз этого и не требуется: если подпись на данных валидна, то не имеет значения, каким способом

эти данные получены - из доверенного и аутентифицированного источника или нет. Поэтому использование TLS в DNS не отменяет DNSSEC, а лишь служит дополнением, защищающим каналы передачи данных.)

Приложение В. TLS и криптосистемы ГОСТ (ГОСТ-TLS)

Современный перечень российских криптосистем, определённых в серии ГОСТ 34.10, 34.11, 34.12, включает два шифра, хеш-функцию и электронную подпись. Этих элементов достаточно для того, чтобы построить вариант протокола TLS, использующий только российскую криптографию. Такие протоколы описаны и используются. Их принято называть ГОСТ-TLS. Помимо внутренних российских спецификаций, практически все элементы описаны в соответствующих RFC, например: [RFC 9189](#), [RFC 9058](#), [RFC 7801](#), [RFC 8891](#), [RFC 7091](#), [GOST with TLS 1.3 \(RFC 9367\)](#) и др.

Сравним основные криптографические подсистемы "обычного" TLS и ГОСТ-TLS.

1. **Хеш-функция.** Пример для TLS: SHA-256. В ГОСТ-TLS используется сходная по свойствам и параметрам хеш-функция "Стрибог" из современной ГОСТ-серии 34.11.
2. **Электронная подпись.** Пример для TLS: ECDSA. В ГОСТ-TLS - алгоритм из ГОСТ-серии 34.10. ГОСТ-подпись (современной версии) работает на том же математическом аппарате, что и ECDSA, то есть, тоже использует группу точек эллиптической кривой.
3. **Шифр.** Пример для TLS: AES. В ГОСТ-TLS используется либо шифр "Магма", ставший уже классическим, поскольку ведёт свою историю из 70-х годов прошлого века, либо более современный "Кузнечик", который является AES-подобным шифром, то есть, схож с AES по основным принципам построения. Оба шифра определены в ГОСТ-серии 34.12.
4. **Режим шифрования.** Пример для TLS: GCM. В ГОСТ-TLS используется AEAD-режим MGM.
5. **Обмен симметричными ключами.** Пример для TLS: ECDH. В ГОСТ-TLS (версии 1.3) используется тоже ECDH, но с собственными параметрами и алгоритмами получения ключей из секрета.

Для ГОСТ-TLS выпускаются TLS-сертификаты, использующие ГОСТ-подпись. Эти сертификаты соответствуют по формату сертификатам TLS, описанным выше. Отличия касаются представления ключей и идентификаторов алгоритмов.